GRAINS: Enabling High-Performance and Low-Cost Graph-Based Genome Analysis via Storage-Aware Algorithm-Architecture Co-Design

Nika Mansouri Ghiasi[1] Harun Mustafa[2,1] Talu Güloglu[1] Rakesh Nadig[1]
Konstantina Koliogeorgi[1] Susana Rebolledo Ruiz[1,3] Marc Rautmann[1] Furkan Eris[1]
Mohammad Sadrosadati[1] Jisung Park[4] Onur Mutlu[1]

[1]ETH Zürich [2]Johns Hopkins University [3]University of Cantabria [4]POSTECH

2026 ACM/IEEE 53rd Annual International Symposium on Computer Architecture (ISCA) | 979-8-3315-5065-3/26/$31.00 ©2026 IEEE | DOI: 10.1109/ISCA66397.2026.00066

Graph-based representations of genome sequences have emerged as a powerful approach for representing massive genomic databases in an expressive and efficient way. Compared to traditional, linear genome sequences, genome graphs enable more accurate and efficient genome analyses, particularly in complex, population-scale settings (e.g., public health, precision medicine, and agriculture). Despite their benefits, analysis on large-scale genome graphs incurs significant data movement overhead from the storage system due to accessing large amounts of low-reuse data. Processing data directly inside the storage device, where data originally resides, can be a fundamental solution for mitigating this overhead. However, none of the existing tools for graph-based genome analysis can be efficiently used inside the storage system due to the limited internal hardware resources in modern SSDs. At the same time, prior storage-centric systems developed for (i) traditional, linear non-graph-based genome analysis or (ii) conventional, non-genomic graph analysis are not suitable for the unique data structures and access patterns of graph-based genome analysis.

We propose GRAINS, the first *system for analysis with large-scale **g**enome g**ra**phs **in** **s**torage. Through our detailed examination of typical analysis pipelines that operate on genome graphs, we perform storage-aware algorithm-architecture co-design to (i) make the graph-based genome analysis pipelines more storage-friendly and (ii) further improve performance, energy-efficiency, and cost via in-storage and in-flash processing. GRAINS's co-design is based on three key aspects. First, we propose a new batching technique and execution flow, based on unique features of genome graphs, that reduces the number of random accesses to graph nodes. Second, via in-flash and in-storage processing, we avoid transferring low-reuse or unused flash pages, preventing SSD channel and external I/O bandwidth waste. Third, to leverage the full parallelism of flash dies during in-flash processing, we design an effective, yet lightweight, scheduling technique, enabled by re-purposing the existing SSD structures. GRAINS's design is versatile and flexible as it supports key operations on genome graphs and can be integrated in various analysis pipelines. GRAINS provides 2.7×–47.8× speedup (4.4×–31.6× energy reduction) over the state-of-the-art software baselines, and 1.5×–17.0× speedup (3.1×–20.7× energy reduction) over a hardware-accelerated baseline.*

1. Introduction

Genome sequence analysis, which examines the genomic information of living organisms and other biological entities, is fundamental to many critical fields, including personalized medicine [1–28], disease outbreak tracing [29–54], cancer research [55–101], maintenance of food resource safety [102,103], agriculture [104–119], scientific discovery [120–122], biodiversity conservation [123,124], evolutionary biology [125–142], and antimicrobial resistance surveillance [143–145]. To enable the computational analysis of an organism's genomic information (typically encoded in DNA or RNA that has been reverse-transcribed into DNA [146,147]), a process called *sequencing* converts the information of DNA molecules in the organism's biological sample to digital data. Since current sequencing technologies cannot process a DNA molecule as a whole, a sequencer generates randomly sampled, inexact fragments of genomic information, called *reads*. Common genome analysis workflows then involve querying reads from a sample against a single *reference* genome or large-scale databases (e.g., containing reference genomes from many species or previously sequenced samples) to determine species or genomic features present in the sample [148–153] and/or to find their differences from genomes in the database [154,155]. These queries either involve k-mer set lookups (where subsequences of length k, i.e., k-mers, in a read set are matched against the database) or read mapping (where each read is analyzed individually). The importance of genome analysis, along with rapid improvements in sequencing technology (i.e., reduced costs and increased throughput [156,157]), has led to its increasing adoption [22–24,26–28,49,54]. This growth has, in turn, motivated a large body of work (e.g., [29,116,158–257]) on accelerating these analyses to keep pace with rapidly growing sequencing throughput and data volumes.

Genome Graphs have emerged as a powerful approach for representing and querying massive and complex genomic databases [148,151,258–264]. Unlike traditional models, which represent genomic databases as a collection of disjoint, linear sequences, sequences in a graph are represented by graph walks [259]. Genome graphs provide two fundamental benefits, making them indispensable, particularly in modern, population-scale genomics [143,148,265–267]. First, the graph topology, along with its associated metadata[1] offer vastly *greater expressive power* [266–269]. A graph naturally encodes the evolutionary history and diversity of the organisms in a database, revealing their shared and distinct se-

[1]Metadata can include the original species or samples of a graph node's sequence [258,259], associations with patient outcomes [148], and more.

979-8-3315-5065-3/26/$31.00 ©2026 IEEE
DOI 10.1109/ISCA66397.2026.00066

quences [263,266,268,270]. This leads to reduced bias and improved analysis accuracy. For example, genome graphs improve disease diagnosis by capturing population-specific variants often missed by traditional, linear sequences [155,266]. Second, genome graphs leverage the inherent redundancy of genomic data to *avoid redundant computation* on shared sequences. This is because shared genomic regions across database entries are stored only once and do not need to be queried separately. This is essential in settings like population-scale pathogen surveillance, where public health systems must rapidly match new pathogen genomes against large national databases [143,148,264,271–273].

Data Movement Overhead. Due to the significance of graph-based genome analysis and its computational challenges, many works (e.g., [231–257]) propose techniques to alleviate these challenges by mitigating computational and main memory bottlenecks. However, to our knowledge, none of them mitigates the I/O cost of reading large amounts of graph data from the storage system to main memory and computation units. Our motivational analysis (§3) shows that the end-to-end performance of analysis with large-scale genome graphs suffers significantly from the data movement overhead of accessing large amounts of low-reuse data (exacerbated by reduced access locality due to the graph structure).

The I/O overhead of moving large amounts of low-reuse genome graph data is hard to avoid. One might think it is possible to eliminate this overhead by (*i*) shrinking graphs through sampling (e.g., [274–279]) or (*ii*) maintaining all required graphs completely and always in main memory. However, neither solution is suitable. The first approach necessarily reduces accuracy [156], making it inapplicable for many use cases (e.g., [156,160,280–287]). The second approach is not suitable since (*i*) genome graph databases, which are already large (tens to hundreds of terabytes [148,288]), grow in size rapidly due to increasing demands in biomedicine [22–24,26–28,49,54] and the exponentially decreasing cost of sequencing [289], and (*ii*) regardless of individual graph sizes, different analyses need to access different graphs. Therefore, it is energy-inefficient, unsustainable, costly, and unscalable to maintain *all* data required for *all possible* analyses in DRAM at all times.

Storage-centric computing (SCC) refers to processing data inside the storage device, either on the SSD[2] controller (in-storage processing, i.e., ISP) or on the flash dies (in-flash processing, i.e., IFP). SCC can be a fundamental solution for alleviating data movement overheads in graph-based genome analysis by processing data where it originally resides. This is due to three reasons. First, SCC reduces unnecessary data movement between the storage system and compute units by handling large amounts of low-reuse data within the storage system. Second, SCC alleviates the overall execution burden of moving and analyzing low-reuse data from the rest of the system (e.g., processing units and main memory), such that these components can be used for other purposes or be turned off to save energy. Third, SCC exploits the large internal bandwidth of the SSD, which is particularly advantageous when computation is performed within the flash dies.

[2]In this work, we focus on the predominant NAND flash-based SSDs [290,291]. We expect that our designs and insights would benefit storage systems with other emerging technologies (e.g., [292–295]) as well.

Limitations of Prior Work. To our knowledge, no prior work alleviates the I/O overheads of graph-based genome analysis. Some works (e.g., [231–244]) accelerate graph-based genome analysis by alleviating its computation or main memory bottlenecks. However, they do not alleviate I/O overheads, whose impact on end-to-end performance gets even larger as other overheads are alleviated. Several works propose SCC to alleviate the I/O overheads of (*i*) *non-genomic* graph analysis (e.g., [296–312]) or (*ii*) genome analysis on conventional, *linear sequences* (e.g., [204,241,313–318]). However, as detailed in §13, they are not directly applicable to the unique structures, access patterns, and operations on genome graphs.

Challenges. Despite the benefits of SCC, no existing tool for graph-based genome analysis can be efficiently implemented in the SSD due to the limitations of hardware resources in modern SSDs. This is because graph-based genome analyses involve extensive random, irregular accesses to genome graphs, causing costly contention in internal SSD components (e.g., channels and NAND flash dies [319–321]). This hinders leveraging the SSD's internal bandwidth, thereby diminishing the benefits of SCC.

Our goal in this work is to improve the performance and energy efficiency of graph-based genome analysis by reducing the large data movement overhead from the storage system in a low-cost way. To this end, we propose GRAINS, the *first* system for analysis with large-scale **g**enome g**ra**phs **in** storage. GRAINS is versatile and flexible as it supports major operations on genome graphs (e.g., k-mer set lookup and read mapping) and can be integrated into various analysis pipelines. Through our detailed examination of typical graph-based genome analysis pipelines, we perform storage-aware algorithm-architecture co-design to (*i*) make the graph-based genome analysis pipelines more storage-friendly and (*ii*) further improve performance, energy-efficiency, and cost-effectiveness via SCC.

Key Mechanism. GRAINS's algorithm-architecture co-design is based on three key aspects. First, we design an efficient batching technique and pipelined execution flow specialized to the properties of genome graphs and queries to reduce random accesses to the genome graph structure. Second, we devise lightweight hardware units in the flash dies to process the needed parts of each page's graph data. This simple IFP design avoids unnecessarily transferring low-reuse or unused parts of flash pages outside the flash die, preventing SSD channel and external I/O bandwidth waste. Third, to fully exploit die-level parallelism in IFP, we design an effective, yet lightweight, scheduling technique, enabled by repurposing existing SSD structures. GRAINS's data layout drastically reduces the required in-DRAM mapping metadata, allowing us to use the freed-up internal DRAM space to maintain small per-die scheduling tables. We devise small ISP units on the SSD controller to interface with these in-DRAM tables and schedule IFP operations at low cost. To efficiently realize this SCC design, while guaranteeing correctness and integrity, we apply simple changes to the flash translation layer (FTL) and adopt a lightweight error correction scheme [322].

Key Results. On systems with two state-of-the-art SSD configurations, we compare GRAINS against state-of-the-art software and hardware tools for graph-based genome analysis: (*i*) Fulgor: a node-centric tool [151], (*ii*) the MetaGraph framework: an edge-centric tool [148,260–262], and (*iii*) an *idealized* hardware-accelerated tool without main memory overheads (IdealAccMem) to query the graph. We show that when performing k-mer set lookups and alignment-free read mapping (two core operations in large-scale genome graph analyses), GRAINS provides 6.8× (11.2×), 10.7× (21.5×), and 5.6× (9.8×) average speedup (energy reduction) over Fulgor, MetaGraph, and IdealAccMem, respectively. GRAINS's workflow also seamlessly supports alignment-based mapping, where the candidate graph regions identified by GRAINS are fed to an alignment engine. We integrate GRAINS, Fulgor, and MetaGraph with SeGraM, a state-of-the-art sequence-to-graph alignment accelerator [231]. SeGraM+GRAINS performs 6.2× and 9.0× better than SeGraM+Fulgor and SeGraM+MetaGraph.

This work makes the following **key contributions**:

- We demonstrate the impact of I/O overheads on the end-to-end performance of graph-based genome analysis.
- We introduce GRAINS, the first system for analysis with large-scale genome graphs in storage.
- We perform storage-aware algorithm-architecture co-design to make graph-based genome analysis more storage-friendly, and further improve performance, efficiency, and cost-effectiveness via storage-centric computing (SCC).
- We show that GRAINS improves performance and energy efficiency over the state-of-the-art graph-based genome analysis tools and accelerators, without relying on costly resources throughout the system, thereby enabling wider adoption of graph-based genome analysis.

2. Background

2.1. Genome Sequence Analysis

Given a genomic sample, a typical workflow consists of three key steps: (*i*) sequencing, (*ii*) basecalling, and (*iii*) analysis. The first step, *sequencing*, reads and converts the sample's DNA[3] to digital signals. Modern sequencers cannot read long DNA molecules as complete sequences. Instead, they produce many shorter, overlapping sequences called *reads*. The second step, *basecalling*, converts each raw read signal into a string, encoded using `A`, `C`, `G`, `T` [323]. The third step is *genome analysis* on a set of reads (i.e., a *read set*). Common genome analysis workflows then involve querying reads from samples against a single *reference* genome or large-scale databases (e.g., containing reference genomes from many species or previously sequenced samples) to determine species or genomic features present in the sample [148–153] and/or to find their differences from genomes in the database [154,155].

In this work, we focus on the analysis performance for two reasons. First, sequencing and basecalling are typically performed once per read set, whereas analysis of the same read set can be performed *many times* or at *different times*. For example, read sets from prior disease-association studies are commonly reanalyzed as human or microbial genome databases improve [154,267,324–326] or expand [264,267], or when new personalized databases can be tailored to the study cohort [266,327]. Second, modern sequencing throughput far outpaces Moore's law [156,289,328], which poses significant computational challenges for analysis on conventional systems. This is why genome analysis tasks have been a key acceleration target in a large body of works (e.g., [29,116,158–230]).

[3]Even if an organism's genome is RNA-based, it is typically converted to DNA before being sequenced [146,147].

2.2. Graph-Based Genome Analysis

Genome Graphs have emerged as a powerful approach for representing and querying massive and complex genomic databases [148,150,151,231,258–264,329]. Unlike traditional models, which represent genomic databases as a collection of disjoint, linear sequences, sequences in a graph are represented by graph walks [258,259]. Genome graphs collapse subsequences shared across different database entries into single nodes and indicate adjacent subsequences with edges, compacting sequence sets (by up to thousand-fold [148]). A node's metadata then indicates which entries contain the represented sequence.

Genome graphs provide two fundamental benefits, which make them indispensable, particularly in modern, population-scale genomics [143,148,265–267]. First, the graph topology, along with its metadata, offer ***vast expressive power*** [266–269,330]. A graph naturally encodes the evolutionary history and diversity of a cohort of organisms, revealing their shared and distinct sequences [263,266,268,270]. This leads to reduced bias and improved accuracy in analyses. For example, genome graphs improve disease diagnosis by capturing population-specific variants often missed by analyses on traditional, linear sequences [155,266,331,332]. Second, genome graphs leverage the inherent redundancy of genomic data to ***avoid redundant computation***. This is because shared genomic regions across database entries are stored only once and do not need to be queried separately. This is essential in settings such as population-scale pathogen surveillance, where public health systems must rapidly match new pathogen genomes against large national databases [143,148,264,271–273], enabling timely detection of local outbreaks and limiting broader spread.

De Bruijn Graphs. To efficiently represent large-scale genome graphs, *de Bruijn graphs* (DBGs), which are a type of overlap graph [148,150,151,231,258–263,329,333,334], have been adopted by many recent works [148–151,258,260–263,329,335]. While other genome graph classes exist [270,336,337], DBGs scale much better with both the entries' size and genetic diversity of genomic databases [148,261,338,339], which is why they are preferred in these contexts. Fig. 1 shows an example of a DBG, where each node is a unique *k-mer* (i.e., a subsequence of length k) that is observed in the input, and there is a directed edge from node u to node v if the suffix $(k-1)$-mer of u is equal to the prefix $(k-1)$-mer of v. Each node is then assigned a *color*, representing its metadata. A common strategy to reduce a DBG's size is to instead store its corresponding *compacted DBG*, where all k-mers from a maximal non-branching path are merged into a single node. There are two common ways to represent (compact) DBGs [340]. A *node-centric* DBG of order k

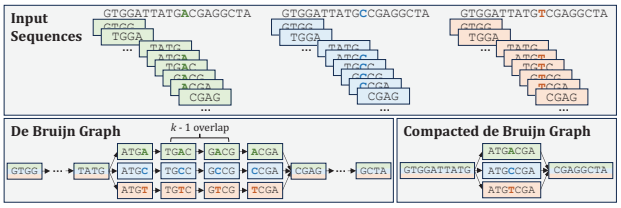

Fig. 1: An example de Bruijn graph and its compacted form.

(e.g., [149,151,341]) stores all k-mers observed in the entries and defines its edges implicitly (Fig. 1). An *edge-centric* DBG of order k (e.g., [148,150,260–262,342,343]) stores observed $(k-1)$-mers as nodes and only stores the edges that represent observed k-mers. In the rest of this paper, we refer to compact DBG as DBG for short.

Graph-based genome analysis involves querying reads against the graph to determine what species or genomic features are present in the sample [148–153] and/or to find their differences from genomes in the database [154,155]. As shown in Fig. 2, queries fall into two categories: ❶ *k-mer set lookups* (where k-mers in a read set are matched against the graph to determine the species present in a sample), and *read mapping* (where each read is analyzed individually). Read mapping can be done by either ❷ alignment-free or ❸ alignment-based approaches. Alignment-free approaches (more common for large-scale studies [148,260]) match all k-mers of a read to the graph, retrieve the metadata associated with the graph nodes to which the k-mers match, and classify the read using this metadata. Alignment-based approaches introduce an additional, computationally expensive approximate string matching step, where candidate regions identified by k-mer matches are refined via dynamic programming to determine precise differences between the read and the graph.

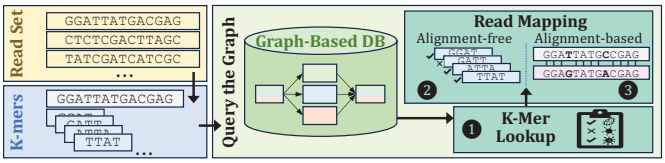

Fig. 2: High-level overview of graph-based genome analysis.

2.3. Unique Characteristics of Genome Graphs

The type of data encoded in genome graphs and the underlying goals of querying genome graphs exhibit unique features not encountered in conventional, non-genome graphs (e.g., social networks, web graphs, road networks). General-purpose graph data structures and methods do not account for these features, leading to significant inefficiencies in storage and computation requirements and missed opportunities. This has led to significant efforts in both software (e.g., [265,331,344–354]) and hardware (e.g., [231–244]) to develop specialized techniques for genome graphs.

The unique semantic characteristics in genome graphs manifest in three key aspects regarding the graph structure, storage (e.g., sparsity and degree distribution), and access patterns. First, in a traditional graph with N nodes, there are $\mathcal{O}(N)$ to $\mathcal{O}(N^2)$ edges that must all be stored explicitly (e.g., adjacency lists [355,356], compressed sparse row formats [357,358]). Genome graphs, most commonly represented as DBGs, are structurally constrained in that they either store nodes or edges (as detailed in §2.2) and the other is obtained implicitly. This is a feature necessitated by the massive scale of genome graphs and enabled by the inherent overlap structure of the sequences they encode.

Second, traditional graphs typically exhibit power-law distributions, in which a small number of nodes have very high degree and many graph-processing optimizations (e.g., [359–361]) exploit this skew. However, DBG node outdegrees and indegrees are at most four (determined by the fixed alphabet size, A, C, G, T, in the underlying data) regardless of the overall graph size. Therefore, optimizations that exploit skewed degree distributions, a common strategy in conventional graph processing systems, are not well-suited to genome graphs, necessitating different optimization approaches.

Third, due to the differences in underlying graph structures, access pattern optimization opportunities vary between genome graphs and non-genome graphs. The compressed data structures used in large-scale genome graphs enforce specific node orderings and, thus, restrict node reordering, a technique commonly used in non-genome graphs to improve access patterns [341,342]. However, the fact that genome graphs encode biological sequences, and that queries are themselves biological sequences, opens new avenues for improving access pattern locality that are unavailable in conventional graph workloads. For example, k-mers from different query reads that share common substrings map to nearby regions in the graph's index structures. In contrast, most conventional non-genome graph queries do not exhibit this kind of structural coupling between the queries and the graph, limiting opportunities for analogous locality-aware optimizations.

2.4. SSD Organization

Fig. 3 depicts the organization of a modern SSD.

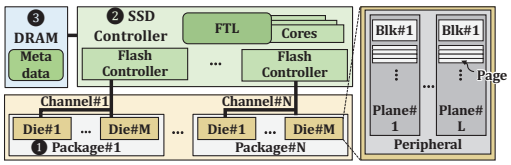

Fig. 3: Organizational overview of a modern SSD.

❶ NAND Flash Memory. A NAND package consists of multiple *dies* or *chips* that share the NAND package's I/O. One or multiple packages share a command/data bus or *channel* to communicate with the SSD controller [290,291,319,362–367]. Each channel can be used by only one die at a time, to communicate with the controller, whereas dies can operate independently. Each die has multiple (e.g., 2 or 4) *planes*, each with thousands of *blocks*, and each block has hundreds to thousands of 4–16 KiB *pages*.

Read/write operations take place at page granularity; erase operations at block granularity. The planes in each die share the peripheral circuitry for access to pages. Thus, planes in a die can operate concurrently when accessing pages (or blocks) at the same offset via *multiplane* operations [291,313,362,368,369].

❷ SSD Controller. An SSD controller consists of two key components: the *flash translation layer (FTL)* and the

flash controllers. The FTL, executed on multiple cores, is responsible for communication with the host, internal I/O scheduling, and various SSD management tasks. The *per-channel hardware flash controllers* manage the request handling [290,291,319,362–367,370] and error correction for the NAND flash chips [290,291,363–366,370–381].

❸ **DRAM.** Modern SSDs employ low-power DRAM to store metadata crucial for SSD management, e.g., logical-to-physical (i.e., L2P) mappings. L2P mappings are typically maintained at a granularity of 4KiB to enhance random access performance. The required capacity for L2P mappings is about 0.1% of the SSD's capacity, considering a 32-bit architecture, with 4B of metadata for every 4KiB of data. This translates to a 4-GB LPDDR4 DRAM for a 4-TB SSD [382].

3. Motivational Analysis

We conduct experimental analyses to assess the impact of I/O overheads on graph-based genome analysis performance.

3.1. Data Movement Overheads

Tools and Datasets. We use two state-of-the-art tools for large genome graph analysis: (*i*) Fulgor [151], a node-centric tool, and (*ii*) the MetaGraph framework [148,260–262], an edge-centric tool. For each, we use the best-performing thread count and color encoding scheme. We evaluate k-mer set lookup, a fundamental task on genome graphs (§2). Graphs are built from a pilot subsample of the global MetaSUB Consortium [143] dataset. The resulting graph (with colors) is 659 GB for Fulgor and 822 GB for MetaGraph. We evaluate queries with varying read counts, ranging from 100K reads[4] to 10M reads (35 MB to 3.5 GB). §11 provides further methodology details.

System Configurations. Our experiments run on a high-end server with an AMD EPYC 7742 CPU [383] and 1.5 TB of DDR4 DRAM [384]. DRAM capacity *exceeds* the total size of all data accessed during the analysis. This ensures that we capture the intrinsic I/O overhead of transferring large, low-reuse data from the storage system to main memory, without being constrained by memory capacity. We analyze I/O overheads using two state-of-the-art SSDs: (*i*) `SSD-G4`, with a PCIe Gen4 interface [385] and (*ii*) `SSD-G5`, with a PCIe Gen5 interface [386].

Observations. Fig. 4 shows throughput (#queries/sec) of the tools, normalized to that of a hypothetical configuration with zero performance overhead due to storage I/O (`No-I/O`). We observe that in all cases, I/O leads to large overheads, even with the state-of-the-art SSDs. Compared to `SSD-G4` (`SSD-G5`), `No-I/O` leads to 16.7× (9.3×) and 7.5× (4.5×) better average performance in Fulgor and MetaGraph, respectively.

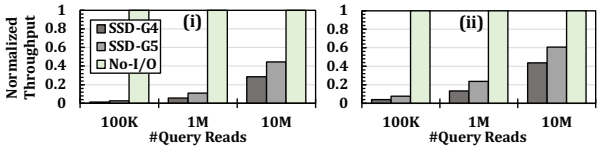

Fig. 4: Normalized throughput of (*i*) Fulgor and (*ii*) MetaGraph under different storage configurations and query sizes.

[4]Note that smaller queries are also critical, as some applications (e.g., searching for antimicrobial resistance [143,148,264,271] or for a single gene of interest [121]) require searching a single or a few genes within a large graph.

Fig. 5 shows the throughput of the tools normalized to `No-I/O`, across different database sizes. The larger graph (G_MetaSUB) is the one discussed earlier. The smaller graph (G_SRArep) is generated from a representative subset of the Sequencing Read Archive's [157] public portion. The resulting graph (with colors) is 161 GB for Fulgor and 231 GB for MetaGraph. We observe that, in all cases, I/O overhead is substantial and increases with database size. For example, the speedup of `No-I/O` over `SSD-G5` increases from 2.7× to 9.2× (as the graph database size grows from 161 GB to 659 GB) in Fulgor and from 4.3× to 13.4× (as the graph database size grows from 231 GB to 822 GB) in MetaGraph.

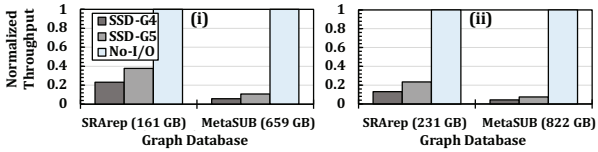

Fig. 5: Normalized throughput of (*i*) Fulgor and (*ii*) MetaGraph under different storage configurations and graph sizes.

Based on these observations, we conclude that storage I/O leads to large overheads in graph-based genome analysis, an overhead that is expected to exacerbate in the future. This I/O overhead is due to moving large amounts of *low-reuse* data from the storage system all the way to main memory, caches, and computational units. Due to its low reuse, caching this data in DRAM during analysis (even if DRAM is larger than the data size, as evaluated in our analyses) does not significantly amortize this overhead. While several works (e.g., [156,160,280–287]) accelerate analysis with genome graphs, to our knowledge, they do not address storage I/O overheads. Although mitigating other bottlenecks (e.g., main memory or computation) leads to large benefits, doing so does not alleviate I/O overheads, whose impact on end-to-end performance becomes even larger as other bottlenecks are alleviated.

This I/O overhead, due to moving large amounts of low-reuse data, is hard to avoid. One might think it is possible to avoid it by (*i*) producing smaller databases through sampling (e.g., [274–279]) or (*ii*) maintaining all required graph data completely and always resident in main memory. However, neither of these is suitable. The first approach necessarily reduces accuracy [156], rendering it inapplicable for many use cases (e.g., [156,160,280–287]). The second approach is energy-inefficient, unsustainable, costly, and unscalable for two reasons. First, the sizes of graph-based databases (which are already large, i.e., tens to hundreds of TBs in some recent examples [148,288]) have been growing rapidly [157,289,328,387]. Second, independent of the sizes of individual graphs, different analyses need different graphs, constructed from different sets of genomes and/or with varying parameters. For example, clinical studies may require graphs with patients' genomes kept separately for privacy [388], while metagenomic studies and wastewater monitoring require highly diverse genome sets [277,282,389,390]. Therefore, keeping all data for all possible analyses in DRAM at all times is prohibitively inefficient and ultimately unsustainable.

Several works (e.g., [204,313–315]) have already demonstrated significant I/O overheads in non-graph-based genome

analysis. While genome graphs enable a more compact representation of population-scale databases (thereby making it feasible to analyze databases that would be prohibitively large under traditional, linear representations), they still suffer from large I/O overheads. This is because the massive and continually growing scale of these databases still leads to very large graphs with low data reuse, and the graph topology introduces irregular, dependent access patterns, further reducing locality.

3.2. Alleviating Data Movement Overheads

3.2.1. Storage-Centric Computing (SCC). Processing data in the storage device can be a fundamental approach for reducing the I/O overheads for three reasons. First, SCC eliminates unnecessary data movement of low-reuse data across the system. Second, SCC reduces the computational burden imposed by low-reuse data on the rest of the system (e.g., main memory and computational units), such that these components can be used for other purposes or be turned off to save energy, or can be made simpler or less costly [315,391]. Third, as highlighted by many prior works (e.g., [307,309,313,392–397]), SCC can leverage the high internal bandwidth of SSDs. In modern SSDs, the internal bandwidth often surpasses the external bandwidth. For example, a state-of-the-art SSD controller [398] delivers 14 GB/s of external bandwidth and up to 57.6 GB/s of internal bandwidth (achieved via 16 channels, each with a peak of 3.6 GB/s). Over-provisioning internal bandwidth in modern SSDs is important to protect user-perceived external I/O performance, mitigating the effects of (*i*) channel contention [319,320,399,400] and (*ii*) the SSD's internal data migration for management tasks [290,321,401,402] and refresh [375,377]. The effective internal bandwidth increases even further when processing directly in the NAND flash dies [322,369,403]. This is because in-flash processing exploits the aggregate bandwidth across all dies, a bandwidth that grows with the number of dies and significantly exceeds both (*i*) the bandwidth available to processing units on the SSD controller and (*ii*) the external SSD bandwidth.

3.2.2. Challenges and Goal. Designing a SCC system for graph-based genome analysis is challenging because none of the existing approaches can be directly implemented inside storage effectively due to modern SSDs' constrained hardware resources. Both node- and edge-centric representations of genome graphs require many random accesses during analysis. These random and irregular accesses cause costly contention in internal SSD components (e.g., channels and NAND flash chips [319–321]), which hinder leveraging the SSD's internal bandwidth. Therefore, directly adopting existing approaches inside storage leads to performance and energy overheads.

Some SCC systems in other domains regularize accesses by performing sorting in either the SSD or the host. However, these approaches are not effective for graph-based genome analysis. Sorting the large number of accesses is impractical within the SSD due to resource constraints. Sorting on the host also fails to fully address access irregularity, as graph-based genome analysis involves dependent accesses. Coordinating sorting between the host and the SSD for each round of dependent accesses incurs significant data movement overhead.

Our goal in this work is to improve the performance and efficiency of graph-based genome analysis by alleviating its data movement overheads from storage cost-effectively.

4. GRAINS

We propose *GRAINS*, a versatile storage-centric system for graph-based genome analysis. GRAINS supports major operations in analysis with genome graphs (e.g., k-mer set lookup and read mapping), and for alignment-based workflows, it flexibly integrates with existing alignment accelerators. GRAINS is primarily designed as a system to accelerate analysis with genome graphs. GRAINS augments the existing SSD controller, flash dies, and FTL, and when not in the analysis acceleration mode, the SSD remains available for other applications, like a conventional general-purpose SSD.

We address the challenges of designing a SCC system for graph-based genome analysis via storage-aware algorithm architecture co-design based on our detailed examination of typical analysis pipelines that operate on genome graphs. To this end, we (*i*) make these pipelines more storage-friendly and (*ii*) further improve their performance, energy-efficiency, and cost-effectiveness via ISP and IFP.

GRAINS's algorithm-architecture co-design comprises three key aspects. First, we design ***efficient batching and reordering techniques and pipelined execution flow*** specialized to the properties of genome graphs to reduce the number of random accesses to the graph. Second, we devise ***lightweight IFP units*** to process the needed parts of each page's graph data in the flash die. This IFP design avoids transferring low-reuse or unused parts of flash pages outside the die, thereby preventing bandwidth waste. Third, to fully exploit die-level parallelism during IFP, we design an ***effective yet lightweight SCC scheduling technique***, enabled by repurposing existing SSD structures. GRAINS's data placement (§8) drastically reduces the required in-DRAM mapping metadata, allowing us to use the freed-up internal DRAM space to maintain small per-die scheduling tables. We devise small ISP units on the SSD controller to interface with these tables and schedule IFP operations at low cost. To efficiently realize these aspects, while guaranteeing correctness and integrity, we apply simple changes to the FTL and adopt a lightweight ECC [322].

Leveraging our optimizations, GRAINS's SCC steps require only simple operations and small buffer spaces, enabling execution on either (*i*) our lightweight, specialized IFP and ISP hardware units or (*ii*) general-purpose IFP (e.g., [369,403–405]) and ISP (e.g., [406–421]) units. Efficient genome graph analysis on both stems from GRAINS's specialized batching, scheduling, and data/computation flow coordination. Execution on special-purpose lightweight hardware leads to higher power efficiency (§12.2), while execution on existing general-purpose cores on the SSD controller or general-purpose IFP and ISP units facilitates near-term adoption. Ultimately, choosing between these GRAINS configurations is a design decision.

Fig. 6 shows an overview of GRAINS, with its lightweight IFP processing elements (PEs) on NAND flash dies and the lightweight ISP PE and scheduler on the SSD controller. GRAINS FTL (§8) orchestrates host communications and data flow across the SSD components. After receiving a request through GRAINS's interface commands (§9) for accelerating

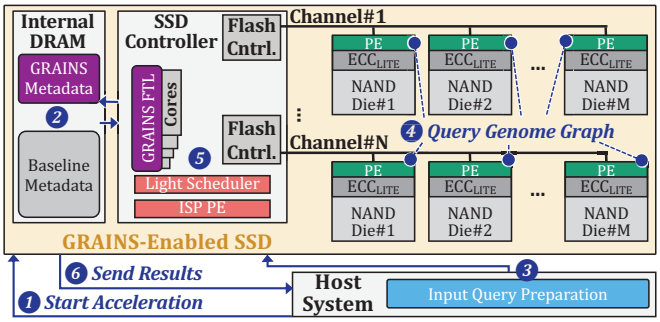

Fig. 6: Overview of GRAINS.

graph-based genome analysis (❶ in Fig. 6), GRAINS prepares (❷) for SCC execution by flushing the conventional FTL metadata and loading the GRAINS FTL metadata (§5). When execution starts, in the *first step* (§6), GRAINS prepares input queries in the host via efficient batching and pipelined execution, and transfers the query batches to the SSD (❸). In the *second step* (§7), GRAINS accesses graph data structures via its IFP (❹) and ISP (❺) units and sends the query results to the host (❻). These two steps are pipelined, with efficient coordination between resources to avoid writes during SCC. Through its efficient execution flow and by avoiding frequent management tasks (by not needing writes during SCC), GRAINS effectively leverages the SSD's large internal bandwidth.

5. Data Structures

GRAINS uses a node-centric DBG representation based on an associative dictionary to compactly represent graph sequences and their associated colors. Similar to prior large-scale genome graph analysis tools (e.g., [151,422–424]), we use SSHash [341] as our dictionary since, by exploiting the statistical features of k-mers, it achieves a substantially better space-time trade-off over prior sequence dictionaries.[5]

Graph Sequences. We store DBG's *unitigs* (i.e., maximal non-branching paths) as contiguous strings in a predetermined order, so that a k-mer occurring in any unitig can be quickly located using a minimal perfect hash function [425] built for the set of k-mers.[6] Fig. 7 shows an overview of the DBG structures. The unitigs are stored in *Strings*, and are indexed via *Offsets* and *Sizes*. Given a query read R, k-mers from the read are extracted and looked up. Consider k-mer g. First, its *minimizer* (i.e., the first m-mer with the smallest hash value) is extracted (❶ in Fig. 7). Second, with the minimizer's minimal perfect hash value (h), Sizes is indexed (❷). Third, with the Sizes values at locations h, Offsets is indexed (❸). $Sizes[h]$ from $Sizes[h+1]$ is then subtracted to find how many Offsets elements to read at $Sizes[h]$. Fourth, with the Offsets values, Strings is indexed (❹), and a window of length $k-m+1$ [341] in Strings is checked to find the minimizer and check if the rest of the k-mer matches.

Graph Colors. For storing graph colors, we adopt an approach similar to [151]. As shown in Fig. 8, we sort the unitigs in Strings based on their colors (i.e., the ID pointing to the metadata for

[5]Since our techniques rely on general DBG properties, they are also applicable to other k-mer dictionaries that may be used as DBG backbones [148–151,329,344,422–424].

[6]Note that in DBGs, edges between unitigs are represented implicitly via $(k-1)$-mer overlaps (§2.2).

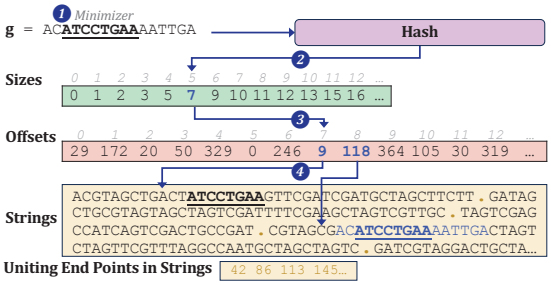

Fig. 7: Overview of the DBG structures and k-mer query.

that graph unitig), and use a simple *Color Bitmap* to mark the start of each new color. This encoding efficiently stores repeated colors at low storage overhead.

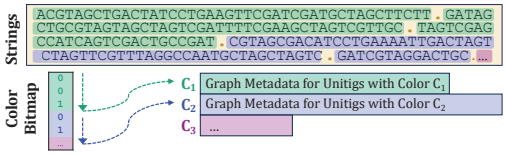

Fig. 8: Graph color encoding used in GRAINS.

6. Input Query Processing

In this step, GRAINS prepares reads for efficient graph queries in the next step (§7). We perform two new optimizations inspired by the characteristics of genome graphs and queries.

Cross-Read K-Mer Batching. Instead of querying each read in a read set individually, we analyze k-mers across different reads together. This way, we can batch k-mers to reduce the number of random accesses to the graph. To enable this, GRAINS populates a lightweight data structure associating each k-mer with the reads it originates from. When graph queries return the colors of matched k-mers, GRAINS assigns colors to each read. We perform this step in the host since extracting k-mers from reads incurs frequent writes, which would reduce the SSD's lifetime if performed inside the SSD.

Genome-Graph-Aware Query Reordering. We leverage a common property of minimal-perfect-hash–based k-mer dictionaries in DBGs: The metadata required for identifying the small per-minimizer lookup ranges (i.e., the Sizes array, as shown in Fig. 7) is much smaller than the graph's Offsets and Strings. Although the full graph is far too large and incurs large I/O overhead to access it in the host (as shown in §3), Sizes occupies only a small fraction of the total space (e.g., <4% in our large-scale graphs). This enables a powerful opportunity: We perform k-mer lookups on Sizes in the host and use the resulting Offsets indices to sort and batch k-mers before sending them to the SSD, thereby reducing the number of accesses and improving the access patterns to Offsets. To execute this step efficiently, as detailed below, we ensure that the time spent sorting the data and transmitting it to the SSD does not introduce significant overheads.

Fig. 9 shows an overview of GRAINS's query processing. GRAINS starts by extracting k-mers from each read (❶ in Fig. 9) and looking them up in Sizes (❷). To reduce the overhead of sorting and transferring k-mers to the SSD, we take two measures. First, we propose a new k-mer processing scheme by improving upon k-mer processing in [315,426]. We *par-*

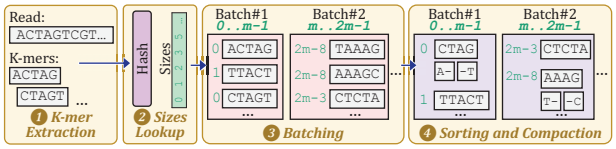

Fig. 9: Overview of GRAINS's query processing.

tition the k-mers into batches corresponding to equal, disjoint ranges of Sizes values (❸). Since these values serve as indices into Offsets, this partitioning enables an efficient pipeline: the sorting of one batch overlaps with the data transfer and Offsets lookups of the previous batch. Second, during sorting, we further compact k-mers by exploiting the fact that consecutive k-mers sorted by Sizes share the same minimizer (❹). Thus, we store the minimizer once and keep only the differences, thereby significantly reducing the transferred data size (e.g., by 2.3× on average across our evaluated datasets).

7. Graph Query

In this step, GRAINS looks up the query k-mers in the graph and, for k-mers that match a graph unitig, retrieves the associated colors. GRAINS executes this step inside the SSD since it involves accessing large graph structures with low reuse and requires only lightweight computation. This enables us to leverage the SSD's high internal bandwidth and reduce the burden of moving and analyzing large, low-reuse data on the rest of the system. In particular, GRAINS performs IFP to avoid transferring unnecessary parts of pages out of flash dies and to leverage multiple dies in parallel. To efficiently execute this step, GRAINS needs to leverage the large internal bandwidth without requiring expensive hardware resources in the SSD (e.g., costly logic units or large DRAM capacity/bandwidth). We demonstrate how GRAINS meets these requirements when accessing genome graph Offsets (§7.1), Strings (§7.2), and Colors (§7.3). A lightweight FSM control unit in the SSD controller coordinates execution flow between these stages.

7.1. Accessing Graph Offsets

Fig. 10 provides an overview of GRAINS's Offsets lookups. GRAINS receives Offsets indices and compacted k-mers from the host in chunks (❶). The internal DRAM holds two small chunks: one incoming from the host and one used for querying Offsets. For an SSD with 16 channels, 8 dies/channel, 4 planes/die, and 4-KiB pages, GRAINS needs two 2-MiB chunks. GRAINS reads the values from DRAM, maps them to physical addresses using its simple mapping (§5), and accesses Offsets (❷). Since the Sizes values are sorted by the host (§6) and Offsets are uniformly placed across dies (as detailed in §5), the lookups lead to sequential accesses that fully exploit the internal bandwidth. After loading a page into a die's page buffer, GRAINS applies ECC_{LITE} [322] to correct errors (❸). The IFP units then select and extract only the targeted Offsets entry

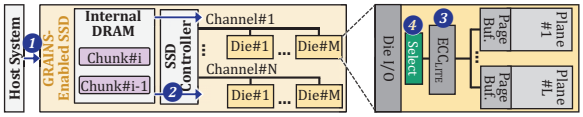

Fig. 10: Overview of GRAINS's Offsets lookups.

and send it to the controller (❹), avoiding the transfer of full pages. GRAINS caches the extracted values in internal DRAM for subsequent Strings queries. Although full pages are read inside each die, only a small part of each page is written to internal DRAM, avoiding the internal DRAM bandwidth from becoming a bottleneck.

7.2. Accessing Graph Strings

We also aim for the accesses to the large Strings structure to also fully exploit the SSD's large internal bandwidth; however, unlike Offsets, the indices into Strings (i.e., the retrieved Offsets entries) arrive unsorted, preventing simple streaming.

Lightweight Scheduling. We introduce a lightweight scheduler that (*i*) avoids redundant page accesses, and (*ii*) maximizes die-level parallelism, without relying on sorting accelerators in the SSD, which are inefficient, given the large number of elements and the limited bandwidth of internal DRAM [420].

As shown in Fig. 11, GRAINS maintains a small per-die table, called GRAINS Scheduler Table (*GST*) in the internal DRAM, with one row per Strings page. Each row stores the bit address within the page (as marked by Offsets entries), the k-mer minimizer and its suffixes/prefixes (Fig. 9), a flag indicating whether the row is full (and if so, a pointer to an extension table), and a small one-hot encoded bitmap marking the target plane in the die. The bitmap allows GRAINS issue *multi-plane* SSD operations, so that multiple planes of the same die serve different accesses concurrently. When accessing Strings, GRAINS schedules requests by simply reading one row per GST sequentially and issuing requests across all dies and planes in a round-robin manner. This naturally coalesces accesses to the same page, avoids redundant page accesses, and achieves high parallelism with small metadata and no complex logic.

GRAINS can store GSTs in the internal DRAM because (*i*) its efficient address mapping (§8) frees most of the DRAM, and (*ii*) our compact k-mer representation (§6) keeps GST entries small. Internal DRAM is sufficient even for large read sets and graphs (e.g., our large evaluated read set with ∼10 million query reads on a large-scale genome graph[7] requires only 2.9 GB, well within the 4-GB internal DRAM in a typical 4-TB SSD). Furthermore, to ensure efficient functionality even when usage exceeds the internal DRAM's capacity, GRAINS keeps some Sizes batches (§6) in host DRAM until the SSD finishes processing the current batch.

Fig. 11 shows an overview of Strings lookups. First, GRAINS's scheduler reads Strings addresses and corresponding k-mers from each GST (❶) and sends the k-mers to the corresponding dies (❷), incrementing each GST's row pointer for the next access. Second, GRAINS reads the corresponding Strings page, and after ECC_{LITE} (❸), the IFP unit performs a comparison between the Stringsin the indexed window and the incoming k-mer from the controller (❹). Finally, GRAINS sends the comparison result and the unitig IDs of the matching k-mers back to the SSD controller.

[7]The graphs we evaluate (§11) are among the largest single graphs constructed in practice. Graph-based databases that encode even larger volumes of data typically consist of several graphs that need to be queried. This is because graph construction time scales super-linearly, making construction of a single very large graph prohibitively slow [342,425].

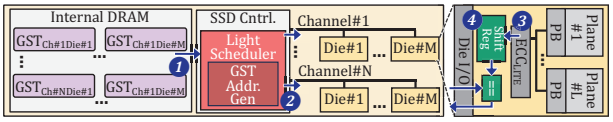

Fig. 11: Overview of GRAINS's Strings lookups.

7.3. Accessing Graph Colors

For each query k-mer that matches a unitig (§7.2), GRAINS retrieves the color of that unitig. Due to GRAINS's efficient scheduling (§7.2), unitig IDs are already retrieved in the internal DRAM in page order. Thus, given the underlying color encoding (§5), the unitigs' colors can be obtained via efficient sequential scans of the Color Bitmap (§5). Fig. 12 illustrates this. GRAINS's ISP units stream the next unitig ID from the internal DRAM into a small register (❶) while concurrently scanning the Colors Bitmap from NAND flash dies (❷). Given that the bitmap is consumed immediately, GRAINS's ISP units directly operate on NAND flash streams (as in prior work [420]), without buffering them in the internal DRAM. This avoids bottlenecking the internal DRAM's bandwidth. GRAINS uses only two small, 32-bit registers per channel: one for the current and one for the incoming bitmap chunk. As the bitmap is processed, each time a "1" is encountered (i.e., a new color is encountered), GRAINS increments *Color Index* (❸). Once the bitmap position matches the unitig ID, GRAINS uses the Color Index to index the Colors (❹). To access Colors, GRAINS uses the same approach as Offsets (§7.1), where, after light ECC (❺), the IFP units select and transmit only the relevant portion of each page (❻). The retrieved color entries are returned to the host, where each color identifies the database entries (e.g., species or samples) containing the matched sequence.

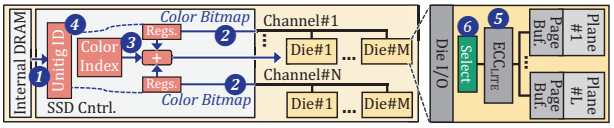

Fig. 12: Overview of GRAINS's Colors lookups.

7.4. Detailed Design of the IFP Processing Elements

GRAINS's IFP PEs follow a high-level flow: (*i*) a page is read into the die's page buffer, (*ii*) the page's content goes through ECC_{LITE}, and (*iii*) the PE operates on the page buffer contents.

IFP Hardware Units. GRAINS's IFP PEs implement two lightweight functions: (*i*) *selection*, which extracts a targeted window of a page, and (*ii*) *comparison*, which performs a bitwise match between a k-mer and a targeted window. The page buffer in a standard NAND flash die is already organized as a byte- or word-addressable latch-based array with existing sense amplifiers and column select logic. GRAINS's selection mechanism repurposes this column decoding circuitry. Given an offset, the column logic isolates the targeted bytes. Instead of streaming the full page over the channel bus, this targeted window is either routed directly to the die's I/O interface or fed internally into the IFP comparison unit. The comparison unit itself consists of a small shift register and a bitwise comparator that checks the k-mer against the targeted window.

Interaction with Flash Controllers. The flash controller sends a small parameter (an offset or a k-mer) to the on-die PE via the existing channel bus. This parameter is stored in a small register on the die and is delivered alongside the standard page read command. After the IFP operations, the controller reads back only the result (instead of the full page).

8. FTL Integration and Reliability Support

GRAINS FTL requires only minor changes to the baseline FTL. It designates each block as either genomic or non-genomic, and identifies genomic accesses through GRAINS commands (§9). For all non-genomic data, vendor-specific FTL features remain untouched, and the SSD behaves conventionally. At the start of its SCC operations, since GRAINS does not perform writes during them, it flushes the standard L2P metadata to the flash and loads the much smaller GRAINS L2P metadata (as detailed below) into internal DRAM, while also keeping the other metadata of a conventional FTL.

Physical Allocation. Since GRAINS regularizes accesses, GRAINS's data placement is simple and enables (*i*) efficiently leveraging the SSD's internal bandwidth and (*ii*) minimizing mapping metadata. We uniformly distribute the blocks of the underlying graph data structures and the query reads across different channels, dies, and planes in a round-robin way. For efficient multi-plane operations, active blocks across different planes are aligned to the same page offset.

This placement works synergistically with GRAINS's batching and scheduling, enabling well-balanced, parallel accesses across all internal SSD resources. That said, for workloads with persistently skewed query distributions (e.g., when specific genomic regions are queried disproportionately often), profile-guided placement could offer additional benefits. Such an optimization would integrate seamlessly with GRAINS, affecting only the initial data layout. We describe this optimization further in an extended version [427].

FTL Metadata Size. GRAINS reduces the required FTL metadata size due to two reasons. First, instead of page-level L2P mappings (which, with 4 KiB pages, leads to ∼4 GB of metadata for a 4-TB SSD), GRAINS stores L2P metadata at block granularity. This is possible because GRAINS performs no writes during SCC, and thus, no fine-grained remapping is needed. For a 1-TB graph with 12-MB blocks, this requires only 0.35 MB. Second, GRAINS's uniform data placement makes physical locations computable from base addresses and stride, minimizing per-block metadata needed for the SSD's reliable operations (e.g., read-disturbance counters). With per-block metadata, overall metadata size is 0.7 MB, freeing most of the internal DRAM for GRAINS's SCC and scheduling tables.

Error Correction. For data accessed by GRAINS's ISP units on the SSD controller (e.g., Color Bitmap), accesses occur after the regular ECC. For structures accessed by GRAINS's on-die hardware units, we adopt a lightweight on-die ECC technique, ECC_{LITE}, along with its associated *Bias Error Encoding* [322].

Other Management Tasks. Since in SCC mode, GRAINS does not incur writes, it does not require write-related management such as GC and wear-leveling. GRAINS performs other tasks for ensuring reliability before or after SCC since (*i*) the duration of each GRAINS process is significantly shorter than the manufacturer-specified threshold for reliable retention age

(e.g., a year [428]), and (*ii*) GRAINS avoids read-disturbance errors [290,291,362,363,377,380] during SCC since, due to its efficient execution, each block is read at most once. After SCC steps, to prevent read disturbance errors in the future, GRAINS refreshes blocks whose read count exceeds a specific threshold.

9. Interface Commands

GRAINS needs three new NVMe commands.

`GRNS_Start` initiates graph genome analysis acceleration. When it receives this command, GRAINS prepares itself for execution in SCC mode (§4). During this phase, GRAINS handles execution/data flow based on GRAINS FTL and FSM controller.

`GRNS_Steps` marks the start and end of each pipeline step and triggers GRAINS's lightweight FSM on the SSD controller, which drives the internal execution during SCC. The host uses this command to signal that a query batch is ready, then transfers the batch to the SSD's internal DRAM via the standard NVMe data path. Unlike conventional writes, these batches remain in the internal DRAM for processing and are not programmed to flash. The SSD then processes the batch through its internal pipeline with GRAINS's FSM on the SSD controller coordinating execution and data flow, and returns results to the host. This enables pipelining multiple batches: as the SSD processes the current batch, the host prepares the next one. No additional commands are needed because the FSM fully determines the internal execution flow once a batch arrives. At the end of the last step, GRAINS returns to operation as a conventional SSD.

`GRNS_Write` is a specialized command for writing genome graphs to the SSD, which updates both the regular FTL's L2P metadata and GRAINS's small L2P metadata.

10. Deployment and Integration

GRAINS offers flexible deployment paths to minimize integration costs across both the hardware and software stacks due to three key reasons. First, as discussed in §4, GRAINS's ISP/IFP steps can run on either specialized lightweight logic for maximum power efficiency, or on general-purpose ISP/IFP engines (e.g., [403,404,406–420]) for near-term adoption. Second, the required FTL modifications (§8) are minimal and non-disruptive, and the SSD behaves identically to a conventional SSD outside its SCC mode. Third, system-level integration is similarly seamless: GRAINS introduces only three new NVMe commands via standard vendor-specific extensions, and file system and driver changes are confined to recognizing genome graph files and dispatching these commands. We explain each of these reasons for GRAINS's flexible deployment in more detail in an extended version [427].

11. Methodology

Performance. We design a simulator that models all of GRAINS's components, including host operations, internal DRAM, accessing flash dies, in-storage hardware units (on flash dies and on the SSD controller), and SSD-host interfaces. We then feed the latency and throughput of each component to this simulator. Using this methodology, as also leveraged in prior SCC works (e.g., [301,313,315,369,392,429]), enables us to flexibly incorporate state-of-the-art system configurations in our evaluations. For **hardware-based** components (e.g., when querying the graph): We implement GRAINS's ISP/IFP logic units in Verilog and synthesize them with a 22nm library [430] using the Synopsys Design Compiler [431]. We use two state-of-the-art simulators, MQSim [367,432] to model SSD's internal operations, and Ramulator [433,434] (new version introduced in [435,436]) to model internal DRAM. For **software-based** components (e.g., when preparing queries), we measure their performance on a real system with 1.5-TB DRAM (in all experiments unless mentioned otherwise) and AMD® EPYC® 7742 CPU [383] with 128 physical cores. We use `SSD-G4` [385] and `SSD-G5` [386] as described in §3 and Table 1 in our real system experiments. In our MQSim simulations for the ISP/IFP steps, we faithfully model these SSDs with configurations summarized in Table 1. For the software baselines, we measure performance using the best-performing thread counts on the same real system used for GRAINS's software-based components. In an extended version [427], we provide further details regarding our methodology and how it manages correct timing composition between host-side and SSD-side time.

Area and Power. For GRAINS's logic units, we use our Design Compiler synthesis results. SSD power is based on values of a Samsung 3D NAND flash-based SSD [382]. DRAM power is based on values from a DDR4 model [384,440]. For the CPU cores, we measure power with AMD μProf [441].

Evaluated Systems. We evaluate three key tasks: (*i*) k-mer set lookup, (*ii*) alignment-free read mapping, and (*iii*) alignment-based read mapping. We evaluate the following tools for analysis with large-scale genome graphs:[8] (*i*) Fulgor [151] (FG), as a state-of-the-art node-centric software tool (which uses SSHash [341]), (*ii*) the MetaGraph framework [148,260–262] (MG) as a state-of-the-art edge-centric software tool; (*iii*) an *idealized* hardware-accelerated baseline (IdealAccMem) where all genome graph query operations execute *without* main memory overheads (i.e., zero latency, infinite bandwidth, infinite capacity). This configuration can represent an *ideal* PIM baseline as well. IdealAccMem still incurs the I/O overhead of bringing the large, low-reuse data from the storage system to main memory. Comparing against this baseline demonstrates the impact of storage I/O as a fundamental problem that persists even when main memory overhead is alleviated; (*iv*) GRAINS with all its optimizations, but with its lightweight hardware as an accelerator outside the SSD (GRN-Ext), to show the benefits of GRAINS's

Table 1: Evaluated SSD configurations.

Specification	SSD-G4	SSD-G5
General	TLC NAND flash-based SSD 4 TB capacity, 4 GB internal DRAM [437]	
Bandwidth (BW)	PCIe Gen4 interface [438]; 7 GB/s sequential-read BW; 1.2-GB/s channel I/O rate	PCIe Gen5 interface [439]; 14.8 GB/s sequential-read BW; 2.4-GB/s channel I/O rate
NAND Config	16 channels, 8 dies/channel, 4 planes/die	

[8] As discussed in §2.2, DBGs scale substantially better than other genome graph structures as the genetic diversity in the database increases. Other tools (e.g., VG [442] and minigraph [354]) that use variation graphs as their graph structure do not scale as well and, hence, are not typically used in scenarios with large-scale, genetically diverse databases [148,338].

optimizations without ISP/IFP, but with more storage-friendly execution/data flow. The lightweight hardware is connected to the host system via a PCIe interface with 16 GB/s bandwidth, and leverages the host DRAM to store its scheduling metadata (§7.2). Similar to GRAINS's full implementation, GRN-Ext performs input query processing on the host (§6), but instead of relying on its ISP/IFP logic units to query the graph (§7), it uses the same lightweight accelerator logic units outside the SSD; (*v*) GRAINS's full implementation, including ISP and IFP (GRN). All these tools achieve the same accuracy since their underlying graphs are *lossless* encodings of the database's k-mer sets [148,151,260–262]. For the alignment-based mapping, we integrate all tools with SeGraM, a state-of-the-art sequence-to-graph alignment accelerator [231].

Datasets. We build graphs as described in §3. The graph (with colors) is 659 GB for FG, GRN, and IdealAccMem, and 822 GB for MG. We evaluate query read sets with 10M (QL), 1M (QM), and 100K (QS) reads. Note that smaller queries are also critical, as detailed in §3.

12. Evaluation

12.1. Performance

K-mer Set Lookup. Fig. 13 shows the speedups of different systems over FG for k-mer set lookups. We make three key observations. First, GRAINS provides significant speedups. In systems with SSD-G4 (SSD-G5), GRN provides 8.4× (5.5×) and 11.9× (8.5×) average speedups over FG and MG, respectively. Second, by making the execution/data flow more storage-friendly, GRAINS's implementation outside the SSD also provides large benefits. GRN-Ext provides to 3.4× and 5.0× average speedup over FG and MG, respectively, across all inputs and SSDs. Third, GRAINS's full implementation provides significantly larger benefits over GRAINS's implementation outside SSD. GRN provides 2× average speedup over GRN-Ext. While GRN-Ext exploits locality across queries by merging requests to the same regions of each graph structure and improving access patterns, the locality it captures cannot amortize the cost of transferring the large volume of low-reuse data across the storage-to-host interface. Thus, GRAINS's full implementation provides additional, significant benefits by alleviating the large I/O overheads through its SCC design. We further evaluate the execution time breakdown of different GRAINS steps in an extended version [427].

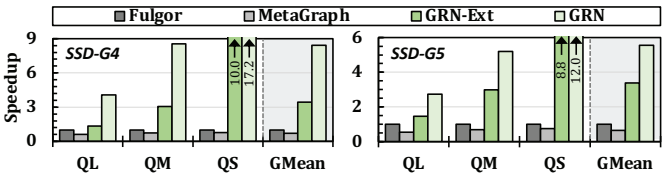

Fig. 13: Speedups with different input query sets and SSDs.

Impact on Cost Efficiency. GRAINS improves system cost-efficiency since it analyzes large amounts of data in the SSD and does *not* rely on either large host DRAM or high host-SSD interface bandwidth. We show this by evaluating two systems: a cost-optimized system ($) with 64-GB host DRAM and SSD-G4, and a costlier, performance-optimized system ($$$) with 1.5-TB host DRAM and SSD-G5. Fig. 14 compares

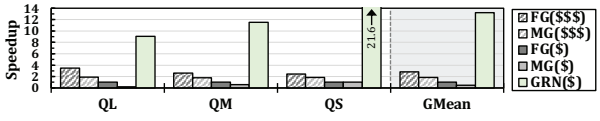

Fig. 14: Speedup of GRAINS on a low-cost system, i.e., GRN($), over baselines on low-cost and performance-optimized systems.

GRAINS on the cost-optimized system with the baselines on both systems (with a representative query set, QM). To fairly evaluate the baselines when host DRAM is smaller than the graph, we reduce I/O overheads as much as possible in software for this scenario. To this end, we partition the graph such that each subgraph fits in the host DRAM, so random accesses to the graph do not repeatedly access the SSD. We partition the graph using database entry metadata to guide partitioning choices, placing entries that likely share many k-mers into the same partition. Despite the benefits of this optimization, two sources of overhead remain: (*i*) the I/O overhead of transferring subgraphs, and (*ii*) the need to query against each subgraph. We make two observations. First, GRAINS on the cost-optimized system significantly outperforms the baselines even on the performance-optimized system. GRN($) provides 4.7× and 5.2× average speedup over FG($$$) and MG($$$), respectively. Second, the performance of baselines on the cost-optimized system suffers significantly. When compared on the same cost-optimized system, GRN($) provides 13.2× and 26.9× average speedup over FG and MG, respectively. We conclude that GRAINS improves both system cost-efficiency and system performance, which is critical for facilitating (*i*) the wide adoption of graph-based genome analysis, and (*ii*) analysis on low-cost, portable devices, a scenario rising in importance with the development of portable sequencing devices [443–446] for on-site genome analysis [159,160,447,448].

Effect of Different GRAINS Optimizations. Fig. 15 demonstrates the benefits of different GRAINS optimizations. GRN-B is a GRAINS configuration including only the batching optimization (§6). GRN-B-S is a GRAINS configuration including both the batching and scheduling (§7.2) optimizations (equivalent to GRN-Ext, §11). GRN-B-S-SCC is GRAINS's full configuration, including its SCC. Please note that we do not show a configuration with naive SCC and without batching and scheduling. This is because simply using SCC without these techniques leads to performance loss due to the irregular access patterns to genome graphs, which are inefficient due limitations of the SSD's hardware (e.g., costly conflicts in internal SSD resources such as channels and NAND flash chips [319–321] during random and dependent accesses). We make three observations. First, GRN-B provides 1.5× and 2.2× average speedup over FG and MG, respectively. This is due to more efficient accesses to the Offsets data structure. Second, GRN-B-S provides 2.3× average speedup over GRN-B as it enables more

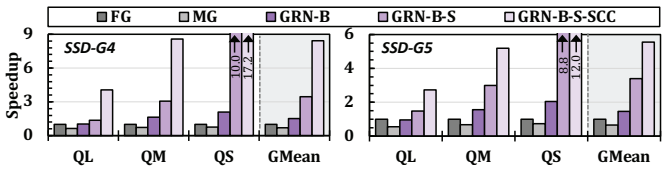

Fig. 15: Ablation tests of different GRAINS optimizations.

storage-friendly accesses to Strings and Colors as well. Finally, through its efficient SCC, GRN-B-S-SCC provides 2.0× and 4.6× average speedup over GRN-B-S and GRN-B, respectively.

Effect of the Number of SSDs. GRAINS can benefit from multiple SSDs in two ways: First, different graphs can be stored on separate SSDs and analyzed concurrently, each benefiting from GRAINS (as shown in Fig. 13). Second, a single graph can be partitioned across SSDs to increase throughput. This is enabled by GRAINS's execution flow supporting efficient graph partitioning, as detailed in an extended version [427]. We evaluate this scenario in Fig. 16. We observe that GRAINS maintains large speedups. This is because, as the external bandwidth increases for the baselines with more SSDs, the internal bandwidth also increases. Although there is a slight decrease in speedup when moving to two SSDs in QS, the speedup is still significant (14.2× and 9.4× with SSD-G4 and SSD-G5). This decrease is because, after increasing internal bandwidth, the impact of software query processing becomes relatively larger. Therefore, we can further accelerate these steps (e.g., with a sorting accelerator) to increase benefits even further. We conclude that GRAINS can efficiently leverage multiple SSDs, making it also suitable for distributed systems.

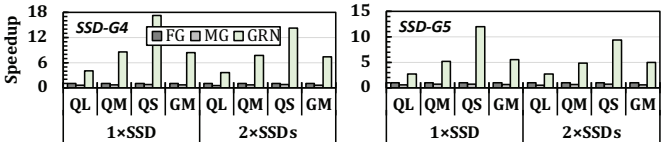

Fig. 16: Speedups with different numbers of SSDs.

Comparison to IdealAccMem. Fig. 17 shows speedups over IdealAccMem. Despite IdealAccMem's benefits, it still incurs I/O accesses to transfer data to main memory, even when DRAM is larger than data size (as in our evaluation). Through its storage-aware design, with efficient ISP/IFP operations, we observe that on the system with SSD-G4 (SSD-G5), GRAINS provides 7.4× (4.3×) average speedup over IdealAccMem.

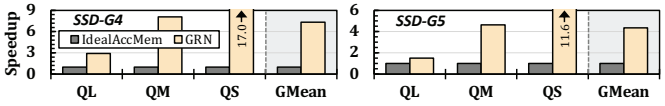

Fig. 17: Speedup over IdealAccMem.

Alignment-Free Read Mapping. Fig. 18 shows speedups over FG when performing alignment-free read mapping. In this case, as discussed in §6, GRAINS sends the colors of all k-mers to the host system, and the host system determines the colors of each read based on the colors of its k-mers. We observe that on systems with SSD-G4 (SSD-G5), GRN provides 7.9× (5.2×) and 11.2× (8.0×) average speedups over FG and MG, respectively.

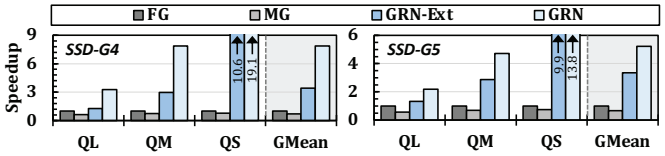

Fig. 18: Speedups for read mapping.

Alignment-Based Mapping. We integrate GRN, FG, and MG with SeGraM [231] to perform alignment on the candidate graph regions identified by each tool (we use the throughput of alignment operations reported by SeGraM [231]). Fig. 19 shows the speedups of different systems over FG +SeGraM. We observe that SeGraM+GRN provides 6.2× and 9.0× average speedup over SeGraM+FG and SeGraM+MG.

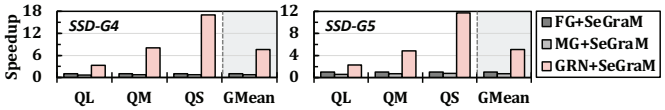

Fig. 19: Speedups for read mapping with alignment.

12.2. Area and Power Overheads

Table 2 summarizes area and power for GRAINS's logic units at 333MHz. While these units can operate at higher frequencies, their throughput is already sufficient since GRAINS's ISP/IFP pipelines are bottlenecked by flash reads. We use area/power values of ECC_{LITE} from the original work [322].[9] GRAINS's hardware overhead is very small. On-controller units are 0.0025 mm^2 (0.7% of the four ARM cores [450] on an SSD controller). GRAINS's on-die logic area is mainly (99%) for ECC_{LITE}, which, as reported by [322], takes only 0.2% of the flash chip area and fits well within its power budget.

As detailed in §7, GRAINS's SCC can execute on either (*i*) our lightweight, specialized ISP/IFP units or (*ii*) general-purpose ISP (e.g., [406–420]) and IFP (e.g., [403,404]). Choosing between these GRAINS configurations is a design tradeoff: general-purpose units facilitate earlier adoption, while specialized units offer better area and power efficiency. For example, GRAINS's ISP units on the SSD controller consume two orders of magnitude lower area and 60.1× lower power than hardware units of a state-of-the-art general-purpose ISP.[10]

Table 2: Area and power consumption of GRAINS's logic units.

Logic unit	Area [mm^2]	Power [mW]
On SSD Controller	0.0025	0.21
On Die (ECC_{LITE})	0.036	18.04
On Die (Others)	0.000093	0.01

12.3. Energy

We find the energy consumption of each evaluated graph-based genome analysis tool (when performing k-mer set lookups) based on the energy consumption of different components (e.g., host processor, host DRAM, host-SSD communication, SSD components, and logic units). We calculate each component's energy based on its dynamic/idle power and execution time. We observe that across our evaluated input queries and SSDs, GRAINS provides 4.4–21.0×, 12.2–31.6×, and 3.1–20.7× energy reduction over FG, MG, and IdealAccMem, respectively, when performing k-mer set lookups.

13. Related Work

To our knowledge, GRAINS is the first storage-centric computing (SCC) system designed to significantly reduce I/O

[9] Since we do not have access to the larger technology node used in [322], we scale ECC_{LITE}'s area to the lower node with the methodology in [449].

[10] Note that the power/area of GRAINS's IFP units are mainly dominated by ECC_{LITE}, needed in both GRAINS and other IFP systems (e.g., [322,403]).

data movement overheads and improve end-to-end performance, energy-efficiency, and cost-effectiveness of graph-based genome analysis.

Analysis with Genome Graphs. Many works (e.g., [265,331,344–354]) propose software tools for genome graphs. Several works propose hardware tools for querying genome graphs by alleviating computation (e.g., [231–240]) and/or memory overheads (e.g., [231,241–244]). However, these works do not alleviate storage I/O overheads, whose impact on end-to-end performance becomes even larger as other overheads are alleviated. As shown in Fig. 19, GRAINS can flexibly integrate with these designs to alleviate their I/O overhead. Some works (e.g., [245–257]) accelerate genome graph *construction*, which is an important, but orthogonal task.

Accelerating Graph Analysis. Many works accelerate graph analysis by alleviating computation (e.g., [360,451–466]), main memory (e.g., [465,467–481]), or I/O (e.g., [297–305]) overheads. However, they are neither sufficient for nor target the unique demands of genome graphs (as detailed in §2.3).

Accelerating Genome Analysis. Many works (e.g., [116,158–203,205–214,217,225,227,229,313,315,482–511]) propose hardware optimizations for conventional genome analysis on linear sequences, including SCC systems (e.g., [204,241,313–318]) for these analyses. However, none of them are suitable for graph-based genome analyses due to fundamentally different data structures, execution flow, and irregular and dependent access patterns.

Particularly, GRAINS exploits the properties of graph-based genome analyses in three ways. First, since GRAINS needs to handle *dependent accesses* across *large data structures residing in the SSD*, it needs to handle scheduling directly in the SSD. To do so, GRAINS introduces a novel data/execution flow and lightweight scheduling (§7), fundamentally different from prior approaches that regularize accesses purely through query scheduling in the host [315] or via offline preprocessing [313]. Second, GRAINS's batching and task partitioning efficiently exploit the underlying properties of genome graphs (§6), which no prior work considers. Third, GRAINS performs end-to-end sequence-to-graph read mapping (which is more complex than traditional sequence-to-sequence mapping).

Several works propose techniques for batching k-mer queries to improve access patterns to genomic data structures (e.g., [512–514]). Despite their benefits, these techniques cannot be directly adopted in SCC designs for genome graphs due to the dependent and random accesses to different data structures. A purely host-side approach would require a separate round trip to the SSD for each dependent access stage, undermining the locality benefits of batching. On the other hand, directly adopting these techniques within the SSD stresses the limited available hardware resources in modern SSDs. GRAINS addresses these limitations with a co-designed approach. First, we perform lightweight, genome-graph-aware batching on the host (exploiting the small Sizes metadata in the host, §6). Second, once these batched requests are dispatched to the SSD, our new, lightweight in-storage scheduling orchestrates all subsequent dependent accesses to large data structures inside the SSD, without any further round-trips to the host (§7).

SCC Designs. Various works propose SCC for different applications (e.g., in AI/ML [299,300,309,322,392,393,396,397,515–523], string matching and read mapping [204,313,316,317,524], k-mer counting [314], graph analytics [296,297,306], and others [307,405,525–545]). Several works propose general-purpose SCC in storage [406–421,429,546], SSDs integrated closely with FPGAs [395,547–552], or with GPUs [553]. None of these works targets analysis with genome graphs.

Some works (e.g., [554,555]) propose performing sub-page reads to avoid unnecessary data movement from the flash dies. However, they focus solely on reducing data transfer granularity and do not perform computation within the dies. Several works propose performing computation in the dies [322,403,404], or using flash memory [369,405,556,557]. However, directly applying these techniques to genome graph analysis is undesirable due to the irregular, data-dependent accesses, which hinders leveraging die-level parallelism. GRAINS addresses this challenge through its genome-graph-aware algorithm-architecture co-design.

Several works (e.g., [397,421,558]) propose SCC designs that combine ISP, IFP, and PIM. However, these works are not sufficient to handle the unique properties of graph-based genome analysis, which pose challenges absent from prior SCC work. We address these challenges via our genome-graph-aware query reordering techniques and lightweight scheduling, enabled via GRAINS's holistic co-design of data/execution flow, data layout, and FTL integration.

14. Conclusion

We introduced GRAINS, the first storage-centric system designed to alleviate the I/O overhead of analysis with large genome graphs. Through detailed examination of these analysis pipelines, we (*i*) make them more storage-friendly and (*ii*) enable efficient in-storage and in-flash processing. Our evaluations show that GRAINS improves performance, energy efficiency, and system cost-effectiveness of graph-based genome analysis at low area and power costs. We hope that GRAINS's optimizations will inspire and benefit storage-centric computing systems in other domains to likewise improve their performance and energy efficiency without relying on expensive hardware resources.

Acknowledgments

We thank the anonymous reviewers of ASPLOS 2026 and ISCA 2026 for feedback. We thank the SAFARI group members for feedback and the stimulating, inclusive, intellectual, and scientific environment. We acknowledge the generous gifts and support provided by our industrial partners, including Google, Huawei, Intel, Microsoft, and VMware. This research was partially supported by the European Union's Horizon Program for research and innovation under Grant No. 101047160 (project BioPIM), the Swiss National Science Foundation (SNSF), Semiconductor Research Corporation (SRC), the ETH Future Computing Laboratory (EFCL), Huawei ZRC Storage Team, and the AI Chip Center for Emerging Smart Systems Limited (ACCESS). Jisung Park was supported by the NRF (RS-2025-00519994) and the IITP (RS-2024-00459026) of Korea.

References

[1] Can Alkan, Jeffrey M Kidd, Tomas Marques-Bonet, Gozde Aksay, Francesca Antonacci, Fereydoun Hormozdiari, Jacob O Kitzman, Carl Baker, Maika Malig, Onur Mutlu, S Cenk Sahinalp, Richard A Gibbs, and Evan E Eichler. Personalized copy number and segmental duplication maps using next-generation sequencing. *Nature Genetics*, 2009.

[2] Gaye Lightbody, Valeriia Haberland, Fiona Browne, Laura Taggart, Huiru Zheng, Eileen Parkes, and Jaine K Blayney. Review of applications of high-throughput sequencing in personalized medicine: barriers and facilitators of future progress in research and clinical application. *Briefings in Bioinformatics*, 2019.

[3] Stefania Morganti, Paolo Tarantino, Emanuela Ferraro, Paolo D'Amico, Bruno Achutti Duso, and Giuseppe Curigliano. Next Generation Sequencing (NGS): A Revolutionary Technology in Pharmacogenomics and Personalized Medicine in Cancer. In *Translational Research and Onco-Omics Applications in the Era of Cancer Personal Genomics*, 2019.

[4] Iuliia Branco and Altino Choupina. Bioinformatics: new tools and applications in life science and personalized medicine. *Applied Microbiology and Biotechnology*, 2021.

[5] Sameer Quazi. Artificial intelligence and machine learning in precision and genomic medicine. *Medical Oncology*, 2022.

[6] Samuel J. Aronson and Heidi L. Rehm. Building the foundation for genomics in precision medicine. *Nature*, 2015.

[7] Hannah F. Löchel and Dominik Heider. Comparative analyses of error handling strategies for next-generation sequencing in precision medicine. *Scientific Reports*, 2020.

[8] Eirini Papadopoulou, Dimitra Bouzarelou, George Tsaousis, Athanasios Papathanasiou, Georgia Vogiatzi, Charalambos Vlachopoulos, Antigoni Miliou, Panagiota Papachristou, Efstathia Prappa, Georgios Servos, Konstantinos Ritsatos, Aristeidis Seretis, Alexandra Frogoudaki, and George Nasioulas. Application of next generation sequencing in cardiology: current and future precision medicine implications. *Frontiers in Cardiovascular Medicine*, 2023.

[9] Alireza Tafazoli, Henk-Jan Guchelaar, Wojciech Miltyk, Adam J. Kretowski, and Jesse J. Swen. Applying Next-Generation Sequencing Platforms for Pharmacogenomic Testing in Clinical Practice. *Frontiers in Pharmacology*, 2021.

[10] Valentina Gambardella, Noelia Tarazona, Juan M. Cejalvo, Pasquale Lombardi, Marisol Huerta, Susana Roselló, Tania Fleitas, Desamparados Roda, and Andres Cervantes. Personalized Medicine: Recent Progress in Cancer Therapy. *Cancers*, 2020.

[11] Rebecca J. Leary, Isaac Kinde, Frank Diehl, Kerstin Schmidt, Chris Clouser, Cisilya Duncan, Alena Antipova, Clarence Lee, Kevin McKernan, Francisco M. De La Vega, Kenneth W. Kinzler, Bert Vogelstein, Luis A. Diaz, and Victor E. Velculescu. Development of Personalized Tumor Biomarkers Using Massively Parallel Sequencing. *Science Translational Medicine*, 2010.

[12] Hamburg Margaret A. and Collins Francis S. The Path to Personalized Medicine. *New England Journal of Medicine*, 2010.

[13] Maaike van der Lee, Marjolein Kriek, Henk-Jan Guchelaar, and Jesse J. Swen. Technologies for Pharmacogenomics: A Review. *Genes*, 2020.

[14] Dabin Moon, Hye W. Park, Dongheon Surl, Dongju Won, Seung-Tae Lee, Saeam Shin, Jong R. Choi, and Jinu Han. Precision Medicine through Next-Generation Sequencing in Inherited Eye Diseases in a Korean Cohort. *Genes*, 2022.

[15] Sumitra Mohan, Victoria Foy, Mahmood Ayub, Hui Sun Leong, Pieta Schofield, Sudhakar Sahoo, Tine Descamps, Bedirhan Kilerci, Nigel K. Smith, Mathew Carter, Lynsey Priest, Cong Zhou, T. Hedley Carr, Crispin Miller, Corinne Faivre-Finn, Fiona Blackhall, Dominic G. Rothwell, Caroline Dive, and Gerard Brady. Profiling of Circulating Free DNA Using Targeted and Genome-wide Sequencing in Patients with SCLC. *Journal of Thoracic Oncology*, 2020.

[16] Claudia C.Y. Chung, Gordon K.C. Leung, Christopher C.Y. Mak, Jasmine L.F. Fung, Mianne Lee, Steven L.C. Pei, Mullin H.C. Yu, Vivian C.C. Hui, Joshua C.K. Chan, Jeffrey F.T. Chau, Marcus C.Y. Chan, Mandy H.Y. Tsang, Wilfred H.S. Wong, Joanna Y.L. Tung, Kin Shing Lun, Yiu Ki Ng, Cheuk Wing Fung, Mabel S.C. Wong, Rosanna M.S. Wong, Yu Lung Lau, Godfrey C.F. Chan, So Lun Lee, Kit San Yeung, and Brian H.Y. Chung. Rapid whole-exome sequencing facilitates precision medicine in paediatric rare disease patients and reduces healthcare costs. *The Lancet Regional Health – Western Pacific*, 2020.

[17] Suzette J. Bielinski, Janet E. Olson, Jyotishman Pathak, Richard M. Weinshilboum, Liewei Wang, Kelly J. Lyke, Euijung Ryu, Paul V. Targonski, Michael D. Van Norstrand, Matthew A. Hathcock, Paul Y. Takahashi, Jennifer B. McCormick, Kiley J. Johnson, Karen J. Maschke, Carolyn R. Rohrer Vitek, Marissa S. Ellingson, Eric D. Wieben, Gianrico Farrugia, Jody A. Morrisette, Keri J. Kruckeberg, Jamie K. Bruflat, Lisa M. Peterson, Joseph H. Blommel, Jennifer M. Skierka, Matthew J. Ferber, John L. Black, Linnea M. Baudhuin, Eric W. Klee, Jason L. Ross, Tamra L. Veldhuizen, Cloann G. Schultz, Pedro J. Caraballo, Robert R. Freimuth, Christopher G. Chute, and Iftikhar J. Kullo. Preemptive Genotyping for Personalized Medicine: Design of the Right Drug, Right Dose, Right Time—Using Genomic Data to Individualize Treatment Protocol. *Mayo Clinic Proceedings*, 2014.

[18] Dean Ho, Stephen R. Quake, Edward R.B. McCabe, Wee Joo Chng, Edward K. Chow, Xianting Ding, Bruce D. Gelb, Geoffrey S. Ginsburg, Jason Hassenstab, Chih-Ming Ho, William C. Mobley, Garry P. Nolan, Steven T. Rosen, Patrick Tan, Yun Yen, and Ali Zarrinpar. Enabling Technologies for Personalized and Precision Medicine. *Trends in Biotechnology*, 2020.

[19] Bashdar Mahmud Hussen, Sara Tharwat Abdullah, Abbas Salihi, Dana Khdr Sabir, Karzan R. Sidiq, Mohammed Fatih Rasul, Hazha Jamal Hidayat, Soudeh Ghafouri-Fard, Mohammad Taheri, and Elena Jamali. The emerging roles of NGS in clinical oncology and personalized medicine. *Pathology - Research and Practice*, 2022.

[20] Laura E. Russell, Yitian Zhou, Ahmed A. Almousa, Jasleen K. Sodhi, Chukwunonso K. Nwabufo, and Volker M. Lauschke. Pharmacogenomics in the era of next generation sequencing – from byte to bedside. *Drug Metabolism Reviews*, 2021.

[21] Renu Verma, Kesia Esther da Silva, Neesha Rockwood, Roeland E. Wasmann, Nombuso Yende, Taeksun Song, Eugene Kim, Paolo Denti, Robert J. Wilkinson, and Jason R. Andrews. A Nanopore Sequencing-based Pharmacogenomic Panel to Personalize Tuberculosis Drug Dosing. *American Journal of Respiratory and Critical Care Medicine*, 2024.

[22] Michelle M. Clark, Amber Hildreth, Sergey Batalov, Yan Ding, Shimul Chowdhury, Kelly Watkins, Katarzyna Ellsworth, Brandon Camp, Cyrielle I. Kint, Calum Yacoubian, Lauge Farnaes, Matthew N. Bainbridge, Curtis Beebe, Joshua J. A. Braun, Margaret Bray, Jeanne Carroll, Julie A. Cakici, Sara A. Caylor, Christina Clarke, Mitchell P. Creed, Jennifer Friedman, Alison Frith, Richard Gain, Mary Gaughran, Shauna George, Sheldon Gilmer, Joseph Gleeson, Jeremy Gore, Haiying Grunenwald, Raymond L. Hovey, Marie L. Janes, Kejia Lin, Paul D. McDonagh, Kyle McBride, Patrick Mulrooney, Shareef Nahas, Daeheon Oh, Albert Oriol, Laura Puckett, Zia Rady, Martin G. Reese, Julie Ryu, Lisa Salz, Erica Sanford, Lawrence Stewart, Nathaly Sweeney, Mari Tokita, Luca Van Der Kraan, Sarah White, Kristen Wigby, Brett Williams, Terence Wong, Meredith S. Wright, Catherine Yamada, Peter Schols, John Reynders, Kevin Hall, David Dimmock, Narayanan Veeraraghavan, Thomas Defay, and Stephen F. Kingsmore. Diagnosis of Genetic Diseases in Seriously Ill Children by Rapid Whole-genome Sequencing and Automated Phenotyping and Interpretation. *Science Translational Medicine*, 2019.

[23] Lauge Farnaes, Amber Hildreth, Nathaly M. Sweeney, Michelle M. Clark, Shimul Chowdhury, Shareef Nahas, Julie A. Cakici, Wendy Benson, Robert H. Kaplan, Richard Kronick, Matthew N. Bainbridge, Jennifer Friedman, Jeffrey J. Gold, Yan Ding, Narayanan Veeraraghavan, David Dimmock, and Stephen F. Kingsmore. Rapid Whole-genome Sequencing Decreases Infant Morbidity and Cost of Hospitalization. *NPJ Genomic Medicine*, 2018.

[24] Nathaly M. Sweeney, Shareef A. Nahas, Shimul Chowdhury, Sergey Batalov, Michelle Clark, Sara Caylor, Julie Cakici, John J. Nigro, Yan Ding, Narayanan Veeraraghavan, Charlotte Hobbs, David Dimmock, and Stephen F. Kingsmore. Rapid Whole Genome Sequencing Impacts Care and Resource Utilization in Infants with Congenital Heart Disease. *NPJ Genomic Medicine*, 2021.

[25] Mauricio Flores, Gustavo Glusman, Kristin Brogaard, Nathan D Price, and Leroy Hood. P4 Medicine: How Systems Medicine Will Transform the Healthcare Sector and Society. *Personalized Medicine*, 2013.

[26] Geoffrey S Ginsburg and Huntington F Willard. Genomic and Personalized Medicine: Foundations and Applications. *Translational Research*, 2009.

[27] Lynda Chin, Jannik N Andersen, and P Andrew Futreal. Cancer Genomics: From Discovery Science to Personalized Medicine. *Nature Medicine*, 2011.

[28] Euan A Ashley. Towards Precision Medicine. *Nature Reviews Genetics*, 2016.

[29] Tim Dunn, Harisankar Sadasivan, Jack Wadden, Kush Goliya, Kuan-Yu Chen, David Blaauw, Reetuparna Das, and Satish Narayanasamy. SquiggleFilter: An accelerator for portable virus detection. In *MICRO*, 2021.

[30] Claire Bertelli and Gilbert Greub. Rapid bacterial genome sequencing: methods and applications in clinical microbiology. *Clinical Microbiology and Infection*, 2013.

[31] Armando Arias, Simon J. Watson, Danny Asogun, Ekaete Alice Tobin, Jia Lu, My V. T. Phan, Umaru Jah, Raoul Emeric Guetiya Wadoum, Luke Meredith, Lucy Thorne, Sarah Caddy, Alimamy Tarawalie, Pinky Langat, Gytis Dudas, Nuno R. Faria, Simon Dellicour, Abdul Kamara, Brima Kargbo, Brima Osaio Kamara, Sahr Gevao, Daniel Cooper, Matthew Newport, Peter Horby, Jake Dunning, Foday Sahr, Tim Brooks, Andrew J.H. Simpson, Elisabetta Groppelli, Guoying Liu, Nisha Mulakken, Kate Rhodes, James Akpablie, Zabulon Yoti, Margaret Lamunu, Esther Vitto, Patrick Otim, Collins Owilli, Isaac Boateng, Lawrence Okoror, Emmanuel Omomoh, Jennifer Oyakhilome, Racheal Omiunu, Ighodalo Yemisis, Donatus Adomeh, Solomon Ehikhiametalor, Patience Akhilomen, Chris Aire, Andreas Kurth, Nicola Cook, Jan Baumann, Martin Gabriel, Roman Wölfel, Antonino Di Caro, Miles W. Carroll, Stephan Günther, John Redd, Dhamari Naidoo, Oliver G. Pybus, Andrew Rambaut, Paul Kellam, Ian Goodfellow, and Matthew Cotten. Rapid outbreak sequencing of Ebola virus in Sierra Leone identifies transmission chains linked to sporadic cases. *Virus Evolution*, 2016.

[32] Jessica Comin, Armando Chaure, Alberto Cebollada, Daniel Ibarz, Jesús Viñuelas, María Asunción Vitoria, María José Iglesias, and Sofía Samper. Investigation of a rapidly spreading tuberculosis outbreak using whole-genome sequencing. *Infection, Genetics and Evolution*, 2020.

[33] Joshua Quick, Nicholas J. Loman, Sophie Duraffour, Jared T. Simpson, Ettore Severi, Lauren Cowley, Joseph Akoi Bore, Raymond Koundouno, Gytis Dudas, Amy Mikhail, Nobila Ouédraogo, Babak Afrough, Amadou Bah, Jonathan H. J. Baum, Beate Becker-Ziaja, Jan Peter Boettcher, Mar Cabeza-Cabrerizo, Álvaro Camino-Sánchez, Lisa L. Carter, Juliane Doerrbecker, Theresa Enkirch, Isabel García Dorival, Nicole Hetzelt, Julia Hinzmann, Tobias Holm, Liana Eleni Kafetzopoulou, Michel Koropogui, Abigael Kosgey, Eeva Kuisma, Christopher H. Logue, Antonio Mazzarelli, Sarah Meisel, Marc Mertens, Janine Michel, Didier Ngabo, Katja Nitzsche, Elisa Pallasch, Livia Victoria Patrono, Jasmine Portmann, Johanna Gabriella Repits, Natasha Y. Rickett, Andreas Sachse, Katrin Singethan, Inês Vitoriano, Rahel L. Yemanaberhan, Elsa G. Zekeng, Trina Racine, Alexander Bello, Amadou Alpha Sall, Ousmane Faye, Oumar Faye, N'Faly Magassouba, Cecelia V. Williams, Victoria Amburgey, Linda Winona, Emily Davis, Jon Gerlach, Frank Washington, Vanessa Monteil, Marine Jourdain, Marion Bererd, Alimou Camara, Hermann Somlare, Abdoulaye Camara, Marianne Gerard, Guillaume Bado, Bernard Baillet, Déborah Delaune, Koumpingnin Yacouba Nebie, Abdoulaye Diarra, Yacouba Savane, Raymond Bernard Pallawo, Giovanna Jaramillo Gutierrez, Natacha Milhano, Isabelle Roger, Christopher J. Williams, Facinet Yattara, Kuiama Lewandowski, James Taylor, Phillip Rachwal, Daniel J. Turner, Georgios Pollakis, Julian A. Hiscox, David A. Matthews, Matthew K. O' Shea, Andrew McD. Johnston, Duncan Wilson, Emma Hutley, Erasmus Smit, Antonino Di Caro, Roman Wölfel, Kilian Stoecker, Erna

Fleischmann, Martin Gabriel, Simon A. Weller, Lamine Koivogui, Boubacar Diallo, Sakoba Keïta, Andrew Rambaut, Pierre Formenty, Stephan Günther, and Miles W. Carroll. Real-time, portable genome sequencing for Ebola surveillance. *Nature*, 2016.

[34] Esther R. Robinson, Timothy M. Walker, and Mark J. Pallen. Genomics and outbreak investigation: from sequence to consequence. *Genome Medicine*, 2013.

[35] Pierre-Edouard Fournier, Gregory Dubourg, and Didier Raoult. Clinical detection and characterization of bacterial pathogens in the genomics era. *Genome Medicine*, 2014.

[36] Claudio U. Köser, Matthew J. Ellington, Edward J. P. Cartwright, Stephen H. Gillespie, Nicholas M. Brown, Mark Farrington, Matthew T. G. Holden, Gordon Dougan, Stephen D. Bentley, Julian Parkhill, and Sharon J. Peacock. Routine Use of Microbial Whole Genome Sequencing in Diagnostic and Public Health Microbiology. *PLOS Pathogens*, 2012.

[37] Marc Eloit and marc lecuit. The diagnosis of infectious diseases by whole genome next generation sequencing: a new era is opening. *Frontiers in Cellular and Infection Microbiology*, 2014.

[38] Gardy Jennifer L., Johnston James C., Sui Shannan J. Ho, Cook Victoria J., Shah Lena, Brodkin Elizabeth, Rempel Shirley, Moore Richard, Zhao Yongjun, Holt Robert, Varhol Richard, Birol Inanc, Lem Marcus, Sharma Meenu K., Elwood Kevin, Jones Steven J.M., Brinkman Fiona S.L., Brunham Robert C., and Tang Patrick. Whole-Genome Sequencing and Social-Network Analysis of a Tuberculosis Outbreak. *New England Journal of Medicine*, 2011.

[39] Taylor Angela J., Lappi Victoria, Wolfgang William J., Lapierre Pascal, Palumbo Michael J., Medus Carlota, and Boxrud David. Characterization of Foodborne Outbreaks of Salmonella enterica Serovar Enteritidis with Whole-Genome Sequencing Single Nucleotide Polymorphism-Based Analysis for Surveillance and Outbreak Detection. *Journal of Clinical Microbiology*, 2015.

[40] Quainoo Scott, Coolen Jordy P. M., van Hijum Sacha A. F. T., Huynen Martijn A., Melchers Willem J. G., van Schaik Willem, and Wertheim Heiman F. L. Whole-Genome Sequencing of Bacterial Pathogens: the Future of Nosocomial Outbreak Analysis. *Clinical Microbiology Reviews*, 2017.

[41] Goldberg Brittany, Sichtig Heike, Geyer Chelsie, Ledeboer Nathan, and Weinstock George M. Making the Leap from Research Laboratory to Clinic: Challenges and Opportunities for Next-Generation Sequencing in Infectious Disease Diagnostics. *mBio*, 2015.

[42] John M. Besser, Heather A. Carleton, Eija Trees, Steven G. Stroika, Kelley Hise, Matthew Wise, and Peter Gerner-Smidt. Interpretation of Whole-Genome Sequencing for Enteric Disease Surveillance and Outbreak Investigation. *Foodborne Pathogens and Disease*, 2019.

[43] Weiwei Li, Qingpo Cui, Li Bai, Ping Fu, Haihong Han, Jikai Liu, and Yunchang Guo. Application of Whole-Genome Sequencing in the National Molecular Tracing Network for Foodborne Disease Surveillance in China. *Foodborne Pathogens and Disease*, 2021.

[44] Yinhua Deng, Min Jiang, Patrick S.L. Kwan, Chao Yang, Qiongcheng Chen, Yiman Lin, Yaqun Qiu, Yinghui Li, Xiaolu Shi, Liqiang Li, Yujun Cui, Qun Sun, and Qinghua Hu. Integrated Whole-Genome Sequencing Infrastructure for Outbreak Detection and Source Tracing of Salmonella enterica Serotype Enteritidis. *Foodborne Pathogens and Disease*, 2021.

[45] Jason C. Kwong, Nadine McCallum, Vitali G. Sintchenko, and Benjamin P. Howden. Whole genome sequencing in clinical and public health microbiology. *Pathology*, 2015.

[46] Ruud H. Deurenberg, Erik Bathoorn, Monika A. Chlebowicz, Natacha Couto, Mithila Ferdous, Silvia García-Cobos, Anna M.D. Kooistra-Smid, Erwin C. Raangs, Sigrid Rosema, Alida C.M. Veloo, Kai Zhou, Alexander W. Friedrich, and John W.A. Rossen. Application of next generation sequencing in clinical microbiology and infection prevention. *Journal of Biotechnology*, 2017.

[47] Patrick Tang, Matthew A. Croxen, Mohammad R. Hasan, William W.L. Hsiao, and Linda M. Hoang. Infection control in the new age of genomic epidemiology. *American Journal of Infection Control*, 2017.

[48] Nicholas J Croucher and Xavier Didelot. The application of genomics to tracing bacterial pathogen transmission. *Host–microbe interactions: bacteria • Genomics*, 2015.

[49] Joshua S. Bloom, Laila Sathe, Chetan Munugala, Eric M. Jones, Molly Gasperini, Nathan B. Lubock, Fauna Yarza, Erin M. Thompson, Kyle M. Kovary, Jimin Park, Dawn Marquette, Stephania Kay, Mark Lucas, TreQuan Love, A. Sina Booeshaghi, Oliver F. Brandenberg, Longhua Guo, James Boocock, Myles Hochman, Scott W. Simpkins, Isabella Lin, Nathan LaPierre, Duke Hong, Yi Zhang, Gabriel Oland, Bianca Judy Choe, Sukantha Chandrasekaran, Evann E. Hilt, Manish J. Butte, Robert Damoiseaux, Clifford Kravit, Aaron R. Cooper, Yi Yin, Lior Pachter, Omai B. Garner, Jonathan Flint, Eleazar Eskin, Chongyuan Luo, Sriram Kosuri, Leonid Kruglyak, and Valerie A. Arboleda. Massively Scaled-up Testing for SARS-CoV-2 RNA via Next-generation Sequencing of Pooled and Barcoded Nasal and Saliva Samples. *Nature Biomedical Engineering*, 2021.

[50] Ramesh Yelagandula, Aleksandr Bykov, Alexander Vogt, Robert Heinen, Ezgi Özkan, Marcus Martin Strobl, Juliane Christina Baar, Kristina Uzunova, Bence Hajdusits, Darja Kordic, Erna Suljic, Amina Kurtovic-Kozaric, Sebija Izetbegovic, Justine Schaeffer, Peter Hufnagl, Alexander Zoufaly, Tamara Seitz, Mariam Al-Rawi, Stefan Ameres, Juliane Baar, Benedikt Bauer, Nikolaus Beer, Katharina Bergauer, Wolfgang Binder, Claudia Blaukopf, Boril Bochev, Julius Brennecke, Selina Brinnich, Aleksandra Bundalo, Meinrad Busslinger, Tim Clausen, Geert de Vries, Marcus Dekens, David Drechsel, Zuzana Dzupinkova, Michaela Eckmann-Mader, Michaela Fellner, Thomas Fellner, Laura Fin, Bianca Valeria Gapp, Gerlinde Grabmann, Irina Grishkovskaya, Astrid Hagelkruys, Dominik Handler, David Haselbach, Louisa Hempel, Louisa Hill, David Hoffmann, Stefanie Horer, Harald Isemann, Robert Kalis, Max Kellner, Juliane Kley, Thomas Köcher, Alwin Köhler, Christian Krauditsch, Sabina Kula, Sonja Lang, Richard Latham, Marie-Christin Leitner, Thomas Leonard, Dominik Lindenhofer, Raphael Arthur Manzenreither, Martin Matl, Karl Mechtler, Anton Meinhart, Stefan Mereiter, Thomas Micheler, Paul Moeseneder, Tobias Neumann, Simon Nimpf, Magnus Nordborg, Egon Ogris, Michaela Pagani, Andrea Pauli, Jan-Michael Peters, Petra Pjevac, Clemens Plaschka, Martina Rath, Daniel Reumann, Sarah Rieser, Marianne Rocha-Hasler, Alan Rodriguez, Nathalie Ropek, James Julian Ross, Harald Scheuch, Karina Schindler, Clara Schmidt, Hannes Schmidt, Jakob Schnabl, Stefan Schüchner, Tanja Schwickert, Andreas Sommer, Daniele Soldoroni, Johannes Stadlmann, Peter Steinlein, Marcus Strobl, Simon Strobl, Qiong Sun, Wen Tang, Linda Trübestein, Johanna Trupke, Christian Umkehrer, Sandor Urmosi-Incze, Gijs Versteeg, Vivien Vogt, Michael Wagner, Martina Weissenboeck, Barbara Werner, Johannes Zuber, Manuela Födinger, Franz Allerberger, Alexander Stark, Luisa Cochella, Ulrich Elling, and VCDI. Multiplexed Detection of SARS-CoV-2 and Other Respiratory Infections in High Throughput by SARSeq. *Nature Communications*, 2021.

[51] Vien Thi Minh Le and Binh An Diep. Selected Insights from Application of Whole Genome Sequencing for Outbreak Investigations. *Current Opinion in Critical Care*, 2013.

[52] Vlad Nikolayevskyy, Katharina Kranzer, Stefan Niemann, and Francis Drobniewski. Whole Genome Sequencing of Mycobacterium Tuberculosis for Detection of Recent Transmission and Tracing Outbreaks: A Systematic Review. *Tuberculosis*, 2016.

[53] Shaofu Qiu, Peng Li, Hongbo Liu, Yong Wang, Nan Liu, Chengyi Li, Shenlong Li, Ming Li, Zhengjie Jiang, Huandong Sun, Ying Li, Jing Xie, Chaojie Yang, Jian Wang, Hao Li, Shengjie Yi, Zhihao Wu, Leili Jia, Ligui Wang, Rongzhang Hao, Yansong Sun, Liuyu Huang, Hui Ma, Zhengquan Yuan, and Hongbin Song. Whole-genome Sequencing for Tracing the Transmission Link between Two ARD Outbreaks Caused by A Novel HAdV Serotype 7 Variant, China. *Scientific Reports*, 2015.

[54] Carol A Gilchrist, Stephen D Turner, Margaret F Riley, William A Petri, and Erik L Hewlett. Whole-genome Sequencing in Outbreak Analysis. *Clinical Microbiology Reviews*, 2015.

[55] Michael S. Lawrence, Petar Stojanov, Paz Polak, Gregory V. Kryukov, Kristian Cibulskis, Andrey Sivachenko, Scott L. Carter, Chip Stewart, Craig H. Mermel, Steven A. Roberts, Adam Kiezun, Peter S. Hammerman, Aaron McKenna, Yotam Drier, Lihua Zou, Alex H. Ramos, Trevor J. Pugh, Nicolas Stransky, Elena Helman, Jaegil Kim, Carrie Sougnez, Lauren Ambrogio, Elizabeth Nickerson, Erica Shefler, Maria L. Cortés, Daniel Auclair, Gordon Saksena, Douglas Voet, Michael Noble, Daniel DiCara, Pei Lin, Lee Lichtenstein, David I. Heiman, Timothy Fennell, Marcin Imielinski, Bryan Hernandez, Eran Hodis, Sylvan Baca, Austin M. Dulak, Jens Lohr, Dan-Avi Landau, Catherine J. Wu, Jorge Melendez-Zajgla, Alfredo Hidalgo-Miranda, Amnon Koren, Steven A. McCarroll, Jaume Mora, Ryan S. Lee, Brian Crompton, Robert Onofrio, Melissa Parkin, Wendy Winckler, Kristin Ardlie, Stacey B. Gabriel, Charles W. M. Roberts, Jaclyn A. Biegel, Kimberly Stegmaier, Adam J. Bass, Levi A. Garraway, Matthew Meyerson, Todd R. Golub, Dmitry A. Gordenin, Shamil Sunyaev, Eric S. Lander, and Gad Getz. Mutational heterogeneity in cancer and the search for new cancer-associated genes. *Nature*, 2013.

[56] Bert Vogelstein, Nickolas Papadopoulos, Victor E. Velculescu, Shibin Zhou, Luis A. Diaz, and Kenneth W. Kinzler. Cancer Genome Landscapes. *Science*, 2013.

[57] Daniel Ramsköld, Shujun Luo, Yu-Chieh Wang, Robin Li, Qiaolin Deng, Omid R Faridani, Gregory A Daniels, Irina Khrebtukova, Jeanne F Loring, Louise C Laurent, Gary P Schroth, and Rickard Sandberg. Full-length mRNA-Seq from single-cell levels of RNA and individual circulating tumor cells. *Nature Biotechnology*, 2012.

[58] Timour Baslan and James Hicks. Unravelling biology and shifting paradigms in cancer with single-cell sequencing. *Nature Reviews Cancer*, 2017.

[59] Ehud Shapiro, Tamir Biezuner, and Sten Linnarsson. Single-cell sequencing-based technologies will revolutionize whole-organism science. *Nature Reviews Genetics*, 2013.

[60] Yoshitaka Sakamoto, Sarun Sereewattanawoot, and Ayako Suzuki. A new era of long-read sequencing for cancer genomics. *Journal of Human Genetics*, 2020.

[61] Qingzhu Jia, Han Chu, Zheng Jin, Haixia Long, and Bo Zhu. High-throughput single-cell sequencing in cancer research. *Signal Transduction and Targeted Therapy*, 2022.

[62] Devon A. Lawson, Kai Kessenbrock, Ryan T. Davis, Nicholas Pervolarakis, and Zena Werb. Tumour heterogeneity and metastasis at single-cell resolution. *Nature Cell Biology*, 2018.

[63] Chuang Liu, Qiangqiang Shi, Xiangang Huang, Seyoung Koo, Na Kong, and Wei Tao. mRNA-based cancer therapeutics. *Nature Reviews Cancer*, 2023.

[64] Bram Van de Sande, Joon Sang Lee, Euphemia Mutasa-Gottgens, Bart Naughton, Wendi Bacon, Jonathan Manning, Yong Wang, Jack Pollard, Melissa Mendez, Jon Hill, Namit Kumar, Xiaohong Cao, Xiao Chen, Mugdha Khaladkar, Ji Wen, Andrew Leach, and Edgardo Ferran. Applications of single-cell RNA sequencing in drug discovery and development. *Nature Reviews Drug Discovery*, 2023.

[65] Debyani Chakravarty and David B. Solit. Clinical cancer genomic profiling. *Nature Reviews Genetics*, 2021.

[66] Isidro Cortés-Ciriano, Doga C. Gulhan, Jake June-Koo Lee, Giorgio E. M. Melloni, and Peter J. Park. Computational analysis of cancer genome sequencing data. *Nature Reviews Genetics*, 2022.

[67] Ira W. Deveson, Binsheng Gong, Kevin Lai, Jennifer S. LoCoco, Todd A. Richmond, Jeoffrey Schageman, Zhihong Zhang, Natalia Novoradovskaya, James C. Willey, Wendell Jones, Rebecca Kusko, Guangchun Chen, Bindu Swapna Madala, James Blackburn, Igor Stevanovski, Ambica Bhandari, Devin Close, Jeffrey Conroy, Michael Hubank, Narasimha Marella, Piotr A. Mieczkowski, Fujun Qiu, Robert Sebra, Daniel Stetson, Lihyun Sun, Philippe Szankasi, Haowen Tan, Lin-ya Tang, Hanane Arib, Hunter Best, Blake Burgher, Pierre R. Bushel, Fergal Casey, Simon Cawley, Chia-Jung Chang, Jonathan Choi, Jorge Dinis, Daniel Duncan, Agda Karina Eterovic, Liang Feng, Abhisek Ghosal, Kristina Giorda, Sean Glenn, Scott Happe, Nathan Haseley, Kyle Horvath, Li-Yuan Hung, Mirna Jarosz, Garima Kushwaha,

Dan Li, Quan-Zhen Li, Zhiguang Li, Liang-Chun Liu, Zhichao Liu, Charles Ma, Christopher E. Mason, Dalila B. Megherbi, Tom Morrison, Carlos Pabón-Peña, Mehdi Pirooznia, Paula Z. Proszek, Amelia Raymond, Paul Rindler, Rebecca Ringler, Andreas Scherer, Rita Shaknovich, Tieliu Shi, Melissa Smith, Ping Song, Maya Strahl, Venkat J. Thodima, Nikola Tom, Suman Verma, Jiashi Wang, Leihong Wu, Wenzhong Xiao, Chang Xu, Mary Yang, Guangliang Zhang, Sa Zhang, Yilin Zhang, Leming Shi, Weida Tong, Donald J. Johann, Timothy R. Mercer, Joshua Xu, and SEQC2 Oncopanel Sequencing Working Group. Evaluating the analytical validity of circulating tumor DNA sequencing assays for precision oncology. *Nature Biotechnology*, 2021.

[68] Wenming Xiao, Luyao Ren, Zhong Chen, Li Tai Fang, Yongmei Zhao, Justin Lack, Meijian Guan, Bin Zhu, Erich Jaeger, Liz Kerrigan, Thomas M. Blomquist, Tiffany Hung, Marc Sultan, Kenneth Idler, Charles Lu, Andreas Scherer, Rebecca Kusko, Malcolm Moos, Chunlin Xiao, Stephen T. Sherry, Ogan D. Abaan, Wanqiu Chen, Xin Chen, Jessica Nordlund, Ulrika Liljedahl, Roberta Maestro, Maurizio Polano, Jiri Drabek, Petr Vojta, Sulev Kõks, Ene Reimann, Bindu Swapna Madala, Timothy Mercer, Chris Miller, Howard Jacob, Tiffany Truong, Ali Moshrefi, Aparna Natarajan, Ana Granat, Gary P. Schroth, Rasika Kalamegham, Eric Peters, Virginie Petitjean, Ashley Walton, Tsai-Wei Shen, Keyur Talsania, Cristobal Juan Vera, Kurt Langenbach, Maryellen de Mars, Jennifer A. Hipp, James C. Willey, Jing Wang, Jyoti Shetty, Yuliya Kriga, Arati Raziuddin, Bao Tran, Yuanting Zheng, Ying Yu, Margaret Cam, Parthav Jailwala, Cu Nguyen, Daoud Meerzaman, Qingrong Chen, Chunhua Yan, Ben Ernest, Urvashi Mehra, Roderick V. Jensen, Wendell Jones, Jian-Liang Li, Brian N. Papas, Mehdi Pirooznia, Yun-Ching Chen, Fayaz Seifuddin, Zhipan Li, Xuelu Liu, Wolfgang Resch, Jingya Wang, Leihong Wu, Gokhan Yavas, Corey Miles, Baitang Ning, Weida Tong, Christopher E. Mason, Eric Donaldson, Samir Lababidi, Louis M. Staudt, Zivana Tezak, Huixiao Hong, Charles Wang, and Leming Shi. Toward best practice in cancer mutation detection with whole-genome and whole-exome sequencing. *Nature Biotechnology*, 2021.

[69] Kelly L. Bolton, Ryan N. Ptashkin, Teng Gao, Lior Braunstein, Sean M. Devlin, Daniel Kelly, Minal Patel, Antonin Berthon, Aijazuddin Syed, Mariko Yabe, Catherine C. Coombs, Nicole M. Caltabellotta, Mike Walsh, Kenneth Offit, Zsofia Stadler, Diana Mandelker, Jessica Schulman, Akshar Patel, John Philip, Elsa Bernard, Gunes Gundem, Juan E. Arango Ossa, Max Levine, Juan S. Medina Martinez, Noushin Farnoud, Dominik Glodzik, Sonya Li, Mark E. Robson, Choonsik Lee, Paul D. P. Pharoah, Konrad H. Stopsack, Barbara Spitzer, Simon Mantha, James Fagin, Laura Boucai, Christopher J. Gibson, Benjamin L. Ebert, Andrew L. Young, Todd Druley, Koichi Takahashi, Nancy Gillis, Markus Ball, Eric Padron, David M. Hyman, Jose Baselga, Larry Norton, Stuart Gardos, Virginia M. Klimek, Howard Scher, Dean Bajorin, Eder Paraiso, Ryma Benayed, Maria E. Arcila, Marc Ladanyi, David B. Solit, Michael F. Berger, Martin Tallman, Montserrat Garcia-Closas, Nilanjan Chatterjee, Luis A. Diaz, Ross L. Levine, Lindsay M. Morton, Ahmet Zehir, and Elli Papaemmanuil. Cancer therapy shapes the fitness landscape of clonal hematopoiesis. *Nature Genetics*, 2020.

[70] Joseph D. Szustakowski, Suganthi Balasubramanian, Erika Kvikstad, Shareef Khalid, Paola G. Bronson, Ariella Sasson, Emily Wong, Daren Liu, J. Wade Davis, Carolina Haefliger, A. Katrina Loomis, Rajesh Mikkilineni, Hyun Ji Noh, Samir Wadhawan, Xiaodong Bai, Alicia Hawes, Olga Krasheninina, Ricardo Ulloa, Alex E. Lopez, Erin N. Smith, Jeffrey F. Waring, Christopher D. Whelan, Ellen A. Tsai, John D. Overton, William J. Salerno, Howard Jacob, Sandor Szalma, Heiko Runz, Gregory Hinkle, Paul Nioi, Slavé Petrovski, Melissa R. Miller, Aris Baras, Lyndon J. Mitnaul, Jeffrey G. Reid, Oleg Moiseyenko, Carlos Rios, Saurabh Saha, Goncalo Abecasis, Nilanjana Banerjee, Christina Beechert, Boris Boutkov, Michael Cantor, Giovanni Coppola, Aris Economides, Gisu Eom, Caitlin Forsythe, Erin D. Fuller, Zhenhua Gu, Lukas Habegger, Marcus B. Jones, Rouel Lanche, Michael Lattari, Michelle LeBlanc, Dadong Li, Luca A. Lotta, Kia Manoochehri, Adam J. Mansfield, Evan K. Maxwell, Jason Mighty, Mrunali Nafde, Sean O'Keeffe, Max Orelus, Maria Sotiropoulos Padilla, Razvan Panea, Tommy Polanco, Manasi Pradhan, Ayesha Rasool, Thomas D. Schleicher, Deepika Sharma, Alan Shuldiner, Jeffrey C. Staples, Cristopher V. Van Hout, Louis Widom, Sarah E. Wolf, Sally John, Chia-Yen Chen, David Sexton, Varant Kupelian, Eric Marshall, Timothy Swan, Susan Eaton, Jimmy Z. Liu, Stephanie Loomis, Megan Jensen, Saranya Duraisamy, Jason Tetrault, David Merberg, Sunita Badola, Mark Reppell, Jason Grundstad, Xiuwen Zheng, Aimee M. Deaton, Margaret M. Parker, Lucas D. Ward, Alexander O. Flynn-Carroll, Caroline Austin, Ruth March, Menelas N. Pangalos, Adam Platt, Mike Snowden, Athena Matakidou, Sebastian Wasilewski, Quanli Wang, Sri Deevi, Keren Carss, Katherine Smith, Morten Sogaard, Xinli Hu, Xing Chen, Zhan Ye, and UKB-ESC Research Team. Advancing human genetics research and drug discovery through exome sequencing of the UK Biobank. *Nature Genetics*, 2021.

[71] Nicholas Navin and James Hicks. Future medical applications of single-cell sequencing in cancer. *Genome Medicine*, 2011.

[72] Mingye Hong, Shuang Tao, Ling Zhang, Li-Ting Diao, Xuanmei Huang, Shaohui Huang, Shu-Juan Xie, Zhen-Dong Xiao, and Hua Zhang. RNA sequencing: new technologies and applications in cancer research. *Journal of Hematology & Oncology*, 2020.

[73] Yalan Lei, Rong Tang, Jin Xu, Wei Wang, Bo Zhang, Jiang Liu, Xianjun Yu, and Si Shi. Applications of single-cell sequencing in cancer research: progress and perspectives. *Journal of Hematology & Oncology*, 2021.

[74] Yingying Han, Dan Wang, Lushan Peng, Tao Huang, Xiaoyun He, Junpu Wang, and Chunlin Ou. Single-cell sequencing: a promising approach for uncovering the mechanisms of tumor metastasis. *Journal of Hematology & Oncology*, 2022.

[75] Giulia Federici and Silvia Soddu. Variants of uncertain significance in the era of high-throughput genome sequencing: a lesson from breast and ovary cancers. *Journal of Experimental & Clinical Cancer Research*, 2020.

[76] Yijie Zhang, Dan Wang, Miao Peng, Le Tang, Jiawei Ouyang, Fang Xiong, Can Guo, Yanyan Tang, Yujuan Zhou, Qianjin Liao, Xu Wu, Hui Wang, Jianjun Yu, Yong Li, Xiaoling Li, Guiyuan Li, Zhaoyang Zeng, Yixin Tan, and Wei Xiong. Single-cell RNA sequencing in cancer research. *Journal of Experimental & Clinical Cancer Research*, 2021.

[77] Xianwen Ren, Boxi Kang, and Zemin Zhang. Understanding tumor ecosystems by single-cell sequencing: promises and limitations. *Genome Biology*, 2018.

[78] Liqing Tian, Yongjin Li, Michael N. Edmonson, Xin Zhou, Scott Newman, Clay McLeod, Andrew Thrasher, Yu Liu, Bo Tang, Michael C. Rusch, John Easton, Jing Ma, Eric Davis, Austyn Trull, J. Robert Michael, Karol Szlachta, Charles Mullighan, Suzanne J. Baker, James R. Downing, David W. Ellison, and Jinghui Zhang. CICERO: a versatile method for detecting complex and diverse driver fusions using cancer RNA sequencing data. *Genome Biology*, 2020.

[79] Eoghan R. Malone, Marc Oliva, Peter J. B. Sabatini, Tracy L. Stockley, and Lillian L. Siu. Molecular profiling for precision cancer therapies. *Genome Medicine*, 2020.

[80] Xiaoning Tang, Yongmei Huang, Jinli Lei, Hui Luo, and Xiao Zhu. The single-cell sequencing: new developments and medical applications. *Cell & Bioscience*, 2019.

[81] Darrell L. Ellsworth, Heather L. Blackburn, Craig D. Shriver, Shahrooz Rabizadeh, Patrick Soon-Shiong, and Rachel E. Ellsworth. Single-cell sequencing and tumorigenesis: improved understanding of tumor evolution and metastasis. *Clinical and Translational Medicine*, 2017.

[82] Yiming Zhong, Feng Xu, Jinhua Wu, Jeffrey Schubert, and Marilyn M. Li. Application of Next Generation Sequencing in Laboratory Medicine. *Annals of Laboratory Medicine*, 2021.

[83] Zsofia K. Stadler, Anna Maio, Debyani Chakravarty, Yelena Kemel, Margaret Sheehan, Erin Salo-Mullen, Kaitlyn Tkachuk, Christopher J. Fong, Bastien Nguyen, Amanda Erakky, Karen Cadoo, Ying Liu, Maria I. Carlo, Alicia Latham, Hongxin Zhang, Ritika Kundra, Shaleigh Smith, Jesse Galle, Carol Aghajanian, Nadeem Abu-Rustum, Anna Varghese, Eileen M. O'Reilly, Michael Morris, Wassim Abida, Michael Walsh, Alexander Drilon, Gowtham Jayakumaran, Ahmet Zehir, Marc Ladanyi, Ozge Ceyhan-Birsoy, David B. Solit, Nikolaus Schultz, Michael F. Berger, Diana Mandelker, Luis A. Diaz, Kenneth Offit, and Mark E. Robson. Therapeutic Implications of Germline Testing in Patients With Advanced Cancers. *Journal of Clinical Oncology*, 2021.

[84] Aaron C. Tan and Daniel S.W. Tan. Targeted Therapies for Lung Cancer Patients With Oncogenic Driver Molecular Alterations. *Journal of Clinical Oncology*, 2022.

[85] Andrea Degasperi, Xueqing Zou, Tauanne Dias Amarante, Andrea Martinez-Martinez, Gene Ching Chiek Koh, João M. L. Dias, Laura Heskin, Lucia Chmelova, Giuseppe Rinaldi, Valerie Ya Wen Wang, Arjun S. Nanda, Aaron Bernstein, Sophie E. Momen, Jamie Young, Daniel Perez-Gil, Yasin Memari, Cherif Badja, Scott Shooter, Jan Czarnecki, Matthew A. Brown, Helen R. Davies, Genomics England Research Consortium3†, Serena Nik-Zainal, J. C. Ambrose, P. Arumugam, R. Bevers, M. Bleda, F. Boardman-Pretty, C. R. Boustred, H. Brittain, M. J. Caulfield, G. C. Chan, T. Fowler, A. Giess, A. Hamblin, S. Henderson, T. J. P. Hubbard, R. Jackson, L. J. Jones, D. Kasperaviciute, M. Kayikci, A. Kousathanas, L. Lahnstein, S. E. A. Leigh, I. U. S. Leong, F. J. Lopez, F. Maleady-Crowe, M. McEntagart, F. Minneci, L. Moutsianas, M. Mueller, N. Murugaesu, A. C. Need, P. O'Donovan, C. A. Odhams, C. Patch, D. Perez-Gil, M. B. Pereira, J. Pullinger, T. Rahim, A. Rendon, T. Rogers, K. Savage, K. Sawant, R. H. Scott, A. Siddiq, A. Sieghart, S. C. Smith, A. Sosinsky, A. Stuckey, M. Tanguy, A. L. Taylor Tavares, E. R. A. Thomas, S. R. Thompson, A. Tucci, M. J. Welland, E. Williams, K. Witkowska, and S. M. Wood. Substitution mutational signatures in whole-genome–sequenced cancers in the UK population. *Science*, 2022.

[86] Junfen Xu, Yifeng Fang, Kelie Chen, Sen Li, Sangsang Tang, Yan Ren, Yixuan Cen, Weidong Fei, Bo Zhang, Yuanming Shen, and Weiguo Lu. Single-Cell RNA Sequencing Reveals the Tissue Architecture in Human High-Grade Serous Ovarian Cancer. *Clinical Cancer Research*, 2022.

[87] Peter Horak, Christoph Heining, Simon Kreutzfeldt, Barbara Hutter, Andreas Mock, Jennifer Hüllein, Martina Fröhlich, Sebastian Uhrig, Arne Jahn, Andreas Rump, Laura Gieldon, Lino Möhrmann, Dorothea Hanf, Veronica Teleanu, Christoph E. Heilig, Daniel B. Lipka, Michael Allgäuer, Leo Ruhnke, Andreas Laßmann, Volker Endris, Olaf Neumann, Roland Penzel, Katja Beck, Daniela Richter, Ulrike Winter, Stephan Wolf, Katrin Pfütze, Christina Geörg, Bettina Meißburger, Ivo Buchhalter, Marinela Augustin, Walter E. Aulitzky, Peter Hohenberger, Matthias Kroiss, Peter Schirmacher, Richard F. Schlenk, Ulrich Keilholz, Frederick Klauschen, Gunnar Folprecht, Sebastian Bauer, Jens Thomas Siveke, Christian H. Brandts, Thomas Kindler, Melanie Boerries, Anna L. Illert, Nikolas von Bubnoff, Philipp J. Jost, Karsten Spiekermann, Michael Bitzer, Klaus Schulze-Osthoff, Christof von Kalle, Barbara Klink, Benedikt Brors, Albrecht Stenzinger, Evelin Schröck, Daniel Hübschmann, Wilko Weichert, Hanno Glimm, and Stefan Fröhling. Comprehensive Genomic and Transcriptomic Analysis for Guiding Therapeutic Decisions in Patients with Rare Cancers. *Cancer Discovery*, 2021.

[88] Xiaoyan Zhang, Sadie L. Marjani, Zhaoyang Hu, Sherman M. Weissman, Xinghua Pan, and Shixiu Wu. Single-Cell Sequencing for Precise Cancer Research: Progress and Prospects. *Cancer Research*, 2016.

[89] Rossella Bruno and Gabriella Fontanini. Next Generation Sequencing for Gene Fusion Analysis in Lung Cancer: A Literature Review. *Diagnostics*, 2020.

[90] Antonella De Luca, Riziero Esposito Abate, Anna M. Rachiglio, Monica R. Maiello, Claudia Esposito, Clorinda Schettino, Francesco Izzo, Guglielmo Nasti, and Nicola Normanno. FGFR Fusions in Cancer: From Diagnostic Approaches to Therapeutic Intervention. *International Journal of Molecular Sciences*, 2020.

[91] Michael R. Waarts, Aaron J. Stonestrom, Young C. Park, and Ross L. Levine. Targeting mutations in cancer. *The Journal of Clinical Investigation*, 2022.

[92] Bora Lim, Yiyun Lin, and Nicholas Navin. Advancing Cancer Research and Medicine with Single-Cell Genomics. *Cancer Cell*, 2020.

[93] Ramon Colomer, Rebeca Mondejar, Nuria Romero-Laorden, Arantzazu Alfranca, Francisco Sanchez-Madrid, and Miguel Quintela-Fandino. When should we order a next generation sequencing test in a patient with cancer? *eClinicalMedicine*, 2020.

[94] Assieh Saadatpour, Shujing Lai, Guoji Guo, and Guo-Cheng Yuan. Single-Cell Analysis in Cancer Genomics. *Trends in Genetics*, 2015.

[95] Nazli Dizman, Zeynep E. Arslan, Matthew Feng, and Sumanta K. Pal. Sequencing Therapies for Metastatic Renal Cell Carcinoma. *Urologic Clinics*, 2020.

[96] Anton Buzdin, Maxim Sorokin, Andrew Garazha, Alexander Glusker, Alex Aleshin, Elena Poddubskaya, Marina Sekacheva, Ella Kim, Nurshat Gaifullin, Alf Giese, Alexander Seryakov, Pavel Rumiantsev, Sergey Moshkovskii, and Alexey Moiseev. RNA sequencing for research and diagnostics in clinical oncology. *Seminars in Cancer Biology*, 2020.

[97] Yi Xiao and Dihua Yu. Tumor microenvironment as a therapeutic target in cancer. *Pharmacology & Therapeutics*, 2021.

[98] Arun Nandwani, Shalu Rathore, and Malabika Datta. LncRNAs in cancer: Regulatory and therapeutic implications. *Cancer Letters*, 2021.

[99] Francesco Marchetti, Renato Cardoso, Connie L. Chen, George R. Douglas, Joanne Elloway, Patricia A. Escobar, Tod Harper, Robert H. Heflich, Darren Kidd, Anthony M. Lynch, Meagan B. Myers, Barbara L. Parsons, Jesse J. Salk, Raja S. Settivari, Stephanie L. Smith-Roe, Kristine L. Witt, Carole L. Yauk, Robert Young, Shaofei Zhang, and Sheroy Minocherhomji. Error-corrected next generation sequencing – Promises and challenges for genotoxicity and cancer risk assessment. *Mutation Research/Reviews in Mutation Research*, 2023.

[100] Xiao Chen, Han Zhu, Chun Qiao, Sishu Zhao, Lu Liu, Yan Wang, Huimin Jin, Sixuan Qian, and Yujie Wu. Next-generation sequencing reveals gene mutations landscape and clonal evolution in patients with acute myeloid leukemia. *Hematology*, 2021.

[101] Nicholas E. Navin. The first five years of single-cell cancer genomics and beyond. *Genome Research*, 2015.

[102] NIHR Global Health Research Unit on Genomic Surveillance of AMR. Whole-genome sequencing as part of national and international surveillance programmes for antimicrobial resistance: a roadmap. *BMJ Global Health*, 2020.

[103] Shanwei Tong, Luyao Ma, Jennifer Ronholm, William Hsiao, and Xiaonan Lu. Whole genome sequencing of Campylobacter in agri-food surveillance. *Current Opinion in Food Science*, 2021.

[104] The Arabidopsis Genome Initiative. Analysis of the genome sequence of the flowering plant Arabidopsis thaliana. *Nature*, 2000.

[105] Haocheng Zhu, Chao Li, and Caixia Gao. Applications of CRISPR–Cas in agriculture and plant biotechnology. *Nature Reviews Molecular Cell Biology*, 2020.

[106] Jae Young Choi, Zoe N. Lye, Simon C. Groen, Xiaoguang Dai, Priyesh Rughani, Sophie Zaaijer, Eoghan D. Harrington, Sissel Juul, and Michael D. Purugganan. Nanopore sequencing-based genome assembly and evolutionary genomics of circum-basmati rice. *Genome Biology*, 2020.

[107] Kristian A Stevens, Jill L Wegrzyn, Aleksey Zimin, Daniela Puiu, Marc Crepeau, Charis Cardeno, Robin Paul, Daniel Gonzalez-Ibeas, Maxim Koriabine, Ann E Holtz-Morris, Pedro J Martínez-García, Uzay U Sezen, Guillaume Marçais, Kathy Jermstad, Patrick E McGuire, Carol A Loopstra, John M Davis, Andrew Eckert, Pieter de Jong, James A Yorke, Steven L Salzberg, David B Neale, and Charles H Langley. Sequence of the Sugar Pine Megagenome. *Genetics*, 2016.

[108] Maria Doroteia Campos, Maria do Rosário Félix, Mariana Patanita, Patrick Materatski, and Carla Varanda. High throughput sequencing unravels tomato-pathogen interactions towards a sustainable plant breeding. *Horticulture Research*, 2021.

[109] Caixia Gao. Genome engineering for crop improvement and future agriculture. *Cell*, 2021.

[110] Aalt Dirk Jan van Dijk, Gert Kootstra, Willem Kruijer, and Dick de Ridder. Machine learning in plant science and plant breeding. *iScience*, 2021.

[111] Yanqing Sun, Lianguang Shang, Qian-Hao Zhu, Longjiang Fan, and Longbiao Guo. Twenty years of plant genome sequencing: achievements and challenges. *Trends in Plant Science*, 2022.

[112] Kyung D. Kim, Yuna Kang, and Changsoo Kim. Application of Genomic Big Data in Plant Breeding: Past, Present, and Future. *Plants*, 2020.

[113] Mahendar Thudi, Ramesh Palakurthi, James C. Schnable, Annapurna Chitikineni, Susanne Dreisigacker, Emma Mace, Rakesh K. Srivastava, C. Tara Satyavathi, Damaris Odeny, Vijay K. Tiwari, Hon-Ming Lam, Yan Bin Hong, Vikas K. Singh, Guowei Li, Yunbi Xu, Xiaoping Chen, Sanjay Kaila, Henry Nguyen, Sobhana Sivasankar, Scott A. Jackson, Timothy J. Close, Wan Shubo, and Rajeev K. Varshney. Genomic resources in plant breeding for sustainable agriculture. *Journal of Plant Physiology*, 2021.

[114] Todd P Michael and Robert VanBuren. Building near-complete plant genomes. *Current Opinion in Plant Biology*, 2020.

[115] Yanting Shen, Guoan Zhou, Chengzhi Liang, and Zhixi Tian. Omics-based interdisciplinarity is accelerating plant breeding. *Current Opinion in Plant Biology*, 2022.

[116] Taha Shahroodi, Mahdi Zahedi, Can Firtina, Mohammed Alser, Stephan Wong, Onur Mutlu, and Said Hamdioui. Demeter: A fast and energy-efficient food profiler using hyperdimensional computing in memory. *IEEE Access*, 2022.

[117] Shiv Prasad, Lal Chand Malav, Jairam Choudhary, Sudha Kannojiya, Monika Kundu, Sandeep Kumar, and Ajar Nath Yadav. Soil Microbiomes for Healthy Nutrient Recycling. *Current Trends in Microbial Biotechnology for Sustainable Agriculture*, 2021.

[118] Martin Mascher, Murukarthick Jayakodi, Hyeonah Shim, and Nils Stein. Promises and challenges of crop translational genomics. *Nature*, 2024.

[119] Mona Schreiber, Murukarthick Jayakodi, Nils Stein, and Martin Mascher. Plant pangenomes for crop improvement, biodiversity and evolution. *Nature Reviews Genetics*, 2024.

[120] Aneta K. Urbanek, Waldemar Rymowicz, and Aleksandra M. Mirończuk. Degradation of plastics and plastic-degrading bacteria in cold marine habitats. *Applied Microbiology and Biotechnology*, 2018.

[121] Robert C. Edgar, Jeff Taylor, Victor Lin, Tomer Altman, Pierre Barbera, Dmitry Meleshko, Dan Lohr, Gherman Novakovsky, Benjamin Buchfink, Basem Al-Shayeb, Jillian F. Banfield, Marcos de la Peña, Anton Korobeynikov, Rayan Chikhi, and Artem Babaian. Petabase-scale sequence alignment catalyses viral discovery. *Nature*, 2022.

[122] Lucas Paoli, Hans-Joachim Ruscheweyh, Clarissa C. Forneris, Florian Hubrich, Satria Kautsar, Agneya Bhushan, Alessandro Lotti, Quentin Clayssen, Guillem Salazar, Alessio Milanese, Charlotte I. Carlström, Chrysa Papadopoulou, Daniel Gehrig, Mikhail Karasikov, Harun Mustafa, Martin Larralde, Laura M. Carroll, Pablo Sánchez, Ahmed A. Zayed, Dylan R. Cronin, Silvia G. Acinas, Peer Bork, Chris Bowler, Tom O. Delmont, Josep M. Gasol, Alvar D. Gossert, André Kahles, Matthew B. Sullivan, Patrick Wincker, Georg Zeller, Serina L. Robinson, Jörn Piel, and Shinichi Sunagawa. Biosynthetic potential of the global ocean microbiome. *Nature*, 2022.

[123] Carolyn J. Hogg. Translating genomic advances into biodiversity conservation. *Nature Reviews Genetics*, 2024.

[124] Harris A. Lewin, Gene E. Robinson, W. John Kress, William J. Baker, Jonathan Coddington, Keith A. Crandall, Richard Durbin, Scott V. Edwards, Félix Forest, M. Thomas P. Gilbert, Melissa M. Goldstein, Igor V. Grigoriev, Kevin J. Hackett, David Haussler, Erich D. Jarvis, Warren E. Johnson, Aristides Patrinos, Stephen Richards, Juan Carlos Castilla-Rubio, Marie-Anne van Sluys, Pamela S. Soltis, Xun Xu, Huanming Yang, and Guojie Zhang. Earth BioGenome Project: Sequencing life for the future of life. *Proceedings of the National Academy of Sciences*, 2018.

[125] Minoru Kanehisa. Toward understanding the origin and evolution of cellular organisms. *Protein Science*, 2019.

[126] Jun Qing, Yi-De Meng, Feng He, Qing-Xin Du, Jian Zhong, Hong-Yan Du, Pan-Feng Liu, Lan-Ying Du, and Lu Wang. Whole genome re-sequencing reveals the genetic diversity and evolutionary patterns of Eucommia ulmoides. *Molecular Genetics and Genomics*, 2022.

[127] Patricia J. Wittkopp and Gizem Kalay. Cis-regulatory elements: molecular mechanisms and evolutionary processes underlying divergence. *Nature Reviews Genetics*, 2012.

[128] Irene Gallego Romero, Ilya Ruvinsky, and Yoav Gilad. Comparative studies of gene expression and the evolution of gene regulation. *Nature Reviews Genetics*, 2012.

[129] Xinchao Wang, Hu Feng, Yuxiao Chang, Chunlei Ma, Liyuan Wang, Xinyuan Hao, A'lun Li, Hao Cheng, Lu Wang, Peng Cui, Jiqiang Jin, Xiaobo Wang, Kang Wei, Cheng Ai, Sheng Zhao, Zhichao Wu, Youyong Li, Benying Liu, Guo-Dong Wang, Liang Chen, Jue Ruan, and Yajun Yang. Population sequencing enhances understanding of tea plant evolution. *Nature Communications*, 2020.

[130] Mark S. Hill, Pétra Vande Zande, and Patricia J. Wittkopp. Molecular and evolutionary processes generating variation in gene expression. *Nature Reviews Genetics*, 2021.

[131] Eeshit Dhaval Vaishnav, Carl G. de Boer, Jennifer Molinet, Moran Yassour, Lin Fan, Xian Adiconis, Dawn A. Thompson, Joshua Z. Levin, Francisco A. Cubillos, and Aviv Regev. The evolution, evolvability and engineering of gene regulatory DNA. *Nature*, 2022.

[132] Xingtan Zhang, Shuai Chen, Longqing Shi, Daping Gong, Shengcheng Zhang, Qian Zhao, Dongliang Zhan, Liette Vasseur, Yibin Wang, Jiaxin Yu, Zhenyang Liao, Xindan Xu, Rui Qi, Wenling Wang, Yunran Ma, Pengjie Wang, Naixing Ye, Dongna Ma, Yan Shi, Haifeng Wang, Xiaokai Ma, Xiangrui Kong, Jing Lin, Liufeng Wei, Yaying Ma, Ruoyu Li, Guiping Hu, Haifang He, Lin Zhang, Ray Ming, Gang Wang, Haibao Tang, and Minsheng You. Haplotype-resolved genome assembly provides insights into evolutionary history of the tea plant Camellia sinensis. *Nature Genetics*, 2021.

[133] Justin C. Fay and Patricia J. Wittkopp. Evaluating the role of natural selection in the evolution of gene regulation. *Heredity*, 2008.

[134] Aquillah M. Kanzi, James Emmanuel San, Benjamin Chimukangara, Eduan Wilkinson, Maryam Fish, Veron Ramsuran, and Tulio de Oliveira. Next Generation Sequencing and Bioinformatics Analysis of Family Genetic Inheritance. *Frontiers in Genetics*, 2020.

[135] Gregory A. Wray, Matthew W. Hahn, Ehab Abouheif, James P. Balhoff, Margaret Pizer, Matthew V. Rockman, and Laura A. Romano. The Evolution of Transcriptional Regulation in Eukaryotes. *Molecular Biology and Evolution*, 2003.

[136] Aiping Wu, Lulan Wang, Hang-Yu Zhou, Cheng-Yang Ji, Shang Zhou Xia, Yang Cao, Jing Meng, Xiao Ding, Sarah Gold, Taijiao Jiang, and Genhong Cheng. One year of SARS-CoV-2 evolution. *Cell Host & Microbe*, 2021.

[137] Sarah A. Signor and Sergey V. Nuzhdin. The Evolution of Gene Expression in cis and trans. *Trends in Genetics*, 2018.

[138] Andrew Whitehead and Douglas L. Crawford. Variation within and among species in gene expression: raw material for evolution. *Molecular Ecology*, 2006.

[139] Joseph D. Coolon, C. Joel McManus, Kraig R. Stevenson, Brenton R. Graveley, and Patricia J. Wittkopp. Tempo and mode of regulatory evolution in Drosophila. *Genome Research*, 2014.

[140] Hans Ellegren. Genome Sequencing and Population Genomics in Non-Model Organisms. *Trends in Ecology & Evolution*, 2014.

[141] Javier Prado-Martinez, Peter H. Sudmant, Jeffrey M. Kidd, Heng Li, Joanna L. Kelley, Belen Lorente-Galdos, Krishna R. Veeramah, August E. Woerner, Timothy D. O'Connor, Gabriel Santpere, Alexander Cagan, Christoph Theunert, Ferran Casals, Hafid Laayouni, Kasper Munch, Asger Hobolth, Anders E. Halager, Maika Malig, Jessica Hernandez-Rodriguez, Irene Hernando-Herraez, Kay Prüfer, Marc Pybus, Laurel Johnstone, Michael Lachmann, Can Alkan, Dorina Twigg, Natalia Petit, Carl Baker, Fereydoun Hormozdiari, Marcos Fernandez-Callejo, Marc Dabad, Michael L. Wilson, Laurie Stevison, Cristina Camprubí, Tiago Carvalho, Aurora Ruiz-Herrera, Laura Vives, Marta Mele, Teresa Abello, Ivanela Kondova, Ronald E. Bontrop, Anne Pusey, Felix Lankester, John A. Kiyang, Richard A. Bergl, Elizabeth Lonsdorf, Simon Myers, Mario Ventura, Pascal Gagneux, David Comas, Hans Siegismund, Julie Blanc, Lidia Agueda-Calpena, Marta Gut, Lucinda Fulton, Sarah A. Tishkoff, James C. Mullikin, Richard K. Wilson, Ivo G. Gut, Mary Katherine Gonder, Oliver A.

Ryder, Beatrice H. Hahn, Arcadi Navarro, Joshua M. Akey, Jaume Bertranpetit, David Reich, Thomas Mailund, Mikkel H. Schierup, Christina Hvilsom, Aida M. Andrés, Jeffrey D. Wall, Carlos D. Bustamante, Michael F. Hammer, Evan E. Eichler, and Tomas Marques-Bonet. Great Ape Genetic Diversity and Population History. *Nature*, 2013.

[142] Ana Prohaska, Fernando Racimo, Andrew J Schork, Martin Sikora, Aaron J Stern, Melissa Ilardo, Morten Erik Allentoft, Lasse Folkersen, Alfonso Buil, J Víctor Moreno-Mayar, Thorfinn Korneliussen, Daniel Geschwind, Andrés Ingason, Thomas Werge, Rasmus Nielsen, and Eske Willerslev. Human Disease Variation in the Light of Population Genomics. *Cell*, 2019.

[143] David Danko, Daniela Bezdan, Evan E. Afshin, Sofia Ahsanuddin, Chandrima Bhattacharya, Daniel J. Butler, Kern Rei Chng, Daisy Donnellan, Jochen Hecht, Katelyn Jackson, Katerina Kuchin, Mikhail Karasikov, Abigail Lyons, Lauren Mak, Dmitry Meleshko, Harun Mustafa, Beth Mutai, Russell Y. Neches, Amanda Ng, Olga Nikolayeva, Tatyana Nikolayeva, Eileen Png, Krista A. Ryon, Jorge L. Sanchez, Heba Shaaban, Maria A. Sierra, Dominique Thomas, Ben Young, Omar O. Abudayyeh, Josue Alicea, Malay Bhattacharyya, Ran Blekhman, Eduardo Castro-Nallar, Ana M. Cañas, Aspassia D. Chatziefthimiou, Robert W. Crawford, Francesca De Filippis, Youping Deng, Christelle Desnues, Emmanuel Dias-Neto, Marius Dybwad, Eran Elhaik, Danilo Ercolini, Alina Frolova, Dennis Gankin, Jonathan S. Gootenberg, Alexandra B. Graf, David C. Green, Iman Hajirasouliha, Jaden J.A. Hastings, Mark Hernandez, Gregorio Iraola, Soojin Jang, Andre Kahles, Frank J. Kelly, Kaymisha Knights, Nikos C. Kyrpides, Paweł P. Łabaj, Patrick K.H. Lee, Marcus H.Y. Leung, Per O. Ljungdahl, Gabriella Mason-Buck, Ken McGrath, Cem Meydan, Emmanuel F. Mongodin, Milton Ozorio Moraes, Niranjan Nagarajan, Marina Nieto-Caballero, Houtan Noushmehr, Manuela Oliveira, Stephan Ossowski, Olayinka O. Osuolale, Orhan Özcan, David Paez-Espino, Nicolás Rascovan, Hugues Richard, Gunnar Rätsch, Lynn M. Schriml, Torsten Semmler, Osman U. Sezerman, Leming Shi, Tieliu Shi, Rania Siam, Le Huu Song, Haruo Suzuki, Denise Syndercombe Court, Scott W. Tighe, Xinzhao Tong, Klas I. Udekwu, Juan A. Ugalde, Brandon Valentine, Dimitar I. Vassilev, Elena M. Vayndorf, Thirumalaisamy P. Velavan, Jun Wu, Maria M. Zambrano, Jifeng Zhu, Sibo Zhu, Christopher E. Mason, and International MetaSUB Consortium. A Global Metagenomic Map of Urban Microbiomes and Antimicrobial Resistance. *Cell*, 2021.

[144] Xavier Didelot, Rory Bowden, Daniel J Wilson, Tim EA Peto, and Derrick W Crook. Transforming clinical microbiology with bacterial genome sequencing. *Nature Reviews Genetics*, 2012.

[145] Simone Marini, Rodrigo A Mora, Christina Boucher, Noelle Robertson Noyes, and Mattia Prosperi. Towards routine employment of computational tools for antimicrobial resistance determination via high-throughput sequencing. *Briefings in Bioinformatics*, 2022.

[146] Charlotte J. Houldcroft, Mathew A. Beale, and Judith Breuer. Clinical and biological insights from viral genome sequencing. *Nature Reviews Microbiology*, 2017.

[147] Natasha Jansz and Geoffrey J. Faulkner. Viral genome sequencing methods: benefits and pitfalls of current approaches. *Biochemical Society Transactions*, 2024.

[148] Mikhail Karasikov, Harun Mustafa, Daniel Danciu, Oleksandr Kulkov, Marc Zimmermann, Christopher Barber, Gunnar Rätsch, and André Kahles. Efficient and accurate search in petabase-scale sequence repositories. *Nature*, 2025.

[149] Jarno N Alanko, Jaakko Vuohtoniemi, Tommi Mäklin, and Simon J Puglisi. Themisto: a scalable colored k-mer index for sensitive pseudoalignment against hundreds of thousands of bacterial genomes. *Bioinformatics*, 2023.

[150] Martin D Muggli, Alexander Bowe, Noelle R Noyes, Paul S Morley, Keith E Belk, Robert Raymond, Travis Gagie, Simon J Puglisi, and Christina Boucher. Succinct colored de Bruijn graphs. *Bioinformatics*, 2017.

[151] Jason Fan, Noor Pratap Singh, Jamshed Khan, Giulio Ermanno Pibiri, and Rob Patro. Fulgor: A Fast and Compact k-mer Index for Large-Scale Matching and Color Queries. In *WABI*, 2023.

[152] Vitor C Piro, Temesgen H Dadi, Enrico Seiler, Knut Reinert, and Bernhard Y Renard. ganon: precise metagenomics classification against large and up-to-date sets of reference sequences. *Bioinformatics*, 2020.

[153] Serghei Mangul and David Koslicki. Reference-free comparison of microbial communities via de Bruijn graphs. In *BCB*, 2016.

[154] Sergey Aganezov, Stephanie M. Yan, Daniela C. Soto, Melanie Kirsche, Samantha Zarate, Pavel Avdeyev, Dylan J. Taylor, Kishwar Shafin, Alaina Shumate, Chunlin Xiao, Justin Wagner, Jennifer McDaniel, Nathan D. Olson, Michael E. G. Sauria, Mitchell R. Vollger, Arang Rhie, Melissa Meredith, Skylar Martin, Joyce Lee, Sergey Koren, Jeffrey A. Rosenfeld, Benedict Paten, Ryan Layer, Chen-Shan Chin, Fritz J. Sedlazeck, Nancy F. Hansen, Danny E. Miller, Adam M. Phillippy, Karen H. Miga, Rajiv C. McCoy, Megan Y. Dennis, Justin M. Zook, and Michael C. Schatz. A complete reference genome improves analysis of human genetic variation. *Science*, 2022.

[155] Cristian Groza, Carl Schwendinger-Schreck, Warren A. Cheung, Emily G. Farrow, Isabelle Thiffault, Juniper Lake, William B. Rizzo, Gilad Evrony, Tom Curran, Guillaume Bourque, and Tomi Pastinen. Pangenome graphs improve the analysis of structural variants in rare genetic diseases. *Nature Communications*, 2024.

[156] Bonnie Berger and Yun William Yu. Navigating bottlenecks and trade-offs in genomic data analysis. *Nature Reviews Genetics*, 2023.

[157] Kenneth Katz, Oleg Shutov, Richard Lapoint, Michael Kimelman, J Rodney Brister, and Christopher O'Sullivan. The Sequence Read Archive: a decade more of explosive growth. *Nucleic Acids Research*, 2021.

[158] Max Doblas, Po Jui Shih, Oscar Lostes-Cazorla, Miquel Moreto, Christopher Batten, and Santiago Marco-Sola. Smx: Heterogeneous architecture for universal sequence alignment acceleration. In *MICRO*, 2025.

[159] Onur Mutlu and Can Firtina. Accelerating Genome Analysis via Algorithm-Architecture Co-Design. In *DAC*, 2023.

[160] Mohammed Alser, Joel Lindegger, Can Firtina, Nour Almadhoun, Haiyu Mao, Gagandeep Singh, Juan Gomez-Luna, and Onur Mutlu. From molecules to genomic variations: Accelerating genome analysis via intelligent algorithms and architectures. *Computational and Structural Biotechnology Journal*, 2022.

[161] Qian Lou, Sarath Chandra Janga, and Lei Jiang. Helix: Algorithm/architecture co-design for accelerating nanopore genome base-calling. In *PACT*, 2020.

[162] Qian Lou and Lei Jiang. Brawl: A spintronics-based portable basecalling-in-memory architecture for nanopore genome sequencing. *IEEE CAL*, 2018.

[163] Taha Shahroodi, Gagandeep Singh, Mahdi Zahedi, Haiyu Mao, Joel Lindegger, Can Firtina, Stephan Wong, Onur Mutlu, and Said Hamdioui. Swordfish: A Framework for Evaluating Deep Neural Network-based Basecalling using Computation-In-Memory with Non-Ideal Memristors. In *MICRO*, 2023.

[164] Ryan Marcus, Andreas Kipf, Alexander van Renen, Mihail Stoian, Sanchit Misra, Alfons Kemper, Thomas Neumann, and Tim Kraska. Benchmarking learned indexes. *Proceedings of the VLDB Endowment*, 2020.

[165] Arun Subramaniyan, Jack Wadden, Kush Goliya, Nathan Ozog, Xiao Wu, Satish Narayanasamy, David Blaauw, and Reetuparna Das. Accelerated seeding for genome sequence alignment with enumerated radix trees. In *ISCA*, 2021.

[166] Wenqin Huangfu, Shuangchen Li, Xing Hu, and Yuan Xie. RADAR: A 3D-ReRAM based DNA Alignment Accelerator Architecture. In *DAC*, 2018.

[167] S Karen Khatamifard, Zamshed Chowdhury, Nakul Pande, Meisam Razaviyayn, Chris H Kim, and Ulya R Karpuzcu. GeNVoM: Read Mapping Near Non-Volatile Memory. *IEEE/ACM TCBB*, 2021.

[168] Saransh Gupta, Mohsen Imani, Behnam Khaleghi, Venkatesh Kumar, and Tajana Rosing. RAPID: A ReRAM Processing In-memory Architecture for DNA Sequence Alignment. In *ISLPED*, 2019.

[169] Xue-Qi Li, Guang-Ming Tan, and Ning-Hui Sun. PIM-Align: A Processing-in-Memory Architecture for FM-Index Search Algorithm. *Journal of Computer Science and Technology*, 2021.

[170] Shaahin Angizi, Jiao Sun, Wei Zhang, and Deliang Fan. Aligns: A Processing-in-memory Accelerator for DNA Short Read Alignment Leveraging SOT-MRAM. In *DAC*, 2019.

[171] Farzaneh Zokaee, Hamid R Zarandi, and Lei Jiang. AligneR: A Process-in-Memory Architecture for Short Read Alignment in ReRAMs. *IEEE CAL*, 2018.

[172] Yatish Turakhia, Gill Bejerano, and William J Dally. Darwin: A Genomics Coprocessor Provides up to 15,000 x Acceleration on Long Read Assembly. In *ASPLOS*, 2018.

[173] Daichi Fujiki, Arun Subramaniyan, Tianjun Zhang, Yu Zeng, Reetuparna Das, David Blaauw, and Satish Narayanasamy. Genax: A Genome Sequencing Accelerator. In *ISCA*, 2018.

[174] Advait Madhavan, Timothy Sherwood, and Dmitri Strukov. Race Logic: A Hardware Acceleration for Dynamic Programming Algorithms. In *ISCA*, 2014.

[175] Haoyu Cheng, Yong Zhang, and Yun Xu. Bitmapper2: A GPU-accelerated All-mapper Based on The Sparse Q-gram Index. *IEEE/ACM TCBB*, 2018.

[176] Ernst Joachim Houtgast, Vlad-Mihai Sima, Koen Bertels, and Zaid Al-Ars. Hardware Acceleration of BWA-MEM Genomic Short Read Mapping for Longer Read Lengths. *Computational Biology and Chemistry*, 2018.

[177] Ernst Joachim Houtgast, VladMihai Sima, Koen Bertels, and Zaid AlArs. An Efficient GPU-accelerated Implementation of Genomic Short Read Mapping with BWA-MEM. *ACM SIGARCH CAN*, 2017.

[178] Alberto Zeni, Giulia Guidi, Marquita Ellis, Nan Ding, Marco D Santambrogio, Steven Hofmeyr, Aydın Buluç, Leonid Oliker, and Katherine Yelick. Logan: High-performance GPU-based X-drop Long-read Alignment. In *IPDPS*, 2020.

[179] Nauman Ahmed, Jonathan Lévy, Shanshan Ren, Hamid Mushtaq, Koen Bertels, and Zaid Al-Ars. GASAL2: A GPU Accelerated Sequence Alignment Library for High-Throughput NGS Data. *BMC Bioinformatics*, 2019.

[180] Takahiro Nishimura, Jacir L Bordim, Yasuaki Ito, and Koji Nakano. Accelerating the Smith-waterman Algorithm Using Bitwise Parallel Bulk Computation Technique on GPU. In *IPDPSW*, 2017.

[181] Edans Flavius de Oliveira Sandes, Guillermo Miranda, Xavier Martorell, Eduard Ayguade, George Teodoro, and Alba Cristina Magalhaes Melo. CUDAlign 4.0: Incremental Speculative Traceback for Exact Chromosome-wide Alignment in GPU Clusters. *IEEE TPDS*, 2016.

[182] Yongchao Liu and Bertil Schmidt. GSWABE: Faster GPU-accelerated Sequence Alignment with Optimal Alignment Retrieval for Short DNA Sequences. *Concurrency and Computation: Practice and Experience*, 2015.

[183] Yongchao Liu, Adrianto Wirawan, and Bertil Schmidt. CUDASW++ 3.0: Accelerating Smith-Waterman Protein Database Search by Coupling CPU and GPU SIMD Instructions. *BMC Bioinformatics*, 2013.

[184] Yongchao Liu, Douglas L Maskell, and Bertil Schmidt. CUDASW++: Optimizing Smith-Waterman Sequence Database Searches for CUDA-enabled Graphics Processing Units. *BMC Research Notes*, 2009.

[185] Yongchao Liu, Bertil Schmidt, and Douglas L Maskell. CUDASW++ 2.0: Enhanced Smith-Waterman Protein Database Search on CUDA-enabled GPUs Based on SIMT and Virtualized SIMD Abstractions. *BMC Research Notes*, 2010.

[186] Richard Wilton, Tamas Budavari, Ben Langmead, Sarah J. Wheelan, Steven L. Salzberg, and Alexander S. Szalay. Arioc: High-throughput Read Alignment with GPU-accelerated Exploration of The Seed-and-extend Search Space. *PeerJ*, 2015.

[187] Amit Goyal, Hyuk Jung Kwon, Kichan Lee, Reena Garg, Seon Young Yun, Yoon Hee Kim, Sunghoon Lee, and Min Seob Lee. Ultra-fast Next Generation Human Genome Sequencing Data Processing Using DRAGENTM Bio-IT Processor for Precision Medicine. *Open Journal of Genetics*, 2017.

[188] Yu-Ting Chen, Jason Cong, Zhenman Fang, Jie Lei, and Peng Wei. When Spark Meets FPGAs: A Case Study for Next-Generation DNA Sequencing Acceleration. In *USENIX HotCloud*, 2016.

[189] Peng Chen, Chao Wang, Xi Li, and Xuehai Zhou. Accelerating the Next Generation Long Read Mapping with the FPGA-based System. *IEEE/ACM TCBB*, 2014.

[190] Yen-Lung Chen, Bo-Yi Chang, Chia-Hsiang Yang, and Tzi-Dar Chiueh. A High-Throughput FPGA Accelerator for Short-Read Mapping of the Whole Human Genome. *IEEE TPDS*, 2021.

[191] Daichi Fujiki, Shunhao Wu, Nathan Ozog, Kush Goliya, David Blaauw, Satish Narayanasamy, and Reetuparna Das. SeedEx: A Genome Sequencing Accelerator for Optimal Alignments in Subminimal Space. In *MICRO*, 2020.

[192] Subho Sankar Banerjee, Mohamed El-Hadedy, Jong Bin Lim, Zbigniew T Kalbarczyk, Deming Chen, Steven S Lumetta, and Ravishankar K Iyer. ASAP: Accelerated Short-read Alignment on Programmable Hardware. *IEEE TC*, 2019.

[193] Xia Fei, Zou Dan, Lu Lina, Man Xin, and Zhang Chunlei. FPGASW: Accelerating Large-scale Smith–Waterman Sequence Alignment Application with Backtracking on FPGA Linear Systolic Array. *Interdisciplinary Sciences: Computational Life Sciences*, 2018.

[194] Hasitha Muthumala Waidyasooriya and Masanori Hariyama. Hardware-acceleration of Short-read Alignment Based on the Burrows-wheeler Transform. *IEEE TPDS*, 2015.

[195] Yu-Ting Chen, Jason Cong, Jie Lei, and Peng Wei. A Novel High-throughput Acceleration Engine for Read Alignment. In *FCCM*, 2015.

[196] Enzo Rucci, Carlos Garcia, Guillermo Botella, Armando De Giusti, Marcelo Naiouf, and Manuel Prieto-Matias. SWIFOLD: Smith-Waterman Implementation on FPGA with OpenCL for Long DNA Sequences. *BMC Systems Biology*, 2018.

[197] Abbas Haghi, Santiago Marco-Sola, Lluc Alvarez, Dionysios Diamantopoulos, Christoph Hagleitner, and Miquel Moreto. An FPGA Accelerator of the Wavefront Algorithm for Genomics Pairwise Alignment. In *FPL*, 2021.

[198] Luyi Li, Jun Lin, and Zhongfeng Wang. PipeBSW: A Two-Stage Pipeline Structure for Banded Smith-Waterman Algorithm on FPGA. In *ISVLSI*, 2021.

[199] Tae Jun Ham, David Bruns-Smith, Brendan Sweeney, Yejin Lee, Seong Hoon Seo, U Gyeong Song, Young H Oh, Krste Asanovic, Jae W Lee, and Lisa Wu Wills. Genesis: A Hardware Acceleration Framework for Genomic Data Analysis. In *ISCA*, 2020.

[200] Tae Jun Ham, Yejin Lee, Seong Hoon Seo, U Gyeong Song, Jae W Lee, David Bruns-Smith, Brendan Sweeney, Krste Asanovic, Young H Oh, and Lisa Wu Wills. Accelerating Genomic Data Analytics With Composable Hardware Acceleration Framework. *IEEE Micro*, 2021.

[201] Lisa Wu, David Bruns-Smith, Frank A. Nothaft, Qijing Huang, Sagar Karandikar, Johnny Le, Andrew Lin, Howard Mao, Brendan Sweeney, Krste Asanović, David A. Patterson, and Anthony D. Joseph. FPGA Accelerated Indel Realignment in the Cloud. In *HPCA*, 2019.

[202] Damla Senol Cali, Gurpreet S. Kalsi, Zülal Bingöl, Can Firtina, Lavanya Subramanian, Jeremie S. Kim, Rachata Ausavarungnirun, Mohammed Alser, Juan Gomez-Luna, Amirali Boroumand, Anant Norion, Allison Scibisz, Sreenivas Subramoneyon, Can Alkan, Saugata Ghose, and Onur Mutlu. GenASM: A High-Performance, Low-Power Approximate String Matching Acceleration Framework for Genome Sequence Analysis. In *MICRO*, 2020.

[203] Fan Zhang, Shaahin Angizi, Jiao Sun, Wei Zhang, and Deliang Fan. Aligner-D: Leveraging In-DRAM Computing to Accelerate DNA Short Read Alignment. *IEEE JETCAS*, 2023.

[204] Melina Soysal, Konstantina Koliogeorgi, Can Firtina, Nika Mansouri Ghiasi, Rakesh Nadig, Haiyu Mao, Geraldo Francisco de Oliveira Junior, Yu Liang, Klea Zambaku, Mohammad Sadrosadati, and Onur Mutlu. MARS: Processing-In-Memory Acceleration of Raw Signal Genome Analysis Inside the Storage Subsystem. In *ICS*, 2025.

[205] Jeremie S. Kim, Damla Senol Cali, Hongyi Xin, Donghyuk Lee, Saugata Ghose, Mohammed Alser, Hasan Hassan, Oguz Ergin, Can Alkan, and Onur Mutlu. GRIM-Filter: Fast Seed Location Filtering in DNA Read Mapping Using Processing-in-memory Technologies. *BMC Genomics*, 2018.

[206] Roman Kaplan, Leonid Yavits, and Ran Ginosasr. BioSEAL: In-memory biological sequence alignment accelerator for large-scale genomic data. In *SYSTOR*, 2020.

[207] Haiyu Mao, Mohammed Alser, Mohammad Sadrosadati, Can Firtina, Akanksha Baranwal, Damla Senol Cali, Aditya Manglik, Nour Almadhoun Alserr, and Onur Mutlu. GenPIP: In-Memory Acceleration of Genome Analysis via Tight Integration of Basecalling and Read Mapping. In *MICRO*, 2022.

[208] Yingqi Cao, Anshu Guta, Jason Liang, and Yatish Turakhia. DP-HLS: A High-Level Synthesis Framework for Accelerating Dynamic Programming Algorithms in Bioinformatics. In *HPCA*, 2026.

[209] Zhehong Wang, Tianjun Zhang, Daichi Fujiki, Arun Subramaniyan, Xiao Wu, Makoto Yasuda, Satoru Miyoshi, Masaru Kawaminami, Reetuparna Das, Satish Narayanasamy, and David Blaauw. A 2.46M Reads/s Seed-Extension Accelerator for Next-Generation Sequencing Using a String-Independent PE Array. *IEEE JSSC*, 2020.

[210] Sumit Walia, Cheng Ye, Arkid Bera, Dhruvi Lodhavia, and Yatish Turakhia. TALCO: Tiling Genome Sequence Alignment Using Convergence of Traceback Pointers. In *HPCA*, 2024.

[211] Harisankar Sadasivan, Artur Klauser, Juergen Hench, Yatish Turakhia, Gagandeep Singh, Alberto Zeni, Sarah Beecroft, Satish Narayanasamy, Jeff Nivala, Bob Robey, Onur Mutlu, Kristof Denolf, and Sriranjani Sitaraman. The Genomic Computing Revolution: Defining the Next Decades of Accelerating Genomics. In *HPEC*, 2024.

[212] Yatish Turakhia. Toward a Generalized Accelerator for Genome Sequence Analysis. *Communications of the ACM*, 2025.

[213] Yatish Turakhia, Sneha D. Goenka, Gill Bejerano, and WIlliam J. Dally. Darwin-WGA: A Co-processor Provides Increased Sensitivity in Whole Genome Alignments with High Speedup. In *HPCA*, 2019.

[214] William Andrew Simon, Leonid Yavits, Konstantina Koliogeorgi, Yann Falevoz, Yoshihiro Shibuya, Dominique Lavenier, Irem Boybat, Klea Zambaku, Berkan Şahin, Mohammad Sadrosadati, Onur Mutlu, Abu Sebastian, Rayan Chikhi, The BioPIM Consortium, and Can Alkan. Processing-in-Memory for Genomics Workloads. *IEEE Micro*, 2026.

[215] Peng Jia, Liming Xuan, Lei Liu, and Chaochun Wei. MetaBinG: Using GPUs to accelerate metagenomic sequence classification. *PLOS One*, 2011.

[216] Robin Kobus, André Müller, Daniel Jünger, Christian Hundt, and Bertil Schmidt. MetaCache-GPU: ultra-fast metagenomic classification. In *ICPP*, 2021.

[217] Xuebin Wang, Taifu Wang, Zhihao Xie, Youjin Zhang, Shiqiang Xia, Ruixue Sun, Xinqiu He, Ruizhi Xiang, Qiwen Zheng, Zhencheng Liu, Jin'An Wang, Honglong Wu, Xiangqian Jin, Weijun Chen, Dongfang Li, and Zengquan He. GPMeta: a GPU-accelerated method for ultrarapid pathogen identification from metagenomic sequences. *Briefings in Bioinformatics*, 2023.

[218] Robin Kobus, Christian Hundt, André Müller, and Bertil Schmidt. Accelerating Metagenomic Read Classification on CUDA-enabled GPUs. *BMC Bioinformatics*, 2017.

[219] Xiaoquan Su, Jian Xu, and Kang Ning. Parallel-META: efficient metagenomic data analysis based on high-performance computation. *BMC Systems Biology*, 2012.

[220] Xiaoquan Su, Xuetao Wang, Gongchao Jing, and Kang Ning. GPU-Meta-Storms: computing the structure similarities among massive amount of microbial community samples using GPU. *Bioinformatics*, 2014.

[221] Masahiro Yano, Hiroshi Mori, Yutaka Akiyama, Takuji Yamada, and Ken Kurokawa. CLAST: CUDA implemented large-scale alignment search tool. *BMC Bioinformatics*, 2014.

[222] Antonio Saavedra, Hans Lehnert, Cecilia Hernández, Gonzalo Carvajal, and Miguel Figueroa. Mining discriminative k-mers in DNA sequences using sketches and hardware acceleration. *IEEE Access*, 2020.

[223] Tianqi Zhang, Antonio González, Niema Moshiri, Rob Knight, and Tajana Rosing. GenoMiX: Accelerated Simultaneous Analysis of Human Genomics, Microbiome Metagenomics, and Viral Sequences. In *BioCAS*, 2023.

[224] Gustavo Henrique Cervi, Cecília Dias Flores, and Claudia Elizabeth Thompson. Metagenomic Analysis: A Pathway Toward Efficiency Using High-Performance Computing. In *ICICT*, 2022.

[225] Lingxi Wu, Rasool Sharifi, Marzieh Lenjani, Kevin Skadron, and Ashish Venkat. Sieve: Scalable in-situ dram-based accelerator designs for massively parallel k-mer matching. In *ISCA*, 2021.

[226] Taha Shahroodi, Mahdi Zahedi, Abhairaj Singh, Stephan Wong, and Said Hamdioui. KrakenOnMem: a memristor-augmented HW/SW framework for taxonomic profiling. In *ICS*, 2022.

[227] Zuher Jahshan, Itay Merlin, Esteban Garzón, and Leonid Yavits. DASH-CAM: Dynamic Approximate SearcH Content Addressable Memory for genome classification. In *MICRO*, 2023.

[228] Robert Hanhan, Esteban Garzón, Zuher Jahshan, Adam Teman, Marco Lanuzza, and Leonid Yavits. Edam: edit distance tolerant approximate matching content addressable memory. In *ISCA*, 2022.

[229] Zhuowen Zou, Hanning Chen, Prathyush Poduval, Yeseong Kim, Mahdi Imani, Elaheh Sadredini, Rosario Cammarota, and Mohsen Imani. BioHD: an efficient genome sequence search platform using HyperDimensional memorization. In *ISCA*, 2022.

[230] Po Jui Shih, Hassaan Saadat, Sri Parameswaran, and Hasindu Gamaarachchi. Efficient real-time selective genome sequencing on resource-constrained devices. *GigaScience*, 2023.

[231] Damla Senol Cali, Konstantinos Kanellopoulos, Joël Lindegger, Zülal Bingöl, Gurpreet S. Kalsi, Ziyi Zuo, Can Firtina, Meryem Banu Cavlak, Jeremie Kim, Nika Mansouri Ghiasi, Gagandeep Singh, Juan Gómez-Luna, Nour Almadhoun Alserr, Mohammed Alser, Sreenivas Subramoney, Can Alkan, Saugata Ghose, and Onur Mutlu. SeGraM: A universal hardware accelerator for genomic sequence-to-graph and sequence-to-sequence mapping. In *ISCA*, 2022.

[232] Yichi Zhang, Dibei Chen, Gang Zeng, Jianfeng Zhu, Zhaoshi Li, Longlong Chen, Shaojun Wei, and Leibo Liu. Harp: Leveraging Quasi-Sequential Characteristics to Accelerate Sequence-to-Graph Mapping of Long Reads. In *ASPLOS*, 2024.

[233] Gang Zeng, Jianfeng Zhu, Yichi Zhang, Ganhui Chen, Zhenhai Yuan, Shaojun Wei, and Leibo Liu. A High-Performance Genomic Accelerator for Accurate Sequence-to-Graph Alignment Using Dynamic Programming Algorithm. *IEEE TPDS*, 2024.

[234] Zhe-Wei Shen, Jheng-Syun Huang, and Yi-Chang Lu. A Memory-Efficient Accelerator for 128-Parallel Sequence-to-Graph Alignment in Variant-Enriched Regions. In *BioCAS*, 2024.

[235] Wen-Jun Li, Bhagwan Narayan Rekadwad, Jian-Yu Jiao, and Nimaichand Salam. Exploring Microbial Dark Matter and the Status of Bacterial and Archaeal Taxonomy: Challenges and Opportunities in the Future. In *Modern Taxonomy of Bacteria and Archaea: New Methods, Technology and Advances*, 2024.

[236] Kalikinkar Mandal, Bo Yang, Guang Gong, and Mark Aagaard. Analysis and Efficient Implementations of a Class of Composited de Bruijn Sequences. *IEEE TC*, 2020.

[237] B. Sharat Chandra Varma, Kolin Paul, M. Balakrishnan, and Dominique Lavenier. FAssem: FPGA Based Acceleration of De Novo Genome Assembly. In *FCCM*, 2013.

[238] Muaaz Gul Awan, Steven Hofmeyr, Rob Egan, Nan Ding, Aydin Buluc, Jack Deslippe, Leonid Oliker, and Katherine Yelick. Accelerating large scale de novo metagenome assembly using GPUs. In *SC*, 2021.

[239] Zonghao Feng and Qiong Luo. Accelerating Sequence-to-Graph Alignment on Heterogeneous Processors. In *ICPP*, 2021.

[240] Yichi Zhang, Jianfeng Zhu, Liangwei Li, Gang Zeng, Dibei Chen, Tairan Zhang, Yeyang Deng, Zhicheng Gong, Aoyang Zhang, Yang Liu, Shaojun Wei, and Leibo Liu. A 28-nm 239-bp/μJ Agile Pangenome Analysis Accelerator for Multi-Scheme Read Mapping. *IEEE JSSC*, 2025.

[241] Heewoo Kim, Sanjay Sri Vallabh Singapuram, Haojie Ye, Joseph Izraelevitz, Trevor Mudge, Ronald Dreslinski, and Nishil Talati. NMP-PaK: Near-Memory Processing Acceleration of Scalable De Novo Genome Assembly. In *ISCA*, 2025.

[242] Yu Huang, Long Zheng, Haifeng Liu, Zhuoran Zhou, Dan Chen, Pengcheng Yao,

Qinggang Wang, Xiaofei Liao, and Hai Jin. MeG2: In-Memory Acceleration for Genome Graphs Analysis. In *DAC*, 2023.
[243] Shaahin Angizi, Naima Ahmed Fahmi, Deniz Najafi, Wei Zhang, and Deliang Fan. PANDA: Processing in Magnetic Random-Access Memory-Accelerated de Bruijn Graph-Based DNA Assembly. *Journal of Low Power Electronics and Applications*, 2024.
[244] Shuang Qiu and Qiong Luo. Parallelizing Big De Bruijn Graph Construction on Heterogeneous Processors. In *ICDCS*, 2017.
[245] Minxuan Zhou, Lingxi Wu, Muzhou Li, Niema Moshiri, Kevin Skadron, and Tajana Rosing. Ultra Efficient Acceleration for De Novo Genome Assembly via Near-Memory Computing. In *PACT*, 2021.
[246] Aritra Sarkar, Zaid Al-Ars, and Koen Bertels. QuASeR: Quantum Accelerated de novo DNA sequence reconstruction. *PLOS ONE*, 2021.
[247] B. Sharat Chandra Varma, Kolin Paul, M. Balakrishnan, and Dominique Lavenier. Hardware acceleration of de novo genome assembly. *International Journal of Embedded Systems*, 2017.
[248] B. Sharat Chandra Varma, Kolin Paul, and M. Balakrishnan. FPGA-Based Acceleration of De Novo Genome Assembly. In *Architecture Exploration of FPGA Based Accelerators for BioInformatics Applications*, 2016.
[249] Sayan Goswami, Kisung Lee, Shayan Shams, and Seung-Jong Park. GPU-Accelerated Large-Scale Genome Assembly. In *IPDPS*, 2018.
[250] Georgios Galanos, Pavlos Malakonakis, and Apostolos Dollas. An FPGA-Based Data Pre-Processing Architecture to Accelerate De-Novo Genome Assembly. In *BIBE*, 2021.
[251] Shaahin Angizi, Naima Ahmed Fahmi, Wei Zhang, and Deliang Fan. PIM-Assembler: A Processing-in-Memory Platform for Genome Assembly. In *DAC*, 2020.
[252] Aman Sinha, Huei-Chun Yang, Pei-Yi Liu, Yen-Shi Kuo, Yuhao Fang, Tien-Shuo Chang, Ke-Han Li, and Bo-Cheng Lai. DSIM: Distributed Sequence Matching on Near-DRAM Accelerator for Genome Assembly. *IEEE JETCAS*, 2022.
[253] Pingfan Meng, Matthew Jacobsen, Motoki Kimura, Vladimir Dergachev, Thomas Anantharaman, Michael Requa, and Ryan Kastner. Hardware accelerated novel optical de novo assembly for large-scale genomes. In *FPL*, 2014.
[254] Yuanqi Hu and Pantelis Georgiou. A Real-Time de novo DNA Sequencing Assembly Platform Based on an FPGA Implementation. *IEEE/ACM TCBB*, 2016.
[255] Yibo Chen, Jun-Han Huang, Yuhui Sun, Yong Zhang, Yuxiang Li, and Xun Xu. VRP Assembler: haplotype-resolvedde novoassembly of diploid and polyploid genomes using quantum computing. *Cell Reports Methods*, 2023.
[256] Santhi Natarajan, N. KrishnaKumar, H. V. Anuchan, Debnath Pal, and S. K. Nandy. ReneGENE-Novo: Co-designed Algorithm-Architecture for Accelerated Preprocessing and Assembly of Genomic Short Reads. In *ARC*, 2018.
[257] Shanshan Ren, Nauman Ahmed, Koen Bertels, and Zaid Al-Ars. An Efficient GPU-Based de Bruijn Graph Construction Algorithm for Micro-Assembly. In *BIBE*, 2018.
[258] Zamin Iqbal, Mario Caccamo, Isaac Turner, Paul Flicek, and Gil McVean. De novo assembly and genotyping of variants using colored de Bruijn graphs. *Nature Genetics*, 2012.
[259] Camille Marchet, Christina Boucher, Simon J. Puglisi, Paul Medvedev, Mikaël Salson, and Rayan Chikhi. Data structures based on k-mers for querying large collections of sequencing data sets. *Genome Research*, 2021.
[260] Mikhail Karasikov, Harun Mustafa, Gunnar Rätsch, and André Kahles. Lossless indexing with counting de Bruijn graphs. *Genome Research*, 2022.
[261] Mikhail Karasikov, Harun Mustafa, Amir Joudaki, Sara Javadzadeh-no, Gunnar Rätsch, and André Kahles. Sparse Binary Relation Representations for Genome Graph Annotation. *Journal of Computational Biology*, 2020.
[262] Daniel Danciu, Mikhail Karasikov, Harun Mustafa, André Kahles, and Gunnar Rätsch. Topology-based sparsification of graph annotations. *Bioinformatics*, 2021.
[263] Phelim Bradley, Henk C Den Bakker, Eduardo PC Rocha, Gil McVean, and Zamin Iqbal. Ultrafast Search of All Deposited Bacterial and Viral Genomic Data. *Nature Biotechnology*, 2019.
[264] Martin Hunt, Leandro Lima, Daniel Anderson, George Bouras, Michael B. Hall, Jane Hawkey, Oliver Schwengers, Wei Shen, John Lees, and Zamin Iqbal. AllTheBacteria - all bacterial genomes assembled, available and searchable. *bioRxiv*, 2025.
[265] Jouni Sirén, Jean Monlong, Xian Chang, Adam M. Novak, Jordan M. Eizenga, Charles Markello, Jonas A. Sibbesen, Glenn Hickey, Pi-Chuan Chang, Andrew Carroll, Namrata Gupta, Stacey Gabriel, Thomas W. Blackwell, Aakrosh Ratan, Kent D. Taylor, Stephen S. Rich, Jerome I. Rotter, David Haussler, Erik Garrison, and Benedict Paten. Pangenomics Enables Genotyping of Known Structural Variants in 5202 Diverse Genomes. *Science*, 2021.
[266] Rachel M. Sherman and Steven L. Salzberg. Pan-genomics in the human genome era. *Nature Reviews Genetics*, 2020.
[267] Dylan J. Taylor, Jordan M. Eizenga, Qiuhui Li, Arun Das, Katharine M. Jenike, Eimear E. Kenny, Karen H. Miga, Jean Monlong, Rajiv C. McCoy, Benedict Paten, and Michael C. Schatz. Beyond the human genome project: The age of complete human genome sequences and pangenome references. *Annual Review of Genomics and Human Genetics*, 2024.
[268] Jordan M. Eizenga, Adam M. Novak, Jonas A. Sibbesen, Simon Heumos, Ali Ghaffaari, Glenn Hickey, Xian Chang, Josiah D. Seaman, Robin Rounthwaite, Jana Ebler, Mikko Rautiainen, Shilpa Garg, Benedict Paten, Tobias Marschall, Jouni Sirén, and Erik Garrison. Pangenome graphs. *Annual Review of Genomics and Human Genetics*, 2020.
[269] Wen-Wei Liao, Mobin Asri, Jana Ebler, Daniel Doerr, Marina Haukness, Glenn Hickey, Shuangjia Lu, Julian K. Lucas, Jean Monlong, Haley J. Abel, Silvia Buonaiuto, Xian H. Chang, Haoyu Cheng, Justin Chu, Vincenza Colonna, Jordan M. Eizenga, Xiaowen Feng, Christian Fischer, Robert S. Fulton, Shilpa Garg, Cristian Groza, Andrea Guarracino, William T. Harvey, Simon Heumos, Kerstin Howe, Miten Jain, Tsung-Yu Lu, Charles Markello, Fergal J. Martin, Matthew W. Mitchell, Katherine M. Munson, Moses Njagi Mwaniki, Adam M. Novak, Hugh E. Olsen, Trevor Pesout, David Porubsky, Pjotr Prins, Jonas A. Sibbesen, Jouni Sirén, Chad Tomlinson, Flavia Villani, Mitchell R. Vollger, Lucinda L. Antonacci-Fulton, Gunjan Baid, Carl A. Baker, Anastasiya Belyaeva, Konstantinos Billis, Andrew Carroll, Pi-Chuan Chang, Sarah Cody, Daniel E. Cook, Robert M. Cook-Deegan, Omar E. Cornejo, Mark Diekhans, Peter Ebert, Susan Fairley, Olivier Fedrigo, Adam L. Felsenfeld, Giulio Formenti, Adam Frankish, Yan Gao, Nanibaa' A. Garrison, Carlos Garcia Giron, Richard E. Green, Leanne Haggerty, Kendra Hoekzema, Thibaut Hourlier, Hanlee P. Ji, Eimear E. Kenny, Barbara A. Koenig, Alexey Kolesnikov, Jan O. Korbel, Jennifer Kordosky, Sergey Koren, HoJoon Lee, Alexandra P. Lewis, Hugo Magalhães, Santiago Marco-Sola, Pierre Marijon, Ann McCartney, Jennifer McDaniel, Jacquelyn Mountcastle, Maria Nattestad, Sergey Nurk, Nathan D. Olson, Alice B. Popejoy, Daniela Puiu, Mikko Rautiainen, Allison A. Regier, Arang Rhie, Samuel Sacco, Ashley D. Sanders, Valerie A. Schneider, Baergen I. Schultz, Kishwar Shafin, Michael W. Smith, Heidi J. Sofia, Ahmad N. Abou Tayoun, Françoise Thibaud-Nissen, Francesca Floriana Tricomi, Justin Wagner, Brian Walenz, Jonathan M. D. Wood, Aleksey V. Zimin, Guillaume Bourque, Mark J. P. Chaisson, Paul Flicek, Adam M. Phillippy, Justin M. Zook, Evan E. Eichler, David Haussler, Ting Wang, Erich D. Jarvis, Karen H. Miga, Erik Garrison, Tobias Marschall, Ira M. Hall, Heng Li, and Benedict Paten. A draft human pangenome reference. *Nature*, 2023.
[270] Joel Armstrong, Glenn Hickey, Mark Diekhans, Ian T. Fiddes, Adam M. Novak, Alden Deran, Qi Fang, Duo Xie, Shaohong Feng, Josefin Stiller, Diane Genereux, Jeremy Johnson, Voichita Dana Marinescu, Jessica Alföldi, Robert S. Harris, Kerstin Lindblad-Toh, David Haussler, Elinor Karlsson, Erich D. Jarvis, Guojie Zhang, and Benedict Paten. Progressive Cactus is a multiple-genome aligner for the thousand-genome era. *Nature*, 2020.
[271] Bahar Alipanahi, Martin D Muggli, Musa Jundi, Noelle R Noyes, and Christina Boucher. Metagenome SNP calling via read-colored de Bruijn graphs. *Bioinformatics*, 2020.
[272] Ana Vieira, Yu Wan, Yan Ryan, Ho Kwong Li, Rebecca L. Guy, Maria Papangeli, Kristin K. Huse, Lucy C. Reeves, Valerie W. C. Soo, Roger Daniel, Alessandra Harley, Karen Broughton, Chenchal Dhami, Mark Ganner, Marjorie A. Ganner, Zaynab Mumin, Maryam Razaei, Emma Rundberg, Rufat Mammadov, Ewurabena A. Mills, Vincenzo Sgro, Kai Yi Mok, Xavier Didelot, Nicholas J. Croucher, Elita Jauneikaite, Theresa Lamagni, Colin S. Brown, Juliana Coelho, and Shiranee Sriskandan. Rapid expansion and international spread of M1UK in the post-pandemic UK upsurge of Streptococcus pyogenes. *Nature Communications*, 2024.
[273] Pranav Gangwar, Pratik Katte, Manu Bhat, and Yatish Turakhia. WEPP: Phylogenetic placement achieves near-haplotype resolution in wastewater-based epidemiology. *PLOS Computational Biology*, 2026.
[274] Daehwan Kim, Li Song, Florian P Breitwieser, and Steven L Salzberg. Centrifuge: Rapid and Sensitive Classification of Metagenomic Sequences. *Genome Research*, 2016.
[275] Derrick E Wood, Jennifer Lu, and Ben Langmead. Improved Metagenomic Analysis with Kraken 2. *Genome Biology*, 2019.
[276] André Müller, Christian Hundt, Andreas Hildebrandt, Thomas Hankeln, and Bertil Schmidt. MetaCache: context-aware classification of metagenomic reads using minhashing. *Bioinformatics*, 2017.
[277] Li Song and Ben Langmead. Centrifuger: lossless compression of microbial genomes for efficient and accurate metagenomic sequence classification. *Genome Biology*, 2024.
[278] Alexander T Dilthey, Chirag Jain, Sergey Koren, and Adam M Phillippy. Strain-level metagenomic assignment and compositional estimation for long reads with MetaMaps. *Nature Communications*, 2019.
[279] Jeremy Fan, Steven Huang, and Samuel D Chorlton. BugSeq: a highly accurate cloud platform for long-read metagenomic analyses. *BMC Bioinformatics*, 2021.
[280] Alessio Milanese, Daniel R. Mende, Lucas Paoli, Guillem Salazar, Hans-Joachim Ruscheweyh, Miguelangel Cuenca, Pascal Hingamp, Renato Alves, Paul I. Costea, Luis Pedro Coelho, Thomas S. B. Schmidt, Alexandre Almeida, Alex L. Mitchell, Robert D. Finn, Jaime Huerta-Cepas, Peer Bork, Georg Zeller, and Shinichi Sunagawa. Microbial abundance, activity and population genomic profiling with mOTUs2. *Nature Communications*, 2019.
[281] Steven L. Salzberg, Florian P. Breitwieser, Anupama Kumar, Haiping Hao, Peter Burger, Fausto J. Rodriguez, Michael Lim, Alfredo Quiñones-Hinojosa, Gary L. Gallia, Jeffrey A. Tornheim, Michael T. Melia, Cynthia L. Sears, and Carlos A. Pardo. Next-generation sequencing in neuropathologic diagnosis of infections of the nervous system. *Neurology - Neuroimmunology Neuroinflammation*, 2016.
[282] Abraham Gihawi, Yuchen Ge, Jennifer Lu, Daniela Puiu, Amanda Xu, Colin S. Cooper, Daniel S. Brewer, Mihaela Pertea, and Steven L. Salzberg. Major data analysis errors invalidate cancer microbiome findings. *mBio*, 2023.
[283] Christopher Pockrandt, Aleksey V. Zimin, and Steven L. Salzberg. Metagenomic classification with KrakenUniq on low-memory computers. *Journal of Open Source Software*, 2022.
[284] Joel Ackelsberg, Jennifer Rakeman, Scott Hughes, Jeannine Petersen, Paul Mead, Martin Schriefer, Luke Kingry, Alex Hoffmaster, and Jay E. Gee. Lack of Evidence for Plague or Anthrax on the New York City Subway. *Cell Systems*, 2015.
[285] Daniel J. Nasko, Sergey Koren, Adam M. Phillippy, and Todd J. Treangen. RefSeq database growth influences the accuracy of k-mer-based lowest common ancestor species identification. *Genome Biology*, 2018.
[286] Fernando Meyer, Adrian Fritz, Zhi-Luo Deng, David Koslicki, Till Robin Lesker, Alexey Gurevich, Gary Robertson, Mohammed Alser, Dmitry Antipov, Francesco Beghini, Denis Bertrand, Jaqueline J. Brito, C. Titus Brown, Jan Buchmann, Aydin Buluç, Bo Chen, Rayan Chikhi, Philip T. L. C. Clausen, Alexandru Cristian, Piotr Wojciech Dabrowski, Aaron E. Darling, Rob Egan, Eleazar Eskin, Evangelos Georganas, Eugene Goltsman, Melissa A. Gray, Lars Hestbjerg Hansen, Steven Hofmeyr, Pingqin Huang, Luiz Irber, Huijue Jia, Tue Sparholt Jørgensen, Silas D.

Kieser, Terje Klemetsen, Axel Kola, Mikhail Kolmogorov, Anton Korobeynikov, Jason Kwan, Nathan LaPierre, Claire Lemaitre, Chenhao Li, Antoine Limasset, Fabio Malcher-Miranda, Serghei Mangul, Vanessa R. Marcelino, Camille Marchet, Pierre Marijon, Dmitry Meleshko, Daniel R. Mende, Alessio Milanese, Niranjan Nagarajan, Jakob Nissen, Sergey Nurk, Leonid Oliker, Lucas Paoli, Pierre Peterlongo, Vitor C. Piro, Jacob S. Porter, Simon Rasmussen, Evan R. Rees, Knut Reinert, Bernhard Renard, Espen Mikal Robertsen, Gail L. Rosen, Hans-Joachim Ruscheweyh, Varuni Sarwal, Nicola Segata, Enrico Seiler, Lizhen Shi, Fengzhu Sun, Shinichi Sunagawa, Søren Johannes Sørensen, Ashleigh Thomas, Chengxuan Tong, Mirko Trajkovski, Julien Tremblay, Gherman Uritskiy, Riccardo Vicedomini, Zhengyang Wang, Ziye Wang, Zhong Wang, Andrew Warren, Nils Peder Willassen, Katherine Yelick, Ronghui You, Georg Zeller, Zhengqiao Zhao, Shanfeng Zhu, Jie Zhu, Ruben Garrido-Oter, Petra Gastmeier, Stephane Hacquard, Susanne Häußler, Ariane Khaledi, Friederike Maechler, Fantin Mesny, Simona Radutoiu, Paul Schulze-Lefert, Nathiana Smit, Till Strowig, Andreas Bremges, Alexander Sczyrba, and Alice Carolyn McHardy. Critical Assessment of Metagenome Interpretation: the second round of challenges. *Nature Methods*, 2022.

[287] Mohammed Alser, Jeremy Rotman, Dhrithi Deshpande, Kodi Taraszka, Huwenbo Shi, Pelin Icer Baykal, Harry Taegyun Yang, Victor Xue, Sergey Knyazev, Benjamin D. Singer, Brunilda Balliu, David Koslicki, Pavel Skums, Alex Zelikovsky, Can Alkan, Onur Mutlu, and Serghei Mangul. Technology Dictates Algorithms: Recent Developments in Read Alignment. *Genome Biology*, 2021.

[288] Karasikov, Mikhail and Mustafa, Harun and Danciu, Daniel and Zimmermann, Marc and Barber, Christopher and Rätsch, Gunnar and Kahles, André. MetaGraph Amazon AWS S3 Bucket. `https://metagraph.s3.amazonaws.com/index.html`, 2025.

[289] Kris A. Wetterstrand. DNA Sequencing Costs: Data from the NHGRI Genome Sequencing Program (GSP). `https://www.genome.gov/sequencingcostsdata`, 2024.

[290] Yu Cai, Saugata Ghose, Erich F. Haratsch, Yixin Luo, and Onur Mutlu. Error Characterization, Mitigation, and Recovery in Flash-Memory-Based Solid-state Drives. *Proceedings of the IEEE*, 2017.

[291] Yu Cai, Saugata Ghose, Erich F. Haratsch, Yixin Luo, and Onur Mutlu. Reliability Issues in Flash-Memory-Based Solid-State Drives: Experimental Analysis, Mitigation, Recovery, 2018. In *Inside Solid State Drives (SSDs)*.

[292] Jagan Singh Meena, Simon Min Sze, Umesh Chand, and Tseung-Yuen Tseng. Overview of emerging nonvolatile memory technologies. *Nanoscale Research Letters*, 2014.

[293] Benjamin C Lee, Engin Ipek, Onur Mutlu, and Doug Burger. Architecting Phase Change Memory as a Scalable DRAM Alternative. In *ISCA*, 2009.

[294] Hiroyuki Akinaga and Hisashi Shima. Resistive Random Access Memory (ReRAM) Based on Metal Oxides. *Proceedings of the IEEE*, 2010.

[295] Said Tehrani, JM Slaughter, E Chen, M Durlam, J Shi, and M DeHerren. Progress and Outlook for MRAM Technology. *IEEE MAG*, 1999.

[296] Sang-Woo Jun, Andy Wright, Sizhuo Zhang, Shuotao Xu, and Arvind. GraFBoost: Using Accelerated Flash Storage for External Graph Analytics. In *ISCA*, 2018.

[297] Kiran Kumar Matam, Gunjae Koo, Haipeng Zha, Hung-Wei Tseng, and Murali Annavaram. GraphSSD: graph semantics aware SSD. In *ISCA*, 2019.

[298] Yitu Wang, Shiyu Li, Qilin Zheng, Linghao Song, Zongwang Li, Andrew Chang, Hai "Helen" Li, and Yiran Chen. NDSEARCH: Accelerating Graph-Traversal-Based Approximate Nearest Neighbor Search through Near Data Processing. In *ISCA*, 2024.

[299] Yunjae Lee, Jinha Chung, and Minsoo Rhu. SmartSAGE: training large-scale graph neural networks using in-storage processing architectures. In *ISCA*, 2022.

[300] Fuping Niu, Jianhui Yue, Jiangqiu Shen, Xiaofei Liao, and Hai Jin. FlashGNN: An In-SSD Accelerator for GNN Training. In *HPCA*, 2024.

[301] Yunjae Lee, Hyeseong Kim, and Minsoo Rhu. PreSto: An In-Storage Data Preprocessing System for Training Recommendation Models. In *ISCA*, 2025.

[302] Soheil Khadirsharbiyani, Nima Elyasi, Armin Haj Aboutalebi, Chun-Yi Liu, Changho Choi, and Mahmut Taylan Kandemir. SmartGraph: A Framework for Graph Processing in Computational Storage. In *SoCC*, 2024.

[303] Xinmiao Zhang, Cheng Liu, Shengwen Liang, Hayden Kwok-Hay So, Ying Wang, Lei Zhang, Huawei Li, and Xiaowei Li. Taijigraph: an Out-Of-Core Graph Processing System Enhanced with Computational Storage. In *IPDPS*, 2025.

[304] Jiyoung An, Esmerald Aliaj, and Sang-Woo Jun. Barad-dur: Near-Storage Accelerator for Training Large Graph Neural Networks. In *PACT*, 2023.

[305] Seongyoung Kang and Sang-Woo Jun. Sting: Near-storage accelerator framework for scalable triangle counting and beyond. In *DAC*, 2024.

[306] Tiantu Xu, Luis Materon Botelho, and Felix Xiaozhu Lin. Vstore: A data store for analytics on large videos. In *EuroSys*, 2019.

[307] Cangyuan Li, Ying Wang, Cheng Liu, Shengwen Liang, Huawei Li, and Xiaowei Li. GLIST: Towards In-Storage Graph Learning. In *USENIX ATC*, 2021.

[308] Seongyoung Kang and Sang-Woo Jun. Near-Storage Accelerator for Bulk Graph Ingestion. In *IPDPSW*, 2023.

[309] Yuyue Wang, Xiurui Pan, Yuda An, Jie Zhang, and Glenn Reinman. BeaconGNN: Large-Scale GNN Acceleration with Out-of-Order Streaming In-Storage Computing. In *HPCA*, 2024.

[310] Yoonyoung Kwon, Yunjong Boo, and Hyungmin Cho. GraphAccel: An In-Storage Accelerator for Efficient Graph-Based Vector Similarity Search Using Page Packing and Speculative Search Optimization. In *DAC*, 2025.

[311] Fuping Niu, Jianhui Yue, Jiangqiu Shen, Xiaofei Liao, Haikun Liu, and Hai Jin. Flashwalker: An in-storage accelerator for graph random walks. In *IPDPS*, 2022.

[312] Weihong Xu, Junwei Chen, Po-Kai Hsu, Jaeyoung Kang, Minxuan Zhou, Sumukh Pinge, Shimeng Yu, and Tajana Rosing. Proxima: Near-storage Acceleration for Graph-based Approximate Nearest Neighbor Search in 3D NAND. *IEEE TC*, 2026.

[313] Nika Mansouri Ghiasi, Jisung Park, Harun Mustafa, Jeremie Kim, Ataberk Olgun, Arvid Gollwitzer, Damla Senol Cali, Can Firtina, Haiyu Mao, Nour Almadhoun Alserr, Rachata Ausavarungnirun, Nandita Vijaykumar, Mohammed Alser, and Onur Mutlu. GenStore: A High-Performance in-Storage Processing System for Genome Sequence Analysis. In *ASPLOS*, 2022.

[314] Lingxi Wu, Minxuan Zhou, Weihong Xu, Ashish Venkat, Tajana Rosing, and Kevin Skadron. Abakus: Accelerating k-mer Counting With Storage Technology. *TACO*, 2023.

[315] Nika Mansouri Ghiasi, Mohammad Sadrosadati, Harun Mustafa, Arvid Gollwitzer, Can Firtina, Julien Eudine, Haiyu Ma, Joël Lindegger, Meryem Banu Cavlak, Mohammed Alser, Jisung Park, and Onur Mutlu. MegIS: High-Performance, Energy-Efficient, and Low-Cost Metagenomic Analysis with In-Storage Processing. In *ISCA*, 2024.

[316] Sang-Woo Jun, Huy T. Nguyen, Vijay Gadepally, and Arvind. In-storage Embedded Accelerator for Sparse Pattern Processing. In *HPEC*, 2016.

[317] You-Kai Zheng, Ming-Liang Wei, Hsiang-Yun Cheng, Chia-Lin Yang, Ming-Hsiang Tsai, Chia-Chun Chien, Yuan-Hao Zhong, Po-Hao Tseng, and Hsiang-Pang Li. In-Storage Read-Centric Seed Location Filtering Using 3D-NAND Flash for Genome Sequence Analysis. In *ASPDAC*, 2025.

[318] Ming-Hsiang Tsai, Ming-Liang Wei, Chia-Chun Chien, Po-Hao Tseng, Yung-Chun Lee, Hsiang-Pang Li, and Chia-Lin Yang. Accelerating Genome Alignment Pipeline with In-NAND Search Technology and Group Testing Techniques. In *ICCAD*, 2025.

[319] Rakesh Nadig, Mohammad Sadrosadati, Haiyu Mao, Nika Mansouri Ghiasi, Arash Tavakkol, Jisung Park, Hamid Sarbazi-Azad, Juan Gómez Luna, and Onur Mutlu. Venice: Improving Solid-State Drive Parallelism at Low Cost via Conflict-Free Accesses. In *ISCA*, 2023.

[320] Jiho Kim, Seokwon Kang, Yongjun Park, and John Kim. Networked SSD: Flash Memory Interconnection Network for High-Bandwidth SSD. In *MICRO*, 2022.

[321] Arash Tavakkol, Mohammad Sadrosadati, Saugata Ghose, Jeremie Kim, Yixin Luo, Yaohua Wang, Nika Mansouri Ghiasi, Lois Orosa, Juan Gómez-Luna, and Onur Mutlu. FLIN: Enabling Fairness and Enhancing Performance in Modern NVMe Solid State Drives. In *ISCA*, 2018.

[322] Jaeyong Lee, Hyeunjoo Kim, Sanghun Oh, Myoungjun Chun, Myungsuk Kim, and Jihong Kim. AiF: Accelerating On-Device LLM Inference Using In-Flash Processing. In *ISCA*, 2025.

[323] Peter J. A. Cock, Christopher J. Fields, Naohisa Goto, Michael L. Heuer, and Peter M. Rice. The Sanger FASTQ file format for sequences with quality scores, and the Solexa/Illumina FASTQ variants. *Nucleic Acids Research*, 2009.

[324] Arang Rhie, Sergey Nurk, Monika Cechova, Savannah J. Hoyt, Dylan J. Taylor, Nicolas Altemose, Paul W. Hook, Sergey Koren, Mikko Rautiainen, Ivan A. Alexandrov, Jamie Allen, Mobin Asri, Andrey V. Bzikadze, Nae-Chyun Chen, Chen-Shan Chin, Mark Diekhans, Paul Flicek, Giulio Formenti, Arkarachai Fungtammasan, Carlos Garcia Giron, Erik Garrison, Ariel Gershman, Jennifer L. Gerton, Patrick G. S. Grady, Andrea Guarracino, Leanne Haggerty, Reza Halabian, Nancy F. Hansen, Robert Harris, Gabrielle A. Hartley, William T. Harvey, Marina Haukness, Jakob Heinz, Thibaut Hourlier, Robert M. Hubley, Sarah E. Hunt, Stephen Hwang, Miten Jain, Rupesh K. Kesharwani, Alexandra P. Lewis, Heng Li, Glennis A. Logsdon, Julian K. Lucas, Wojciech Makalowski, Christopher Markovic, Fergal J. Martin, Ann M. Mc Cartney, Rajiv C. McCoy, Jennifer McDaniel, Brandy M. McNulty, Paul Medvedev, Alla Mikheenko, Katherine M. Munson, Terence D. Murphy, Hugh E. Olsen, Nathan D. Olson, Luis F. Paulin, David Porubsky, Tamara Potapova, Fedor Ryabov, Steven L. Salzberg, Michael E. G. Sauria, Fritz J. Sedlazeck, Kishwar Shafin, Valery A. Shepelev, Alaina Shumate, Jessica M. Storer, Likhitha Surapaneni, Angela M. Taravella Oill, Françoise Thibaud-Nissen, Winston Timp, Marta Tomaszkiewicz, Mitchell R. Vollger, Brian P. Walenz, Allison C. Watwood, Matthias H. Weissensteiner, Aaron M. Wenger, Melissa A. Wilson, Samantha Zarate, Yiming Zhu, Justin M. Zook, Evan E. Eichler, Rachel J. O'Neill, Michael C. Schatz, Karen H. Miga, Kateryna D. Makova, and Adam M. Phillippy. The complete sequence of a human Y chromosome. *Nature*, 2023.

[325] Karen H. Miga, Sergey Koren, Arang Rhie, Mitchell R. Vollger, Ariel Gershman, Andrey Bzikadze, Shelise Brooks, Edmund Howe, David Porubsky, Glennis A. Logsdon, Valerie A. Schneider, Tamara Potapova, Jonathan Wood, William Chow, Joel Armstrong, Jeanne Fredrickson, Evgenia Pak, Kristof Tigyi, Milinn Kremitzki, Christopher Markovic, Valerie Maduro, Amalia Dutra, Gerard G. Bouffard, Alexander M. Chang, Nancy F. Hansen, Amy B. Wilfert, Françoise Thibaud-Nissen, Anthony D. Schmitt, Jon-Matthew Belton, Siddarth Selvaraj, Megan Y. Dennis, Daniela C. Soto, Ruta Sahasrabudhe, Gulhan Kaya, Josh Quick, Nicholas J. Loman, Nadine Holmes, Matthew Loose, Urvashi Surti, Rosa ana Risques, Tina A. Graves Lindsay, Robert Fulton, Ira Hall, Benedict Paten, Kerstin Howe, Winston Timp, Alice Young, James C. Mullikin, Pavel A. Pevzner, Jennifer L. Gerton, Beth A. Sullivan, Evan E. Eichler, and Adam M. Phillippy. Telomere-to-telomere Assembly of A Complete Human X Chromosome. *Nature*, 2020.

[326] Nae-Chyun Chen, Luis F. Paulin, Fritz J. Sedlazeck, Sergey Koren, Adam M. Phillippy, and Ben Langmead. Improved sequence mapping using a complete reference genome and lift-over. *Nature Methods*, 2024.

[327] Jouni Sirén, Parsa Eskandar, Matteo Tommaso Ungaro, Glenn Hickey, Jordan M. Eizenga, Adam M. Novak, Xian Chang, Pi-Chuan Chang, Mikhail Kolmogorov, Andrew Carroll, Jean Monlong, and Benedict Paten. Personalized pangenome references. *Nature Methods*, 2024.

[328] Zachary D. Stephens, Skylar Y. Lee, Faraz Faghri, Roy H. Campbell, Chengxiang Zhai, Miles J. Efron, Ravishankar Iyer, Michael C. Schatz, Saurabh Sinha, and Gene E. Robinson. Big Data: Astronomical or Genomical? *PLOS Biology*, 2015.

[329] Isaac Turner, Kiran V Garimella, Zamin Iqbal, and Gil McVean. Integrating long-range connectivity information into de Bruijn graphs. *Bioinformatics*, 2018.

[330] Harun Mustafa, Mikhail Karasikov, Nika Mansouri Ghiasi, Gunnar Rätsch, and André Kahles. Label-guided seed-chain-extend alignment on annotated De Bruijn graphs. *Bioinformatics*, 2024.

[331] Chirag Jain, Sanchit Misra, Haowen Zhang, Alexander Dilthey, and Srinivas Aluru. Accelerating Sequence Alignment to Graphs. In *IPDPS*, 2019.

[332] Shloka Negi, Sarah L. Stenton, Seth I. Berger, Paolo Canigiula, Brandy McNulty, Ivo Violich, Joshua Gardner, Todd Hillaker, Sara M. O'Rourke, Melanie C. O'Leary, Elizabeth Carbonell, Christina Austin-Tse, Gabrielle Lemire, Jillian Serrano, Brian Mangilog, Grace VanNoy, Mikhail Kolmogorov, Eric Vilain, Anne O'Donnell-Luria, Emmanuèle Délot, Karen H. Miga, Jean Monlong, and Benedict Paten. Advancing long-read nanopore genome assembly and accurate variant calling for rare disease detection. *The American Journal of Human Genetics*, 2025.

[333] Pavel A. Pevzner, Haixu Tang, and Michael S. Waterman. An Eulerian path approach to DNA fragment assembly. *Proceedings of the National Academy of Sciences*, 2001.

[334] Harun Mustafa. *Algorithms for efficient sensitive search and sample comparison on petabase-scale genomics data*. PhD thesis, ETH Zurich, 2022.

[335] Andrea Cracco and Alexandru I Tomescu. Extremely fast construction and querying of compacted and colored de Bruijn graphs with GGCAT. *Genome Research*, 2023.

[336] Jasmijn A. Baaijens, Paola Bonizzoni, Christina Boucher, Gianluca Della Vedova, Yuri Pirola, Raffaella Rizzi, and Jouni Sirén. Computational graph pangenomics: a tutorial on data structures and their applications. *Natural Computing*, 2022.

[337] Benedict Paten, Adam M Novak, Jordan M Eizenga, and Erik Garrison. Genome graphs and the evolution of genome inference. *Genome Research*, 2017.

[338] Karel Břinda, Leandro Lima, Simone Pignotti, Natalia Quinones-Olvera, Kamil Salikhov, Rayan Chikhi, Gregory Kucherov, Zamin Iqbal, and Michael Baym. Efficient and robust search of microbial genomes via phylogenetic compression. *Nature Methods*, 2025.

[339] Rayan Chikhi and Guillaume Rizk. Space-efficient and exact de bruijn graph representation based on a bloom filter. *Algorithms for Molecular Biology*, 2013.

[340] Jarno N. Alanko, Simon J. Puglisi, and Jaakko Vuohtoniemi. Small Searchable κ-Spectra via Subset Rank Queries on the Spectral Burrows-Wheeler Transform. In *ACDA*, 2023.

[341] Giulio Ermanno Pibiri. Sparse and skew hashing of k-mers. *Bioinformatics*, 2022.

[342] Alexander Bowe, Taku Onodera, Kunihiko Sadakane, and Tetsuo Shibuya. Succinct de Bruijn graphs. In *WABI*, 2012.

[343] Dinghua Li, Chi-Man Liu, Ruibang Luo, Kunihiko Sadakane, and Tak-Wah Lam. MEGAHIT: an ultra-fast single-node solution for large and complex metagenomics assembly via succinct de Bruijn graph. *Bioinformatics*, 2015.

[344] Mikko Rautiainen and Tobias Marschall. GraphAligner: Rapid and Versatile Sequence-to-Graph Alignment. *Genome Biology*, 2020.

[345] Daehwan Kim, Joseph M Paggi, Chanhee Park, Christopher Bennett, and Steven L Salzberg. Graph-based Genome Alignment and Genotyping with HISAT2 and HISAT-genotype. *Nature Biotechnology*, 2019.

[346] Yan Gao, Yongzhuang Liu, Yanmei Ma, Bo Liu, Yadong Wang, and Yi Xing. abPOA: an SIMD-based C library for fast partial order alignment using adaptive band. *Bioinformatics*, 2020.

[347] Mikko Rautiainen, Veli Mäkinen, and Tobias Marschall. Bit-parallel sequence-to-graph alignment. *Bioinformatics*, 2019.

[348] Ghanshyam Chandra and Chirag Jain. Sequence to Graph Alignment Using Gap-Sensitive Co-linear Chaining. In *RECOMB*, 2023.

[349] Pesho Ivanov, Benjamin Bichsel, and Martin Vechev. Fast and Optimal Sequence-to-Graph Alignment Guided by Seeds. In *RECOMB*, 2022.

[350] Jun Ma, Manuel Cáceres, Leena Salmela, Veli Mäkinen, and Alexandru I Tomescu. Chaining for accurate alignment of erroneous long reads to acyclic variation graphs. *Bioinformatics*, 2023.

[351] Charlotte A Darby, Ravi Gaddipati, Michael C Schatz, and Ben Langmead. Vargas: heuristic-free alignment for assessing linear and graph read aligners. *Bioinformatics*, 2020.

[352] Stephen Hwang, Nathaniel K. Brown, Omar Y. Ahmed, Katharine M. Jenike, Sam Kovaka, Michael C. Schatz, and Ben Langmead. Mem-based pangenome indexing for k-mer queries. *Algorithms for Molecular Biology*, 2025.

[353] Sandra Romain and Claire Lemaitre. SVJedi-graph: improving the genotyping of close and overlapping structural variants with long reads using a variation graph. *Bioinformatics*, 2023.

[354] Heng Li, Xiaowen Feng, and Chong Chu. The design and construction of reference pangenome graphs with minigraph. *Genome Biology*, 2020.

[355] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. Introduction to Algorithms, 2009.

[356] John Hopcroft and Robert Tarjan. Algorithm 447: efficient algorithms for graph manipulation. *Communications of the ACM*, 1973.

[357] Julian Shun and Guy E Blelloch. Ligra: a Lightweight Graph Processing Framework for Shared Memory. In *PPoPP*, 2013.

[358] Duane Merrill, Michael Garland, and Andrew Grimshaw. Scalable GPU graph traversal. In *PPoPP*, 2012.

[359] Yuze Chi, Licheng Guo, and Jason Cong. Accelerating SSSP for power-law graphs. In *FPGA*, 2022.

[360] Vidushi Dadu, Sihao Liu, and Tony Nowatzki. PolyGraph: Exposing the Value of Flexibility for Graph Processing Accelerators. In *ISCA*, 2021.

[361] Pengcheng Yao, Long Zheng, Yu Huang, Qinggang Wang, Chuangyi Gui, Zhen Zeng, Xiaofei Liao, Hai Jin, and Jingling Xue. ScalaGraph: A Scalable Accelerator for Massively Parallel Graph Processing. In *HPCA*, 2022.

[362] Rino Micheloni, Luca Crippa, and Alessia Marelli. *Inside NAND Flash Memories*. Springer Science & Business Media, 2010.

[363] Yu Cai, Saugata Ghose, Erich F. Haratsch, Yixin Luo, and Onur Mutlu. Errors in Flash-Memory-Based Solid-State Drives: Analysis, Mitigation, and Recovery. *arXiv*, 2018.

[364] Yu Cai, Yixin Luo, Saugata Ghose, and Onur Mutlu. Read Disturb Errors in MLC NAND Flash Memory: Characterization, Mitigation, and Recovery. In *IEEE/IFIP DSN*, 2015.

[365] Yu Cai, Saugata Ghose, Yixin Luo, Ken Mai, Onur Mutlu, and Erich F. Haratsch. Vulnerabilities in MLC NAND Flash Memory Programming: Experimental Analysis, Exploits, and Mitigation Techniques. In *HPCA*, 2017.

[366] Yu Cai, Yixin Luo, Erich F Haratsch, Ken Mai, and Onur Mutlu. Data Retention in MLC NAND Flash Memory: Characterization, Optimization, and Recovery. In *HPCA*, 2015.

[367] Arash Tavakkol, Juan Gómez-Luna, Mohammad Sadrosadati, Saugata Ghose, and Onur Mutlu. MQSim: A Framework for Enabling Realistic Studies of Modern Multi-queue SSD Devices. In *FAST*, 2018.

[368] Sungjun Cho, Beomjun Kim, Hyunuk Cho, Gyeongseob Seo, Onur Mutlu, Myungsuk Kim, and Jisung Park. AERO: Adaptive Erase Operation for Improving Lifetime and Performance of Modern NAND Flash-Based SSDs. In *ASPLOS*, 2024.

[369] Jisung Park, Roknoddin Azizi, Geraldo F Oliveira, Mohammad Sadrosadati, Rakesh Nadig, David Novo, Juan Gómez-Luna, Myungsuk Kim, and Onur Mutlu. Flash-Cosmos: In-Flash Bulk Bitwise Operations Using Inherent Computation Capability of NAND Flash Memory. In *MICRO*, 2022.

[370] Yu Cai, Onur Mutlu, Erich F Haratsch, and Ken Mai. Program Interference in MLC NAND Flash Memory: Characterization, Modeling, and Mitigation. In *ICCD*, 2013.

[371] Guiqiang Dong, Ningde Xie, and Tong Zhang. On the Use of Soft-Decision Error-Correction Codes in NAND Flash Memory. *TCAS*, 2010.

[372] Kai Zhao, Wenzhe Zhao, Hongbin Sun, Xiaodong Zhang, Nanning Zheng, and Tong Zhang. LDPC-in-SSD: Making Advanced Error Correction Codes Work Effectively in Solid State Drives. In *FAST*, 2013.

[373] Raj Chandra Bose and Dwijendra K Ray-Chaudhuri. On a Class of Error Correcting Binary Group Codes. *Information and control*, 1960.

[374] Jiadong Wang, Kasra Vakilinia, Tsung-Yi Chen, Thomas Courtade, Guiqiang Dong, Tong Zhang, Hari Shankar, and Richard Wesel. Enhanced Precision through Multiple Reads for LDPC Decoding in Flash Memories. *JSAC*, 2014.

[375] Yixin Luo, Saugata Ghose, Yu Cai, Erich F Haratsch, and Onur Mutlu. Improving 3D NAND Flash Memory Lifetime by Tolerating Early Retention Loss and Process Variation. *ACM POMACS*, 2018.

[376] Yixin Luo, Saugata Ghose, Yu Cai, Erich F. Haratsch, and Onur Mutlu. HeatWatch: Improving 3D NAND Flash Memory Device Reliability by Exploiting Self-recovery and Temperature Awareness. In *HPCA*, 2018.

[377] Yu Cai, Gulay Yalcin, Onur Mutlu, Erich F Haratsch, Adrian Crista, Osman S Unsal, and Ken Mai. Error Analysis and Management for MLC NAND Flash Memory. *Intel Technology*, 2013.

[378] Keonsoo Ha, Jaeyong Jeong, and Jihong Kim. An Integrated Approach for Managing Read Disturbs in High-density NAND Flash Memory. *IEEE TCAD*, 2015.

[379] Yixin Luo, Yu Cai, Saugata Ghose, Jongmoo Choi, and Onur Mutlu. WARM: Improving NAND Flash Memory Lifetime with Write-Hotness Aware Retention Management. In *MSST*, 2015.

[380] Yu Cai, Erich F Haratsch, Onur Mutlu, and Ken Mai. Error Patterns in MLC NAND Flash Memory: Measurement, Characterization, and Analysis. In *DATE*, 2012.

[381] Yu Cai, Gulay Yalcin, Onur Mutlu, Erich F. Haratsch, Adrian Cristal, Osman S. Unsal, and Ken Mai. Flash Correct-and-refresh: Retention-aware Error Management for Increased Flash Memory lifetime. In *ICCD*, 2012.

[382] Samsung. Samsung SSD 860 PRO. `https://www.samsung.com/semiconductor/minisite/ssd/product/consumer/860pro/`, 2018.

[383] AMD. AMD® EPYC® 7742 CPU. `https://www.amd.com/en/products/cpu/amd-epyc-7742`.

[384] Micron Technology Inc. 4Gb: x4, x8, x16 DDR4 SDRAM Data Sheet, 2016.

[385] Samsung. Samsung SSD PM1735. `https://www.samsung.com/semiconductor/ssd/enterprise-ssd/MZPLJ3T2HBJR-00007/`, 2020.

[386] Samsung. Samsung SSD 9100 PRO. `https://semiconductor.samsung.com/consumer-storage/internal-ssd/9100-pro/`, 2025.

[387] European Bioinformatics Institute. European Nucleotide Archive Statistics. `https://www.ebi.ac.uk/ena/browser/about/statistics`, 2023.

[388] Bonnie Berger and Hyunghoon Cho. Emerging technologies towards enhancing privacy in genomic data sharing. *Genome Biology*, 2019.

[389] Michael D Parkins, Bonita E Lee, Nicole Acosta, Maria Bautista, Casey RJ Hubert, Steve E Hrudey, Kevin Frankowski, and Xiao-Li Pang. Wastewater-based surveillance as a tool for public health action: Sars-cov-2 and beyond. *Clinical Microbiology Reviews*, 2024.

[390] Aitor Blanco-Míguez, Francesco Beghini, Fabio Cumbo, Lauren J. McIver, Kelsey N. Thompson, Moreno Zolfo, Paolo Manghi, Leonard Dubois, Kun D. Huang, Andrew Maltez Thomas, William A. Nickols, Gianmarco Piccinno, Elisa Piperni, Michal Punčochář, Mireia Valles-Colomer, Adrian Tett, Francesca Giordano, Richard Davies, Jonathan Wolf, Sarah E. Berry, Tim D. Spector, Eric A. Franzosa, Edoardo Pasolli, Francesco Asnicar, Curtis Huttenhower, and Nicola Segata. Extending and improving metagenomic taxonomic profiling with uncharacterized species using metaphlan 4. *Nature Biotechnology*, 2023.

[391] A. Boroumand, S. Ghose, B. Akin, R. Narayanaswami, G. F. Oliveira, X. Ma, E. Shiu, and O. Mutlu. Google Neural Network Models for Edge Devices: Analyzing and Mitigating Machine Learning Inference Bottlenecks. In *PACT*, September 2021.

[392] Siqi Li, Fengbin Tu, Liu Liu, Jilan Lin, Zheng Wang, Yangwook Kang, Yufei Ding, and Yuan Xie. ECSSD: Hardware/Data Layout Co-Designed In-Storage-Computing Architecture for Extreme Classification. In *ISCA*, 2023.

[393] Vikram Sharma Mailthody, Zaid Qureshi, Weixin Liang, Ziyan Feng, Simon Garcia De Gonzalo, Youjie Li, Hubertus Franke, Jinjun Xiong, Jian Huang, and Wen-mei Hwu. Deepstore: In-storage Acceleration for Intelligent Queries. In *MICRO*, 2019.

[394] Seongyoung Kang, Jiyoung An, Jinpyo Kim, and Sang-Woo Jun. Mithrilog: Near-storage accelerator for high-performance log analytics. In *MICRO*, 2021.

[395] Gunjae Koo, Kiran Kumar Matam, I Te, HV Krishna Giri Narra, Jing Li, Hung-Wei Tseng, Steven Swanson, and Murali Annavaram. Summarizer: Trading Communication with Computing Near Storage. In *MICRO*, 2017.

[396] Hongsun Jang, Jaeyong Song, Jaewon Jung, Jaeyoung Park, Youngsok Kim, and Jinho Lee. Smart-Infinity: Fast Large Language Model Training using Near-Storage Processing on a Real System. In *HPCA*, 2024.

[397] Junkyum Kim, Myeonggu Kang, Yunki Han, Yang-Gon Kim, and Lee-Sup Kim. OptimStore: In-Storage Optimization of Large Scale DNNs with On-Die Processing. In *HPCA*, 2023.

[398] Microchip Technology Incorporated. Flashtec® NVMe® 5106: Performance 16-Channel Gen 5 PCIe® Flash Controller. `https://ww1.microchip.com/downloads/aemDocuments/documents/DCS/ProductDocuments/Brochures/Flashtec-NVMe-5016-Sell-Sheet-00005529.pdf`, 2024.

[399] Jiho Kim, Myoungsoo Jung, and John Kim. Decoupled SSD: Rethinking SSD Architecture through Network-based Flash Controllers. In *ISCA*, 2023.

[400] Arash Tavakkol, Mohammad Arjomand, and Hamid Sarbazi-Azad. Design for scalability in enterprise SSDs. In *PACT*, 2014.

[401] Myungsuk Kim, Jisung Park, Genhee Cho, Yoona Kim, Lois Orosa, Onur Mutlu, and Jihong Kim. Evanesco: Architectural Support for Efficient Data Sanitization in Modern Flash-Based Storage Systems. In *ASPLOS*, 2020.

[402] Jisung Park, Youngdon Jung, Jonghoon Won, Minji Kang, Sungjin Lee, and Jihong Kim. RansomeBlocker: a Low-Overhead Ransomware-Proof SSD. In *DAC*, 2019.

[403] Myungjun Chun, Jaeyong Lee, Sanggu Lee, Myungsuk Kim, and Jihong Kim. PiF: In-flash acceleration for data-intensive applications. In *HotStorage*, 2022.

[404] Yun-Chih Chen, Yuan-Hao Chang, and Tei-Wei Kuo. Search-In-Memory: Reliable, Versatile, and Efficient Data Matching in SSD's NAND Flash Memory Chip for Data Indexing Acceleration. *IEEE TCAD*, 2024.

[405] Jian Chen, Congming Gao, Youyou Lu, Yuhao Zhang, and Jiwu Shu. Ares-Flash: Efficient Parallel Integer Arithmetic Operations Using NAND Flash Memory. In *MICRO*, 2024.

[406] Boncheol Gu, Andre S. Yoon, Duck-Ho Bae, Insoon Jo, Jinyoung Lee, Jonghyun Yoon, Jeong-Uk Kang, Moonsang Kwon, Chanho Yoon, Sangyeun Cho, Jaeheon Jeong, and Duckhyun Chang. Biscuit: A Framework for Near-data Processing of Big Data Workloads. In *ISCA*, 2016.

[407] Yangwook Kang, Yang-suk Kee, Ethan L Miller, and Chanik Park. Enabling Cost-effective Data Processing with Smart SSD. In *MSST*, 2013.

[408] Xiaohao Wang, Yifan Yuan, You Zhou, Chance C Coats, and Jian Huang. Project Almanac: A Time-traveling Solid-state Drive. In *EuroSys*, 2019.

[409] Anurag Acharya, Mustafa Uysal, and Joel Saltz. Active Disks: Programming Model, Algorithms and Evaluation. In *ASPLOS*, 1998.

[410] Kimberly Keeton, David A Patterson, and Joseph M Hellerstein. A Case for Intelligent Disks (IDISKs). *SIGMOD Record*, 1998.

[411] Erik Riedel, Garth Gibson, and Christos Faloutsos. Active Storage for Large-Scale Data Mining and Multimedia Applications. *VLDB*, 1998.

[412] Erik Riedel, Christos Faloutsos, Garth A Gibson, and David Nagle. Active Disks for Large-Scale Data Processing. *Computer*, 2001.

[413] Farnood Merrikh-Bayat, Xinjie Guo, Michael Klachko, Mirko Prezioso, Konstantin K Likharev, and Dmitri B Strukov. High-performance mixed-signal neurocomputing with nanoscale floating-gate memory cell arrays. *IEEE TNNLS*, 2017.

[414] Devesh Tiwari, Simona Boboila, Sudharshan Vazhkudai, Youngjae Kim, Xiaosong Ma, Peter Desnoyers, and Yan Solihin. Active flash: Towards energy-efficient, in-situ data analytics on extreme-scale machines. In *FAST*, 2013.

[415] Devesh Tiwari, Sudharshan S Vazhkudai, Youngjae Kim, Xiaosong Ma, Simona Boboila, and Peter J Desnoyers. Reducing Data Movement Costs Using Energy-Efficient, Active Computation on SSD. In *HotPower*, 2012.

[416] Simona Boboila, Youngjae Kim, Sudharshan S Vazhkudai, Peter Desnoyers, and Galen M Shipman. Active flash: Out-of-core data analytics on flash storage. In *MSST*, 2012.

[417] Duck-Ho Bae, Jin-Hyung Kim, Sang-Wook Kim, Hyunok Oh, and Chanik Park. Intelligent SSD: a turbo for big data mining. In *CIKM*, 2013.

[418] Mahdi Torabzadehkashi, Siavash Rezaei, Vladimir Alves, and Nader Bagherzadeh. Compstor: An in-storage computation platform for scalable distributed processing. In *IPDPSW*, 2018.

[419] Luyi Kang, Yuqi Xue, Weiwei Jia, Xiaohao Wang, Jongryool Kim, Changhwan Youn, Myeong Joon Kang, Hyung Jin Lim, Bruce Jacob, and Jian Huang. Iceclave: A trusted execution environment for in-storage computing. In *MICRO*, 2021.

[420] Chen Zou and Andrew A Chien. ASSASIN: Architecture Support for Stream Computing to Accelerate Computational Storage. In *MICRO*, 2022.

[421] Rakesh Nadig, Vamanan Arulchelvan, Mayank Kabra, Harshita Gupta, Rahul Bera, Nika Mansouri Ghiasi, Nanditha Rao, Qingcai Jiang, Andreas Kosmas Kakolyris, Yu Liang, Mohammad Sadrosadati, and Onur Mutlu. Conduit: Programmer-Transparent Near-Data Processing Using Multiple Compute-Capable Resources in Solid State Drives. In *HPCA*, 2026.

[422] Alessio Campanelli, Giulio Ermanno Pibiri, and Rob Patro. Fast Pseudoalignment Queries on Compressed Colored de Bruijn Graphs. In *WABI*, 2025.

[423] Giulio Ermanno Pibiri, Jason Fan, and Rob Patro. Meta-colored Compacted de Bruijn Graphs. In *RECOMB*, 2024.

[424] Alessio Campanelli, Giulio Ermanno Pibiri, Jason Fan, and Rob Patro. Where the Patterns Are: Repetition-Aware Compression for Colored de Bruijn Graphs. *Journal of Computational Biology*, 2024.

[425] Giulio Ermanno Pibiri and Roberto Trani. PTHash: Revisiting FCH Minimal Perfect Hashing. In *SIGIR*, 2021.

[426] Marek Kokot, Maciej Długosz, and Sebastian Deorowicz. KMC 3: counting and manipulating k-mer statistics. *Bioinformatics*, 2017.

[427] Nika Mansouri Ghiasi, Harun Mustafa, Talu Güloglu, Rakesh Nadig, Konstantina Koliogeorgi, Susana Rebolledo Ruiz, Marc Rautmann, Furkan Eris, Mohammad Sadrosadati, Jisung Park, and Onur Mutlu. GRAINS: Storage-Aware Algorithm-Architecture Co-Design Enabling High-Performance and Low-Cost Graph-Based Genome Analysis. In *arXiv*, 2026.

[428] Micron. Product Flyer: Micron 3D NAND Flash Memory. `https://www.micron.com/-/media/client/global/documents/products/product-flyer/3d_nand_flyer.pdf?la=en`, 2016.

[429] Nika Mansouri Ghiasi, Talu Güloglu, Harun Mustafa, Can Firtina, Konstantina Koliogeorgi, Konstantinos Kanellopoulos, Haiyu Mao, Rakesh Nadig, Mohammad Sadrosadati, Jisung Park, and Onur Mutlu. SAGe: A Lightweight Algorithm-Architecture Co-Design for Mitigating the Data Preparation Bottleneck in Large-Scale Genome Sequence Analysis. In *HPCA*, 2026.

[430] Global Foundries. 22nm CMOS FD-SOI technology. `https://gf.com/technology-platforms/fdx-fd-soi/`.

[431] Synopsys, Inc. Design Compiler. `https://www.synopsys.com/implementation-and-signoff/rtl-synthesis-test/design-compiler-graphical.html`.

[432] Arash Tavakkol, Juan Gómez-Luna, Mohammad Sadrosadati, Saugata Ghose, and Onur Mutlu. MQSim Source Code. `https://github.com/CMU-SAFARI/MQSim`.

[433] Yoongu Kim, Weikun Yang, and Onur Mutlu. Ramulator: A Fast and Extensible DRAM Simulator. *IEEE CAL*, 2015.

[434] Yoongu Kim, Weikun Yang, and Onur Mutlu. Ramulator 1.0 Source Code. `https://github.com/CMU-SAFARI/ramulator`.

[435] Haocong Luo, Yahya Can Tuğrul, F Nisa Bostancı, Ataberk Olgun, A Giray Yağlıkçı, and Onur Mutlu. Ramulator 2.0: A modern, modular, and extensible dram simulator. *IEEE CAL*, 2023.

[436] Haocong Luo, Yahya Can Tuğrul, F Nisa Bostancı, Ataberk Olgun, A Giray Yağlıkçı, and Onur Mutlu. Ramulator 2.0 Source Code. `https://github.com/CMU-SAFARI/ramulator2`.

[437] Samsung. LPDDR4. `https://semiconductor.samsung.com/dram/lpddr/lpddr4/`.

[438] PCI-SIG. PCI Express Base Specification Revision 4.0, Version 1.0. `https://pcisig.com/specifications`.

[439] PCI-SIG. PCI Express Base Specification Revision 5.0, Version 1.0. `https://pcisig.com/PCIExpress/Specs/Base/_5.0_1.0`.

[440] Saugata Ghose, Tianshi Li, Nastaran Hajinazar, Damla Senol Cali, and Onur Mutlu. Demystifying Complex Workload-DRAM Interactions: An Experimental Study. *ACM POMACS*, 2019.

[441] Advanced Micro Devices. AMD® μProf. `https://developer.amd.com/amd-uprof/`, 2021.

[442] Erik Garrison, Jouni Sirén, Adam M Novak, Glenn Hickey, Jordan M Eizenga, Eric T Dawson, William Jones, Shilpa Garg, Charles Markello, Michael F Lin, Benedict Paten, and Richard Durbin. Variation Graph Toolkit Improves Read Mapping by Representing Genetic Variation in the Reference. *Nature Biotechnology*, 2018.

[443] Oxford Nanopore Technologies. MinION Mk1CO. `https://nanoporetech.com/document/requirements/minion-mk1c-spec`, 2024.

[444] Aspyn Palatnick, Bin Zhou, Elodie Ghedin, and Michael C Schatz. iGenomics: Comprehensive DNA sequence analysis on your Smartphone. *GigaScience*, 2020.

[445] Zachary S. Ballard, Calvin Brown, and Aydogan Ozcan. Mobile Technologies for the Discovery, Analysis, and Engineering of the Global Microbiome. *ACS Nano*, 2018.

[446] Josephine B. Oehler, Helen Wright, Zornitza Stark, Andrew J. Mallett, and Ulf Schmitz. The application of long-read sequencing in clinical settings. *Human Genomics*, 2023.

[447] Aaron Pomerantz, Nicolás Peñafiel, Alejandro Arteaga, Lucas Bustamante, Frank Pichardo, Luis A Coloma, César L Barrio-Amorós, David Salazar-Valenzuela, and Stefan Prost. Real-time DNA Barcoding in a Rainforest Using Nanopore Sequencing: Opportunities for Rapid Biodiversity Assessments and Local Capacity Building. *GigaScience*, 2018.

[448] Augusto Dulanto Chiang and John P Dekker. From the Pipeline to the Bedside: Advances and Challenges in Clinical Metagenomics. *The Journal of Infectious Diseases*, 2019.

[449] Aaron Stillmaker and Bevan Baas. Scaling Equations for the Accurate Prediction of CMOS Device Performance from 180 Nm to 7 Nm. *Integration*, 2017.

[450] ARM Holdings. Cortex-R4. `https://developer.arm.com/ip-products/processors/cortex-r/cortex-r4`, 2011.

[451] Tae Jun Ham, Lisa Wu, Narayanan Sundaram, Nadathur Satish, and Margaret Martonosi. Graphicionado: A high-performance and energy-efficient accelerator for graph analytics. In *MICRO*, 2016.

[452] Shafiur Rahman, Nael Abu-Ghazaleh, and Rajiv Gupta. GraphPulse: An Event-Driven Hardware Accelerator for Asynchronous Graph Processing. In *MICRO*, 2020.

[453] Linghao Song, Youwei Zhuo, Xuehai Qian, Hai Li, and Yiran Chen. GraphR: Accelerating Graph Processing Using ReRAM. In *HPCA*, 2018.

[454] Xinyu Chen, Yao Chen, Feng Cheng, Hongshi Tan, Bingsheng He, and Weng-Fai Wong. ReGraph: Scaling Graph Processing on HBM-enabled FPGAs with Heterogeneous Pipelines. In *MICRO*, 2022.

[455] Jiaqi Yang, Hao Zheng, and Ahmed Louri. I-DGNN: A Graph Dissimilarity-based Framework for Designing Scalable and Efficient DGNN Accelerators. In *HPCA*, 2025.

[456] Jiale Yan, Hiroaki Ito, Yuta Nagahara, Kazushi Kawamura, Masato Motomura, Thiem Van Chu, and Daichi Fujiki. BingoGCN: Towards Scalable and Efficient GNN Acceleration with Fine-Grained Partitioning and SLT. In *ISCA*, 2025.

[457] Hongwu Peng, Xi Xie, Kaustubh Shivdikar, Md Amit Hasan, Jiahui Zhao, Shaoyi Huang, Omer Khan, David Kaeli, and Caiwen Ding. MaxK-GNN: Extremely Fast GPU Kernel Design for Accelerating Graph Neural Networks Training. In *ASPLOS*, 2024.

[458] Mingyu Yan, Lei Deng, Xing Hu, Ling Liang, Yujing Feng, Xiaochun Ye, Zhimin Zhang, Dongrui Fan, and Yuan Xie. HyGCN: A GCN Accelerator with Hybrid Architecture. In *HPCA*, 2020.

[459] Haoran You, Tong Geng, Yongan Zhang, Ang Li, and Yingyan Lin. GCoD: Graph Convolutional Network Acceleration via Dedicated Algorithm and Accelerator Co-Design. In *HPCA*, 2022.

[460] Ranggi Hwang, Minhoo Kang, Jiwon Lee, Dongyun Kam, Youngjoo Lee, and Minsoo Rhu. GROW: A Row-Stationary Sparse-Dense GEMM Accelerator for Memory-Efficient Graph Convolutional Neural Networks. In *HPCA*, 2023.

[461] Rishov Sarkar, Stefan Abi-Karam, Yuqi He, Lakshmi Sathidevi, and Cong Hao. FlowGNN: A Dataflow Architecture for Real-Time Workload-Agnostic Graph Neural Network Inference. In *HPCA*, 2023.

[462] Cen Chen, Kenli Li, Yangfan Li, and Xiaofeng Zou. ReGNN: A Redundancy-Eliminated Graph Neural Networks Accelerator. In *HPCA*, 2022.

[463] Jiajun Li, Ahmed Louri, Avinash Karanth, and Razvan Bunescu. GCNAX: A Flexible and Energy-efficient Accelerator for Graph Convolutional Neural Networks. In *HPCA*, 2021.

[464] Tong Geng, Ang Li, Runbin Shi, Chunshu Wu, Tianqi Wang, Yanfei Li, Pouya Haghi, Antonino Tumeo, Shuai Che, Steve Reinhardt, and Martin C. Herbordt. AWB-GCN: A Graph Convolutional Network Accelerator with Runtime Workload Rebalancing. In *MICRO*, 2020.

[465] Dan Chen, Haiheng He, Hai Jin, Long Zheng, Yu Huang, Xinyang Shen, and Xiaofei Liao. MetaNMP: Leveraging Cartesian-Like Product to Accelerate HGNNs with Near-Memory Processing. In *ISCA*, 2023.

[466] Wenju Zhao, Pengcheng Yao, Dan Chen, Long Zheng, Xiaofei Liao, Qinggang Wang, Shaobo Ma, Yu Li, Haifeng Liu, Wenjing Xiao, Yufei Sun, Bing Zhu, Hai Jin, and Jingling Xue. MeHyper: Accelerating Hypergraph Neural Networks by Exploring Implicit Dataflows. In *HPCA*, 2025.

[467] Junwhan Ahn, Sungpack Hong, Sungjoo Yoo, Onur Mutlu, and Kiyoung Choi. A scalable processing-in-memory accelerator for parallel graph processing. In *ISCA*, 2015.

[468] Zhe Zhou, Cong Li, Xuechao Wei, Xiaoyang Wang, and Guangyu Sun. GNNear: Accelerating Full-Batch Training of Graph Neural Networks with near-Memory Processing. In *PACT*, 2022.

[469] Mingxing Zhang, Youwei Zhuo, Chao Wang, Mingyu Gao, Yongwei Wu, Kang Chen, Christos Kozyrakis, and Xuehai Qian. GraphP: Reducing Communication for PIM-based Graph Processing with Efficient Data Partition. In *HPCA*, 2018.

[470] Mikhail Asiatici and Paolo Ienne. Large-Scale Graph Processing on FPGAs with Caches for Thousands of Simultaneous Misses. In *ISCA*, 2021.

[471] Changmin Shin, Jaeyong Song, Hongsun Jang, Dogeun Kim, Jun Sung, Taehee Kwon, Jae Hyung Ju, Frank Liu, Yeonkyu Choi, and Jinho Lee. Piccolo: Large-Scale Graph Processing with Fine-Grained in-Memory Scatter-Gather. In *HPCA*, 2025.

[472] Yu Huang, Long Zheng, Pengcheng Yao, Qinggang Wang, Xiaofei Liao, Hai Jin, and Jingling Xue. Accelerating Graph Convolutional Networks Using Crossbar-based Processing-In-Memory Architectures. In *HPCA*, 2022.

[473] Maciej Besta, Raghavendra Kanakagiri, Grzegorz Kwasniewski, Rachata Ausavarungnirun, Jakub Beránek, Konstantinos Kanellopoulos, Kacper Janda, Zur Vonarburg-Shmaria, Lukas Gianinazzi, Ioana Stefan, Juan Gómez Luna, Jakub Golinowski, Marcin Copik, Lukas Kapp-Schwoerer, Salvatore Di Girolamo, Nils Blach, Marek Konieczny, Onur Mutlu, and Torsten Hoefler. SISA: Set-Centric Instruction Set Architecture for Graph Mining on Processing-in-Memory Systems. In *MICRO*, 2021.

[474] Shuangchen Li, Dimin Niu, Yuhao Wang, Wei Han, Zhe Zhang, Tianchan Guan, Yijin Guan, Heng Liu, Linyong Huang, Zhaoyang Du, Fei Xue, Yuanwei Fang, Hongzhong Zheng, and Yuan Xie. Hyperscale FPGA-as-a-service architecture for large-scale distributed graph neural network. In *ISCA*, 2022.

[475] Shu-Ting Wang, Hanyang Xu, Amin Mamandipoor, Rohan Mahapatra, Byung Hoon Ahn, Soroush Ghodrati, Krishnan Kailas, Mohammad Alian, and Hadi Esmaeilzadeh. Data Motion Acceleration: Chaining Cross-Domain Multi Accelerators. In *HPCA*, 2024.

[476] Yue Dai, Youtao Zhang, and Xulong Tang. CEGMA: Coordinated Elastic Graph Matching Acceleration for Graph Matching Networks. In *HPCA*, 2023.

[477] Taehwan Kim, Yunki Han, Seohye Ha, Jiwan Kim, and Lee-Sup Kim. EOD: Enabling Low Latency GNN Inference via Near-Memory Concatenate Aggregation. In *ISCA*, 2025.

[478] Yi Li, Tsun-Yu Yang, Ming-Chang Yang, Zhaoyan Shen, and Bingzhe Li. Celeritas: Out-of-Core Based Unsupervised Graph Neural Network via Cross-Layer Computing. In *HPCA*, 2024.

[479] Junwhan Ahn, Sungjoo Yoo, Onur Mutlu, and Kiyoung Choi. PIM-Enabled Instructions: A Low-Overhead, Locality-Aware Processing-in-Memory Architecture. In *ISCA*, 2015.

[480] Konstantinos Kanellopoulos, Nandita Vijaykumar, Christina Giannoula, Roknoddin Azizi, Skanda Koppula, Nika Mansouri Ghiasi, Taha Shahroodi, Juan Gomez Luna, and Onur Mutlu. SMASH: Co-designing Software Compression and Hardware-Accelerated Indexing for Efficient Sparse Matrix Operations. In *MICRO*, 2019.

[481] Guohao Dai, Tianhao Huang, Yuze Chi, Jishen Zhao, Guangyu Sun, Yongpan Liu, Yu Wang, Yuan Xie, and Huazhong Yang. GraphH: A Processing-in-memory Architecture for Large-Scale Graph Processing. *IEEE TCAD*, 2018.

[482] Hasindu Gamaarachchi, Chun Wai Lam, Gihan Jayatilaka, Hiruna Samarakoon, Jared T Simpson, Martin A Smith, and Sri Parameswaran. GPU accelerated adaptive banded event alignment for rapid comparative nanopore signal analysis. *BMC Bioinformatics*, 2020.

[483] Kisaru Liyanage, Hiruna Samarakoon, Sri Parameswaran, and Hasindu Gamaarachchi. Efficient end-to-end long-read sequence mapping using minimap2-fpga integrated with hardware-accelerated chaining. *Scientific Reports*, 2023.

[484] Wenqin Huangfu, Krishna T. Malladi, Andrew Chang, and Yuan Xie. BEACON: Scalable Near-Data-Processing Accelerators for Genome Analysis near Memory Pool with the CXL Support. In *MICRO*, 2022.

[485] Wenqin Huangfu, Xueqi Li, Shuangchen Li, Xing Hu, Peng Gu, and Yuan Xie. MEDAL: Scalable DIMM based Near Data Processing Accelerator for DNA Seeding Algorithm. In *MICRO*, 2019.

[486] Joël Lindegger, Damla Senol Cali, Mohammed Alser, Juan Gómez-Luna, Nika Mansouri Ghiasi, and Onur Mutlu. Scrooge: a fast and memory-frugal genomic sequence aligner for CPUs, GPUs, and ASICs. *Bioinformatics*, 2023.

[487] Safaa Diab, Amir Nassereldine, Mohammed Alser, Juan Gómez Luna, Onur Mutlu, and Izzat El Hajj. A framework for high-throughput sequence alignment using real processing-in-memory systems. *Bioinformatics*, 2023.

[488] Safaa Diab, Amir Nassereldine, Mohammed Alser, Juan Gómez-Luna, Onur Mutlu, and Izzat El Hajj. A framework for high-throughput sequence alignment using real processing-in-memory systems. *arXiv*, 2022.

[489] Jialei Sun, Lingyi Liu, Kunyue Li, Shuaipeng Li, Sai Gao, Zizheng Dong, Jianfei Jiang, and Fangzhen Wu. An Energy Efficient CPU-FPGA Heterogeneous Acceleration System for Third-Generation Genomic Sequencing Based on Minimap2. *IEEE TCAS-II*, 2025.

[490] Hamza E. Barkam, Sanggeon Yun, Paul R. Genssler, Che-Kai Liu, Zhuowen Zou, Hussam Amrouch, and Mohsen Imani. In-Memory Acceleration of Hyperdimensional Genome Matching on Unreliable Emerging Technologies. *IEEE TCAS-I*, 2024.

[491] William Andrew Simon, Irem Boybat, Riselda Kodra, Elena Ferro, Gagandeep Singh, Mohammed Alser, Shubham Jain, Hsinyu Tsai, Geoffrey W. Burr, Onur Mutlu, and Abu Sebastian. CiMBA: Accelerating Genome Sequencing Through On-Device Basecalling via Compute-in-Memory. *IEEE TPDS*, 2025.

[492] Juechu Dong, Xueshen Liu, Harisankar Sadasivan, Sriranjani Sitaraman, and Satish Narayanasamy. mm2-gb: GPU accelerated minimap2 for long read dna mapping. In *BCB*, 2024.

[493] Florestan De Moor, Meven Mognol, Charles Deltel, Erwan Drezen, Julien Legriel, and Dominique Lavenier. MiMyCS: A Processing-in-Memory Read Mapper for Compressing Next-Gen Sequencing Datasets. In *BIBM*, 2024.

[494] Yi-Chung Wu, Yen-Lung Chen, Chung-Hsuan Yang, Chao-Hsi Lee, Wen-Ching Chen, Liang-Yi Lin, Nian-Shyang Chang, Chun-Pin Lin, Chi-Shi Chen, Jui-Hung Hung, and Chia-Hsiang Yang. A 28nm Fully Integrated End-to-End Genome Analysis Accelerator for Next-Generation Sequencing. *IEEE TBioCAS*, 2025.

[495] Mohammed Alser, Zülal Bingöl, Damla Senol Cali, Jeremie Kim, Saugata Ghose, Can Alkan, and Onur Mutlu. Accelerating Genome Analysis: A Primer on an Ongoing Journey. *IEEE Micro*, 2020.

[496] Gagandeep Singh, Mohammed Alser, Damla Senol Cali, Dionysios Diamantopoulos, Juan Gómez-Luna, Henk Corporaal, and Onur Mutlu. FPGA-based Near-Memory Acceleration of Modern Data-Intensive Applications. *IEEE Micro*, 2021.

[497] Gagandeep Singh, Mohammed Alser, Kristof Denolf, Can Firtina, Alireza Khodamoradi, Meryem Banu Cavlak, Henk Corporaal, and Onur Mutlu. RUBICON: a framework for designing efficient deep learning-based genomic basecallers. *Genome Biology*, 2024.

[498] Julien Eudine, Chu Li, Zhuo Cheng, Renzo Andri, Can Firtina, Mohammad Sadrosadati, Nika Mansouri Ghiasi, Konstantina Koliogeorgi, Anirban Nag, Arash Tavakkol, Haiyu Mao, Onur Mutlu, Shai Bergman, and Ji Zhang. GenPairX: A Hardware-Algorithm Co-Designed Accelerator for Paired-End Read Mapping. In *HPCA*, 2026.

[499] Julian Pavon, Ivan Vargas Valdivieso, Carlos Rojas, Cesar Hernandez, Mehmet Aslan, Roger Figueras, Yichao Yuan, Joël Lindegger, Mohammed Alser, Francesc Moll, Santiago Marco-Sola, Oguz Ergin, Nishil Talati, Onur Mutlu, Osman Unsal, Mateo Valero, and Adrian Cristal. QUETZAL: Vector Acceleration Framework for Modern Genome Sequence Analysis Algorithms. In *ISCA*, 2024.

[500] Mohammed Alser, Taha Shahroodi, Juan Gómez-Luna, Can Alkan, and Onur Mutlu. SneakySnake: A Fast and Accurate Universal Genome Pre-alignment Filter for CPUs, GPUs and FPGAs. *Bioinformatics*, 2020.

[501] Mohammed Alser, Hasan Hassan, Hongyi Xin, Oğuz Ergin, Onur Mutlu, and Can Alkan. GateKeeper: A New Hardware Architecture for Accelerating Pre-alignment in DNA Short Read Mapping. *Bioinformatics*, 2017.

[502] Zülal Bingöl, Mohammed Alser, Onur Mutlu, Ozcan Ozturk, and Can Alkan. GateKeeper-GPU: Fast and Accurate Pre-Alignment Filtering in Short Read Mapping. In *IPDPSW*, 2021.

[503] Nika Mansouri Ghiasi. *Storage-Centric System Designs for Enabling Fast, Efficient, and Low-Cost Genomic and Metagenomic Analyses.* PhD thesis, ETH Zurich, 2026.

[504] Mohammed Alser, Hasan Hassan, Akash Kumar, Onur Mutlu, and Can Alkan. Shouji: A Fast and Efficient Pre-alignment Filter for Sequence Alignment. *Bioinformatics*, 2019.

[505] Alejandro Alonso-Marín, Ivan Fernandez, Quim Aguado-Puig, Juan Gómez-Luna, Santiago Marco-Sola, Onur Mutlu, and Miquel Moreto. BIMSA: Accelerating Long Sequence Alignment Using Processing-In-Memory. *Bioinformatics*, 2024.

[506] Can Firtina, Kamlesh Pillai, Gurpreet S. Kalsi, Bharathwaj Suresh, Damla Senol Cali, Jeremie S. Kim, Taha Shahroodi, Meryem Banu Cavlak, Joël Lindegger, Mohammed Alser, Juan Gómez Luna, Sreenivas Subramoney, and Onur Mutlu. ApHMM: Accelerating Profile Hidden Markov Models for Fast and Energy-efficient Genome Analysis. *TACO*, 2024.

[507] Can Firtina. Enabling Fast, Accurate, and Efficient Real-Time Genome Analysis via New Algorithms and Techniques. *arXiv*, 2025.

[508] Damla Senol. *Accelerating Genome Sequence Analysis via Efficient Hardware/Algorithm Co-Design.* PhD thesis, Carnegie Mellon University, 2021.

[509] Konstantina Koliogeorgi. *Hardware acceleration techniques for computation and data intensive machine learning and bioinformatic applications.* PhD thesis, NTUA, 2023.

[510] Max Doblas, Oscar Lostes-Cazorla, Quim Aguado-Puig, Nick Cebry, Pau Fontova-Musté, Christopher Frances Batten, Santiago Marco-Sola, and Miquel Moretó. GMX: Instruction Set Extensions for Fast, Scalable, and Efficient Genome Sequence Alignment. In *MICRO*, 2023.

[511] Anirban Nag, CN Ramachandra, Rajeev Balasubramonian, Ryan Stutsman, Edouard

Giacomin, Hari Kambalasubramanyam, and Pierre-Emmanuel Gaillardon. GenCache: Leveraging In-cache Operators for Efficient Sequence Alignment. In *MICRO*, 2019.
[512] Jarno N Alanko, Elena Biagi, Joel Mackenzie, and Simon J Puglisi. Batched-mer lookup on the Spectral Burrows-Wheeler Transform. In *ALENEX*, 2025.
[513] Jing Zhang, Heshan Lin, Pavan Balaji, and Wu-chun Feng. Optimizing Burrows-Wheeler Transform-Based Sequence Alignment on Multicore Architectures. In *CCGRID*, 2013.
[514] Yifan Li and Giulia Guidi. High-Performance Sorting-Based K-mer Counting in Distributed Memory with Flexible Hybrid Parallelism. In *ICPP*, 2024.
[515] Muhammad Adnan, Yassaman Ebrahimzadeh Maboud, Divya Mahajan, and Prashant J. Nair. Heterogeneous Acceleration Pipeline for Recommendation System Training. In *ISCA*, 2024.
[516] Li Peng, Wenbo Wu, Shushu Yi, Xianzhang Chen, Chenxi Wang, Shengwen Liang, Zhe Wang, Nong Xiao, Qiao Li, Mingzhe Zhang, and Jie Zhang. XHarvest: Rethinking High-Performance and Cost-Efficient SSD Architecture with CXL-Driven Harvesting. In *ISCA*, 2025.
[517] Chen Tang, Xinyuan Lin, Zongle Huang, Wenyu Sun, Hongyang Jia, and Yongpan Liu. A 28nm 4.35TOPS/mm2 Transformer Accelerator with Basis-vector Based Ultra Storage Compression, Decomposed Computation and Unified LUT-Assisted Cores. In *VLSI Technology and Circuits*, 2024.
[518] Shengwen Liang, Ying Wang, Cheng Liu, Huawei Li, and Xiaowei Li. InS-DLA: An In-SSD deep learning accelerator for near-data processing. In *FPL*, 2019.
[519] Kangqi Chen, Rakesh Nadig, Manos Frouzakis, Nika Mansouri Ghiasi, Yu Liang, Haiyu Mao, Jisung Park, Mohammad Sadrosadati, and Onur Mutlu. REIS: A High-Performance and Energy-Efficient Retrieval System with In-Storage Processing. In *ISCA*, 2025.
[520] Zhongkai Yu, Shengwen Liang, Tianyun Ma, Yunke Cai, Ziyuan Nan, Di Huang, Xinkai Song, Yifan Hao, Jie Zhang, Tian Zhi, Yongwei Zhao, Zidong Du, Xing Hu, Qi Guo, and Tianshi Chen. Cambricon-LLM: A Chiplet-Based Hybrid Architecture for On-Device Inference of 70B LLM. In *MICRO*, 2024.
[521] Weiyi Sun, Mingyu Gao, Zhaoshi Li, Aoyang Zhang, Iris Ying Chou, Jianfeng Zhu, Shaojun Wei, and Leibo Liu. Lincoln: Real-Time 50 100B LLM Inference on Consumer Devices with LPDDR-Interfaced, Compute-Enabled Flash Memory. In *HPCA*, 2025.
[522] Raúl Taranco, José-Maria Arnau, and Antonio González. IRIS: Unleashing ISP-Software Cooperation to Optimize the Machine Vision Pipeline. In *HPCA*, 2025.
[523] Xiurui Pan, Endian Li, Qiao Li, Shengwen Liang, Yizhou Shan, Ke Zhou, Yingwei Luo, Xiaolin Wang, and Jie Zhang. InstAttention: In-Storage Attention Offloading for Cost-Effective Long-Context LLM Inference. In *HPCA*, 2025.
[524] Po-Kai Hsu, Vaidehi Garg, Anni Lu, and Shimeng Yu. A Heterogeneous Platform for 3D NAND-Based In-Memory Hyperdimensional Computing Engine for Genome Sequencing Applications. *IEEE TCAS-I*, 2024.
[525] Rohan Mahapatra, Harsha Santhanam, Christopher Priebe, Hanyang Xu, and Hadi Esmaeilzadeh. In-Storage Acceleration of Retrieval Augmented Generation as a Service. In *ISCA*, 2025.
[526] Shengwen Liang, Ying Wang, Youyou Lu, Zhe Yang, Huawei Li, and Xiaowei Li. Cognitive SSD: A Deep Learning Engine for In-Storage Data Retrieval. In *USENIX ATC*, 2019.
[527] Minsub Kim and Sungjin Lee. Reducing tail latency of dnn-based recommender systems using in-storage processing. In *APSys*, 2020.
[528] Minje Lim, Jeeyoon Jung, and Dongkun Shin. Lsm-tree compaction acceleration using in-storage processing. In *ICCE-Asia*, 2021.
[529] Jianguo Wang, Dongchul Park, Yang-Suk Kee, Yannis Papakonstantinou, and Steven Swanson. SSD in-storage computing for list intersection. In *DaMoN*, 2016.
[530] Sung-Tae Lee and Jong-Ho Lee. Neuromorphic computing using NAND flash memory architecture with pulse width modulation scheme. *Frontiers in Neuroscience*, 2020.
[531] Myeonggu Kang, Hyeonuk Kim, Hyein Shin, Jaehyeong Sim, Kyeonghan Kim, and Lee-Sup Kim. S-FLASH: A NAND flash-based deep neural network accelerator exploiting bit-level sparsity. *IEEE TC*, 2021.
[532] Runze Han, Yachen Xiang, Peng Huang, Yihao Shan, Xiaoyan Liu, and Jinfeng Kang. Flash memory array for efficient implementation of deep neural networks. *Advanced Intelligent Systems*, 2021.
[533] Shaodi Wang. MemCore: Computing-in-Flash Design for Deep Neural Network Acceleration. In *EDTM*, 2022.
[534] Panni Wang, Feng Xu, Bo Wang, Bin Gao, Huaqiang Wu, He Qian, and Shimeng Yu. Three-dimensional NAND flash for vector–matrix multiplication. *IEEE TVLSI*, 2018.
[535] Runze Han, Peng Huang, Yachen Xiang, Chen Liu, Zhen Dong, Zhiqiang Su, Yongbo Liu, Lu Liu, Xiaoyan Liu, and Jinfeng Kang. A novel convolution computing paradigm based on NOR flash array with high computing speed and energy efficiency. *IEEE TCAS-I*, 2019.
[536] Won Ho Choi, Pi-Feng Chiu, Wen Ma, Gertjan Hemink, Tung Thanh Hoang, Martin Lueker-Boden, and Zvonimir Bandic. An in-flash binary neural network accelerator with SLC NAND flash array. In *ISCAS*, 2020.
[537] Shuyi Pei, Jing Yang, and Qing Yang. REGISTOR: A Platform for Unstructured Data Processing inside SSD Storage. *ACM TOS*, 2019.
[538] Jaeyoung Do, Yang-Suk Kee, Jignesh M Patel, Chanik Park, Kwanghyun Park, and David J DeWitt. Query Processing on Smart SSDs: Opportunities and Challenges. In *ACM SIGMOD*, 2013.
[539] Sudharsan Seshadri, Mark Gahagan, Sundaram Bhaskaran, Trevor Bunker, Arup De, Yanqin Jin, Yang Liu, and Steven Swanson. Willow: A User-Programmable SSD. In *USENIX OSDI*, 2014.
[540] Sungchan Kim, Hyunok Oh, Chanik Park, Sangyeun Cho, Sang-Won Lee, and Bongki Moon. In-storage Processing of Database Scans and Joins. *Information Sciences*, 2016.
[541] Won Seob Jeong, Changmin Lee, Keunsoo Kim, Myung Kuk Yoon, Won Jeon, Myoungsoo Jung, and Won Woo Ro. REACT: Scalable and High-performance Regular Expression Pattern Matching Accelerator for In-storage Processing. *IEEE TPDS*, 2019.
[542] Carl Duffy, Jaehoon Shim, Sang-Hoon Kim, and Jin-Soo Kim. Dotori: A Key-Value SSD Based KV Store. *Proceedings of the VLDB Endowment*, 2023.
[543] Chanyoung Park, Jungho Lee, Chun-Yi Liu, Kyungtae Kang, Mahmut Taylan Kandemir, and Wonil Choi. AnyKey: A Key-Value SSD for All Workload Types. In *ASPLOS*, 2025.
[544] Ryan Wong, Nikita Kim, Aniket Das, Kevin Higgs, Engin Ipek, Sapan Agarwal, Saugata Ghose, and Ben Feinberg. ANVIL: An In-Storage Accelerator for Name–Value Data Stores. In *ISCA*, 2025.
[545] Rohan Mahapatra, Soroush Ghodrati, Byung Hoon Ahn, Sean Kinzer, Shu-Ting Wang, Hanyang Xu, Lavanya Karthikeyan, Hardik Sharma, Amir Yazdanbakhsh, Mohammad Alian, and Hadi Esmaeilzadeh. In-storage domain-specific acceleration for serverless computing. In *ASPLOS*, 2024.
[546] Nika Mansouri Ghiasi, Talu Güloglu, Harun Mustafa, Can Firtina, Konstantina Koliogeorgi, Konstantinos Kanellopoulos, Haiyu Mao, Rakesh Nadig, Mohammad Sadrosadati, Jisung Park, and Onur Mutlu. SAGe: A Lightweight Algorithm-Architecture Co-Design for Mitigating the Data Preparation Bottleneck in Large-Scale Genome Sequence Analysis. *arXiv*, 2026.
[547] Sang-Woo Jun, Ming Liu, Sungjin Lee, Jamey Hicks, John Ankcorn, Myron King, Shuotao Xu, and Arvind. Bluedbm: An Appliance for Big Data Analytics. In *ISCA*, 2015.
[548] Sang-Woo Jun, Ming Liu, Sungjin Lee, Jamey Hicks, John Ankcorn, Myron King, and Shuotao Xu. Bluedbm: Distributed Flash Storage for Big Data Analytics. *ACM TOCS*, 2016.
[549] Mahdi Torabzadehkashi, Siavash Rezaei, Ali Heydarigorji, Hosein Bobarshad, Vladimir Alves, and Nader Bagherzadeh. Catalina: In-storage Processing Acceleration for Scalable Big Data Analytics. In *Euromicro PDP*, 2019.
[550] Joo Hwan Lee, Hui Zhang, Veronica Lagrange, Praveen Krishnamoorthy, Xiaodong Zhao, and Yang Seok Ki. SmartSSD: FPGA Accelerated Near-Storage Data Analytics on SSD. *IEEE CAL*, 2020.
[551] Mohammadamin Ajdari, Pyeongsu Park, Joonsung Kim, Dongup Kwon, and Jangwoo Kim. CIDR: A Cost-effective In-line Data Reduction System for Terabit-per-second Scale SSD Arrays. In *HPCA*, 2019.
[552] Ipoom Jeong, Jinghan Huang, Chuxuan Hu, Dohyun Park, Jaeyoung Kang, Nam Sung Kim, and Yongjoo Park. UPP: Universal Predicate Pushdown to Smart Storage. In *ISCA*, 2025.
[553] Benjamin Y Cho, Won Seob Jeong, Doohwan Oh, and Won Woo Ro. XSD: Accelerating MapReduce by Harnessing the GPU inside an SSD. In *Near-Data Processing*, 2013.
[554] Mincheol Kang, Wonyoung Lee, Jinkwon Kim, and Soontae Kim. PR-SSD: Maximizing partial read potential by exploiting compression and channel-level parallelism. *IEEE TC*, 2022.
[555] Jeongho Lee, Sangjun Kim, Jaeyong Lee, Jaeyoung Kang, Sungjin Lee, Nam Sung Kim, and Jihong Kim. srNAND: A Novel NAND Flash Organization for Enhanced Small Read Throughput in SSDs. *IEEE CAL*, 2025.
[556] Congming Gao, Xin Xin, Youyou Lu, Youtao Zhang, Jun Yang, and Jiwu Shu. ParaBit: Processing Parallel Bitwise Operations in NAND Flash Memory Based SSDs. In *MICRO*, 2021.
[557] Myoungjun Chun, Jaeyong Lee, Myungsuk Kim, Jisung Park, and Jihong Kim. Rif: Improving read performance of modern ssds using an on-die early-retry engine. In *HPCA*, 2024.
[558] Shengwen Liang, Ziming Yuan, Ying Wang, Dawen Xu, Huawei Li, and Xiaowei Li. HyQA: Hybrid Near-Data Processing Platform for Embedding Based Question Answering System. In *DATE*, 2024.